CMDBUF RD 完整说明手册

设计 时序 注错 GUI 操作 验证与迁移

文档版本 2026-10-13　代码基线 1949e75

本文用于设计评审、RTL 集成、验证复核和实际 GUI 操作。项目说明以 PDF 与 Word 交付：PDF 用于阅读与打印，Word 保留可编辑的正文、表格和图片。

当前芯片 RTL 为 Verilog-2001。SBY 已实际完成完整求解；当前设备未安装真实 FormalMC，模型交叉验证与能力入口不能当作 FormalMC 实机证明。

历史设计建议、阶段报告和截图均保留所属版本。正文中的历史 pending、工具缺失或单任务结果，不改写为当前完整验证 PASS。

阅读方式

按主题阅读下列分册，或使用 PDF 书签和 Word 导航窗格。时序图采用横向页面。GUI 截图保持真实像素，并提供关键操作区域的放大视图；放大仅改变文档中的图片视口。

01 项目使用与部署说明

02 CMDBUF RD 详细设计方案

03 CMDBUF RD 时序图集

04 CMDBUF RD 注错与容量保护清单

05 UCAgent Tk GUI 详细操作说明

06 CMDBUF RD Formal 验证说明

07 CMDBUF RD 仿真基线与历史记录

现有 Markdown 和 HTML 是可追溯的内容源与历史阅读页，不是新增说明的最终交付格式。代码、约束、机器可读结果和工具自带第三方文档保持各自原格式。

项目使用与部署说明

内容来源 README.md

UCAgent CMDBUF RD RTL 形式验证与 Tk GUI

本仓库保存当前已对齐的 `cmdbuf_rd` 项目：9 项 CMDBUF/OTB、3 个 XAU、最老 5 条未成功发射命令窗口、三周期前向流水、MEM 成功发射后四拍返回，以及每个 XAU 保序的 RR 响应。

项目说明统一提供 **PDF 阅读版和 Word 编辑版**。完整资料见 完整说明手册 PDF / 完整说明手册 Word，各主题分册见下表。原 Markdown、HTML 和历史证据作为维护及追溯来源保留。

说明内容	PDF	Word
项目使用、RTL 集成与服务部署	阅读	编辑
详细 RTL 设计方案与架构	阅读	编辑
13 个已对齐时序场景	阅读	编辑
49 组注错与容量保护	阅读	编辑
Tk GUI 操作图示与放大视图	阅读	编辑
完整 Formal 流程、177 CK 与验证结果	阅读	编辑
仿真基线与历史阶段记录	阅读	编辑

当前芯片 RTL 使用 **Verilog-2001**，入口为 cmdbuf_rd.v、cmdbuf_rd_cmd.v 和 cmdbuf_rd_otb.v。集成与 Bank 状态位序见 RTL 使用说明 PDF。形式验证属性与观测器继续使用验证工具支持的 SystemVerilog。

操作界面为 **Python Tkinter/ttk 原生 GUI**。在 UCAgent 已关联的 SBY 任务中点击“切换到 FormalMC”，即可进入完整项目的 FormalMC 操作页；两种后端独立保存配置和结果。HTML 文件是报告与文档。

目录

目录	内容
rtl/	当前三份 RTL，与封存验证包的 RTL 字节相同
formal/cmdbuf_rd_dual_backend/	可迁移的 SBY / FormalMC 验证包、22 个任务、177 个检查点与历史验证证据
ucagent-tool/	已更新的 UCAgent 桌面与服务完整源码，保留上游 Apache-2.0 许可证
gui/	Tk 桌面 zipapp 和当前项目启动入口
docs/design/	详细设计方案与架构图
docs/timing/	已对齐时序图、5A / 6B / 10B 决策
docs/fault_catalog/	注错清单及容量保底场景
docs/gui/	15 张真实 Tk 截图和操作说明的维护来源
documents/	全部项目说明的 PDF / Word 完整手册与主题分册
simulation/	早期 RTL 仿真工程、独立参考模型与历史结果
verification/closure/	完整 UCAgent 流程报告与原始证据分卷
verification/gui/	Tk GUI 回归日志、实际界面截图与验收记录
verification/verilog_20261012/	本次 Verilog 转换、新旧等价性与重新验证的证据
provenance/	GUI 变更及交付来源说明

运行 SBY FormalMC

在有对应工具的 Linux 设备上：

```
cd formal/cmdbuf_rd_dual_backend
python3 run.py --list
python3 run.py --backend sby --tool /your/oss-cad-suite/bin/sby --tool-dir /your/oss-cad-suite/bin
--timeout 1200 --sby-timeout 900
python3 run.py --backend formalmc --tool /your/install/bin/FormalMC
```

FormalMC 入口沿用当前 UCAgent 支持的命令接口，并先执行能力自检。当前开发设备没有真实 FormalMC，因此未宣称已完成 FormalMC 实机证明。许可证及厂商环境由执行设备提供。

这是已验证版本的快照：形式任务使用 `jobs/` 中锁定的输入和模型。只修改顶层 `rtl/` 不会改变形式验证对象；迭代设计应重新生成任务输入并重新验证。

使用 Tk GUI

详细操作图示 PDF / Word 覆盖启动连接、一键切换 SBY / FormalMC、保存配置、运行验证、结果判断、日志排错及阶段工作台，包含真实截图和关键操作区域的放大视图。

```
python3 gui/UCAgent-Desktop.pyz --server http://127.0.0.1:8814
```

执行服务需使用仓库内匹配的 UCAgent 源码，并登记 `formal/cmdbuf_rd_dual_backend` 的实际绝对路径。登记与迁移说明见 项目使用与部署 PDF。新设备可以从“**双后端验证**”入口打开登记项目；原 SBY 任务的一键导航需填入该设备的真实任务 ID。

`gui/Start-UCAgent-CMDBUF.pyw` 是本次原任务入口，默认连接本机 SSH 转发端口 18814。新设备使用上面的通用命令并填写自己的服务地址。仓库没有保存 SSH 密码、API token 或许可证。

实际 Tk GUI



实际 Tk GUI

已完成的实际验证

本次 Verilog 修订已重新完成严格 Verilog 编译、3162 项顺序等价证明、23 项仿真回归、22 项完整 SBY 任务、22 个重建模型交叉验证和 49 类注错回放。当前与历史验证说明见 Formal 验证 PDF / Word；实际输入和日志位于 `verification/verilog_20261012/`。以下原 UCAgent 14 阶段等记录保留其历史验收范围。

- 原 UCAgent 正式流程 14/14 阶段完成；规格拆分为 15 FG → 35 FC → 177 CK。
- 107 个安全检查点获得无界证明，65 个业务覆盖获得见证；5 个环境假设单列。
- 49 个代表注错变异检出并完成原版/故障版同输入回放。
- 可迁移入口原始 SBY 22/22 任务通过；FormalMC 规范化输入由 SBY 完成 22/22 交叉验证。
- Tk 桌面测试 82/82、服务及平台入口回归 39/39；实际 Tk 界面完成原 SBY 页一键切换及 SBY 单任务运行。
- FormalMC 实际环境检查记录工具缺失，没有将其显示为完整验证通过。

来源、范围及历史失败尝试见 Formal 验证说明 和 GUI 验收记录。仿真历史分册 只描述当时的仿真阶段，完整形式验证结论以后续正式报告为准。

复核上传完整性与原始证据

```
python3 tools/verify_publication.py
python3 tools/assemble_evidence.py
tar -xzf verification/closure/cmdbuf_rd_evidence.tar.gz -C verification/closure
python3 verification/closure/evidence/verify_delivery_sha256.py verification/closure/evidence
```

完整原始证据包共 212,829,414 字节，因单文件大小限制分为 5 卷。重组后的 SHA256 必须为 `9cc646b2c13a659256b45d1b30a0cd82444ca6a7d8bcbe16a6cd9e9df4ccda00`。上传不重写原始签名、证明结果或封存输入。

本项目结论适用于已定义的初始复位事务域和输入输出约束；无条件最终完成、运行中 MEM 协同复位及物理时序签核不在已完成验证范围内。第三方工具运行时不随本仓库打包。

内容来源　rtl/README.md

cmdbuf_rd　Verilog 2001 实现

当前可综合 RTL 使用 IEEE 1364-2001 Verilog，文件为：

- cmdbuf_rd.v：顶层端口与 CMDBUF / OTB 连接。
- cmdbuf_rd_cmd.v：9 项命令、年龄矩阵、最老 5 条待发窗口、发射流水、容量保底、每来源保序和响应 RR。
- cmdbuf_rd_otb.v：9 项 128 位数据、pc_err、四拍返回标记与捕获。

RTL 使用 `wire` / `reg`、`always @*`、时钟沿 `always` 和显式端口连接。每个过程独立声明循环变量；截断、零扩展和复位常量都写出位宽。

Bank 状态接口的 Verilog 表达

Verilog-2001 用一维端口保存原来 `[1:0][7:0]` 的 16 位状态：

```
input wire [15:0] rd_mem_status;
```

原逻辑位	当前 Verilog 位	含义
`rd_mem_status[0][b]`	`rd_mem_status[b]`	Bank b 的偶半区状态
`rd_mem_status[1][b]`	`rd_mem_status[8+b]`	Bank b 的奇半区状态

0 表示空闲，1 表示忙。命令选择直接读取 `rd_mem_status[addr_q[i][6:3]]`：`addr[6]` 选择奇偶，`addr[5:3]` 选择 Bank 0～7。连接原二维状态源时，传入 `{odd_status[7:0], even_status[7:0]}`。

其他端口名称、位宽和协议继续使用当前设计：18 位 AR 地址、12 位下游地址 `[17:6]`、128 位读数据、8 位 ID、5 位 port_id、ready 内 RR，MEM 返回与 R 响应之间至少一拍。

严格语法检查

```
iverilog -g2001 -s cmdbuf_rd -o cmdbuf_rd.vvp rtl/cmdbuf_rd.v rtl/cmdbuf_rd_cmd.v
rtl/cmdbuf_rd_otb.v
verilator --lint-only --Wall --assert --language 1364-2001 --top-module cmdbuf_rd rtl/cmdbuf_rd.v
rtl/cmdbuf_rd_cmd.v rtl/cmdbuf_rd_otb.v
```

仿真诊断也使用 Verilog 函数、任务和 `$stop`，合成时定义 `SYNTHESIS` 将其排除。Icarus 运行使用 `vvp -N`，使实际诊断停止返回失败状态。

形式验证中的属性、观测器和 FormalMC 模型属于验证环境，保留 `.sv` / `.svh`；它们不作为芯片 RTL 集成文件。验证任务中 `.v` 的 `FORMAL` 条件 include 用来接入外置观测器，正常集成只使用本目录三份 `.v`。

新旧实现等价性、仿真回归与完整形式验证的当前记录见 Verilog 迁移验证说明。历史 SystemVerilog 验收证据保留其原始输入与时间，不作为新的执行记录使用。

从 GitHub 使用当前项目

当前 RTL 入口为 `rtl/*.v`，使用 Verilog-2001；Bank 状态用 `[15:0]` 保存原二维位序，详见 RTL 集成说明。SBY 与 FormalMC 的验证环境保留其属性文件语法。

桌面的逐步操作和实际截图见 Tk GUI 详细操作图示；下载后可打开 离线阅读版。

原生桌面界面

客户端使用 Python 3.8+ 自带的 Tkinter/ttk；Linux 桌面系统需提供 Tk 和图形显示环境。执行服务使用 Python 3.11+ 和 `ucagent-tool/requirements.txt` 所列依赖。若已有相同 UCAgent 环境，先核对本仓库的 GUI 变更清单和版本，再配套更新客户端与服务。

```
python3 gui/UCAgent-Desktop.pyz --server http://127.0.0.1:8814
```

平台客户端入口显示原生 Tk 窗口，不打开浏览器报告。源码入口也可使用 `python3 ucagent-tool/desktop/ucagent_app.py --server http://127.0.0.1:8814`。

登记完整验证包

按照 登记格式，在执行设备建立本地登记 JSON，填写：

- `id`: `cmdbuf_rd`。
- `root`: 本仓库中 `formal/cmdbuf_rd_dual_backend` 的绝对路径。
- `session_ids`: 该执行设备真实的 SBY 阶段任务 ID；无现有任务时可以是空数组，从侧栏“双后端验证”进入。
- `default_configs`: 本机的 SBY、FormalMC 工具路径和超时配置。

给已有执行服务启动参数增加 `--portable-formal-registry /your/local/portable-formal.json`，保留原来的 workspace、工具链、导入目录及认证设置。源码服务入口是 `python3 -m ucagent.server.platform_main`，工作目录为 `ucagent-tool/`。新设备不会继承原虚拟机数据库或原任务完成状态。

连接后从“**双后端验证**”打开 `cmdbuf_rd`，选择 SBY / FormalMC，保存工具配置，执行环境检查、能力自检或完整验证。原 SBY 任务已登记关联时，可在任务页直接点击“切换到 FormalMC”。

本次原任务入口

`gui/Start-UCAgent-CMDBUF.pyw` 默认使用 `http://127.0.0.1:18814` 和原会话 ID，只适用于已连接本次虚拟机服务的当前电脑。可以通过 `--server` / `--session` 显式覆盖；其他电脑优先使用通用 zipapp 命令。

仿真与报告

`simulation/` 是早期开发交付的完整快照，工程原路径和 GCC/Verilator 要求保存在其 README 与 Makefile。新设备运行前按该工程的适配配置填写本机路径。已经可迁移且不依赖原虚拟机数据库的运行入口是 `formal/cmdbuf_rd_dual_backend/run.py`。

`verification/closure/` 原始归档经过完整离线校验并分卷上传。先运行 `tools/assemble_evidence.py`，再按原证据 README 校验全部 9,252 个载荷文件。普通 SHA 校验确认交付字节一致，不等同于持有原平台 HMAC 密钥的签名验证。

内容来源　gui/README.md

UCAgent 现有 GUI SBY　FormalMC 一键切换

当前项目的 UCAgent 执行服务已更新，原 SBY 阶段任务保持完成状态。更新后的桌面客户端在原任务页提供 **切换到 FormalMC** 按钮，点击一次直接进入完整项目的 FormalMC 操作界面。

当前电脑

已建立到虚拟机任务服务的本地转发 `http://127.0.0.1:18814`。`Start-UCAgent-CMDBUF.pyw` 打开更新后的 UCAgent 并定位到原 SBY 任务。此入口不会自动运行工具。

1. 在原 SBY 页面点击“切换到 FormalMC”。
2. 按实际安装情况填写 FormalMC 在执行设备上的工具路径，保存。
3. 点击“环境检查”“能力自检”或“运行完整验证”。
4. 顶部按钮可以切回 SBY。配置、运行记录和结果分别保存。

当前转发仅在本机保持运行；重新启动电脑后，可先建立 SSH 转发：

```
ssh -L 18814:127.0.0.1:8814 root@192.168.31.116
```

登录凭据不保存在本交付物中。

其他已有 UCAgent　FormalMC 的设备

客户端和执行服务需配套更新。此目录包含 `UCAgent-Desktop.pyz`、完整更新源包 `UCAgent-GUI-dual-backend-source.zip` 和可审查增量 `gui_patch.zip`。完整 RTL 验证输入仍使用上一版 `cmdbuf_rd_dual_backend.tar.gz`；本次没有修改 RTL、监视器、检查点或求解模型。

在已有 UCAgent Python 环境中使用更新源包，给服务启动命令增加：

```
--portable-formal-registry /path/to/portable-formal.json
```

登记模板及说明位于更新源包的 `docs/content/02_usage/13_portable_formal_gui.md`。设置包的本机路径、真实 SBY 任务 ID 和工具安装路径后，GUI 就能关联原 SBY 任务并打开完整双后端项目。不要照

搬本虚拟机的绝对安装路径。增量补丁必须先核对 `change_manifest.json` 中的源文件哈希；不同版本不能盲目覆盖。

桌面客户端入口：

```
python3 UCAgent-Desktop.pyz --server http://127.0.0.1:8814
```

实际验证

- 桌面测试：82/82 通过。
- 服务端及平台入口回归：39/39 通过。
- 真实 UCAgent GUI + 已部署服务：原 SBY 页一键进入 FormalMC，来回切换无求解请求，配置草稿及结果独立。
- 新入口实际执行 SBY 环境检查及 `quota_math` 单任务证明通过；单任务明确显示为部分验证，不算全套通过。
- 当前设备没有 FormalMC。真实环境检查显示工具缺失，并保留错误日志；没有伪造 FormalMC PASS。
- 原 SBY 阶段任务的版本、状态和阶段记录在部署及界面测试前后保持一致。

新的 GUI 运行使用独立验证包的证据格式，不改写原 UCAgent 14 阶段签名验收。原始完整 SBY 107 项安全性质及 65 项覆盖的证据随上一版运行包保留。

内容来源　ucagent-tool/docs/content/02_usage/13_portable_formal_gui.md

SBY　FormalMC 一键切换

完整双后端项目可以在 UCAgent 现有桌面 GUI 内运行。它使用包内同一份检查点清单，分别调用 SBY 和 FormalMC 入口。传统 guided 工程的迁移功能继续保留。

操作

1. 启动更新后的 UCAgent 桌面客户端，连接匹配版本的执行服务。
2. 打开已关联完整验证包的 SBY 阶段任务，点击“切换到 FormalMC”。界面直接进入该项目的 FormalMC 操作页。
3. 工具不在服务进程的 PATH 中时，填写其在执行设备上的安装路径，保存配置。
4. 点击环境检查、能力自检或完整验证。完整 FormalMC 验证会执行运行器自带的能力自检。
5. 使用顶部 SBY / FormalMC 按钮随时切换。两个后端分别保存配置、日志及结果。

切换页面不会启动工具，也不会创建新的阶段验收结论。运行中的任务始终归属启动它的后端；切换页面不会重启或改用另一工具。服务同时只接受一个新任务，停止按钮针对真实活动任务。

服务登记

管理员启动执行服务时添加 `--portable-formal-registry /path/to/portable-formal.json`。现有 workspace、工具链、导入目录及身份配置继续沿用。

登记文件示例：

```
{
  "schema_version": 1,
  "packages": [
    {
      "id": "cmdbuf_rd",
      "title": "cmdbuf_rd",
      "root": "/work/cmdbuf_rd_dual_backend",
      "session_ids": ["关联的 SBY 阶段任务 ID"],
      "default_configs": {
        "sby": {
          "tool": "/opt/oss-cad-suite/bin/sby",
          "tool_dirs": ["/opt/oss-cad-suite/bin"],
          "timeout": 1200,
          "sby_timeout": 900
```

```
        },
        "formalmc": {
          "tool": "/opt/formalmc/bin/FormalMC",
          "tool_dirs": [],
          "timeout": 1200
        }
      }
    }
  ]
}
```

`root` 必须指向管理员审查过、包含 `run.py`、`manifest.json` 和 `release_hashes.json` 的完整运行包。服务账户应能读取该目录。工具路径、许可证和运行环境以服务所在设备为准，不是桌面客户端所在设备。服务账户的许可证环境仍由管理员设置，GUI 不收集许可证密钥。

`session_ids` 只建立导航关联，不修改原任务的证据和完成状态。也可以从桌面的双后端验证入口打开已登记项目。迁移后的新设备应填写其自身目录和任务 ID，不复制旧主机的绝对路径。

结果语义

环境检查通过、能力自检通过、部分任务通过、完整证明通过分别显示。只有当前后端完整执行并满足全部检查点时，才能显示完整通过。FormalMC 初始为尚未运行，不继承 SBY 的结论。缺工具、许可证失败、编译错误、超时和未识别结果均保留诊断。

这些运行仍使用双后端包的独立证据格式；不会替代 UCAgent 原 14 阶段签名验收。当前开发设备没有真实 FormalMC，界面接入和 SBY 实测不能被解释为已经完成 FormalMC 实机证明。

交付来源

本仓库是 2026-10-12 项目交付快照。原始可迁移形式输入、RTL、模型和证据均按字节复制；`formal/cmdbuf_rd_dual_backend/release_hashes.json` 保持原清单。

UCAgent 源码来自已验收的 GUI 工程副本。源码里的 `SOURCE_SNAPSHOT.json` 是先前平台快照来源记录，不能作为本次修改后源码的当前哈希清单；本次 GUI 变更由 `gui_change_manifest.json` 记录，仓库内全部交付文件由根目录 `publication_manifest.json` 记录。

原始完整证据归档没有重新压缩或改写，只按字节分为小于 50 MiB 的 5 个文件。`verification/closure/evidence_parts.json` 同时保存每卷与整包的长度和 SHA256。

上游 UCAgent 源码的 Apache-2.0 许可证完整保存在 `ucagent-tool/LICENSE`，本仓库不重新许可或捆绑第三方 EDA 工具和运行时。缓存、当前客户端偏好、平台整库、认证信息和机器本地启动凭据不在此次交付范围内。

CMDBUF RD 详细设计方案

当前实现与设计基线的关系

当前集成入口是三份 .v 文件。端口 rd_mem_status 为 16 位，低 8 位为偶 Bank、高 8 位为奇 Bank，按 araddr[6:3] 索引。逻辑组织仍为 [1:0][7:0]。下游地址为 12 位 araddr[17:6]。

当前 RTL 使用低有效异步复位 rst_n，初始 RR 指针为 0，按 0→1→2 扫描，返回后从服务来源的下一个来源开始。错误响应仍返回捕获数据。OTB 在复位、分配、退休时物理清零。选择后不因 Bank 状态变化撤销请求，由同拍双 grant 判定成败。

这些是已有 RTL 的可追溯实现选择。系统仍未定义运行中 MEM 协同复位、非法 src/arresp 的恢复协议和上游公平性；本文不追加这些未对齐的外部要求。

内容来源　docs/design/cmdbuf_rd_design.md

cmdbuf_rd 详细 RTL 设计方案

版本日期：2026-10-10。设计基线：九项 CMDBUF 与九项 OTB、五条待发命令乱序窗口、5A / 6B / 10B 时序、成功发射后固定四拍返回、MEM 捕获后最早下一拍响应，以及每 XAU 保底的容量保护方案。

实现语言更新：2026-10-12，当前三份芯片 RTL 采用 Verilog-2001（.v）。Bank 状态的逻辑组织仍为 2×8 位，实际端口为 `rd_mem_status[15:0]`：偶 Bank 位于 `[7:0]`，奇 Bank 位于 `[15:8]`，按 `araddr[6:3]` 索引。形式属性和观测器属于验证文件，保留 SystemVerilog。当前集成文件和验证记录见 RTL 说明。

本方案用于 RTL 编码、模块接口评审和验证建模。已确认的行为作为实现约束；固定 entry、年龄矩阵、相同物理索引等属于本方案的实现选择。复位、精确 RR 顺序等尚未最终定义的边界在文末列出，不将建议冒充已经确认的接口要求。

时序图集 · 注错清单 · 容量隔离说明

1 设计目标和能力边界

cmdbuf_rd 位于上游共享 AR/R 接口与下游 Arbit、MEM Bank 之间。它将命令接收、Bank 可用性判断、仲裁发射、数据捕获和响应返回解耦，允许窗口内较新但目标 Bank 空闲的命令先发射，同时保证同一 XAU 的响应按接收次序返回。

项目	设计基线
上游来源	最多三个 XAU，src 为 0、1、2
AR 接口	一组共享负载，每拍最多接收一条；并非三条独立 AR 总线
总容量	九条未退休命令；CMDBUF 与 OTB 各九项、一一对应
入口容量保护	每个 XAU 逻辑保留一项，其余六项共享；物理槽位统一分配
下游命令带宽	每拍最多尝试发射一条，最多成功一条
乱序范围	最老五条未成功发射命令
最短前向路径	C1 接收、C2 判断、C3 请求并获得同拍仲裁结果
MEM 返回	成功发射与 MEM 接收同拍，四拍后返回一组数据及 pc_err，仅有效一拍
R 返回	每拍最多一条；同 XAU 保序，不同 XAU 在合格通道间 RR
最短完整路径	C1 接收、C3 成功、C7 捕获、C8 返回并释放
释放条件	实际向上游返回 R 响应后，同时释放 CMDBUF 与 OTB

本模块能够缓冲 Bank 冲突，并利用五项窗口内的其他空闲 Bank。它不会增大 MEM 的物理接收带宽，也不保证所有状态下持续有命令可发。最老五条全部被阻塞时，第六条即使 Bank 空闲也继续等待。

容量保留保证零占用 XAU 至少有首条命令的接收名额。用户已确认上游可以切换 src，但公平性未定；模块不能强迫上游选择某个来源，也不能无条件保证其请求最终被调度。

2 顶层接口

以下方向均相对于 cmdbuf_rd。为避免同名，AR/R 的 port_id 在本文分别命名为 ar_port_id 和 r_port_id；最终端口命名可按工程规范调整。

2 1 时钟和复位

信号	方向	位宽	说明
clk	输入	1	统一时钟；当前方案按单时钟域设计
rst_n	输入	1	当前 RTL 为低有效异步复位；运行中 MEM 协同复位协议未定义

若上游、Arbit 或 MEM 实际跨时钟域，需在模块边界增加单独的跨域协议，不能直接沿用本文固定四拍关系。

2 2 上游 AR 通道

信号	方向	位宽	含义
arvalid	输入	1	当前共享 AR 负载有效
arready	输出	3	每个 XAU 的接收资格；三位允许不同，也不要求 one-hot
araddr	输入	18	命令地址

信号	方向	位宽	含义
src	输入	2	来源 XAU，合法值 0～2
arid	输入	8	事务标签，保存后透传到 rid
ar_port_id	输入	5	保存后透传到 r_port_id
arresp	输入	2	00 正常、11 地址越界；其他值未定义

```
ar_fire = arvalid && (src < 3) && aready[src]
```

实现时先判断 src 合法性，再选择 ready 位，避免对 src=3 做越界索引。上游保证 aready 全零时 arvalid=0；这不等价于“任一 ready 为 1 就可接收当前 src”。

arid 不参与年龄排序，不要求数值随命令先后递增。本实现不依赖 ID 唯一性；响应与 entry 的归属使用内部标识和接收年龄。

2 3 上游 R 通道

信号	方向	位宽	含义
rvalid	输出	3	通过哪一位为 1 指示接收 XAU；至多一位为 1
rready	输入	3	三个 XAU 当前接收响应的能力
rdata	输出	128	一组共享数据

信号	方向	位宽	含义
rid	输出	8	原 arid
r_port_id	输出	5	原 ar_port_id
raddr	输出	3	原 araddr[2:0]
rresp	输出	2	00 正常、11 地址越界、01 奇偶校验异常

```
(rvalid & ~rready) == 0
onehot0(rvalid)
retire_fire = |(rvalid & rready)
```

本接口约定只在 ready=1 的通道上产生 valid，因此没有“先拉高 valid，再锁住共享负载等待 ready”的状态。所有 R 字段来自同一获选命令。

2 4 Arbit 接口

信号	方向	位宽	含义
req	输出	8	Bank 请求，one-hot 或全零
arb_addr	输出	12	请求级保存地址的 araddr[17:6]
pending_cnt	输出	4	全部已接收但未成功发射的命令数，范围 0～9
ar_rd_gnt	输入	8	下游仲裁响应

pending_cnt 表示全局待发数，不是窗口内命令数，也不是总占用数。输出使用当前寄存状态，不把当拍 grant 反馈形成的下一状态计数直接送回 Arbit。

2 5 MEM 接口

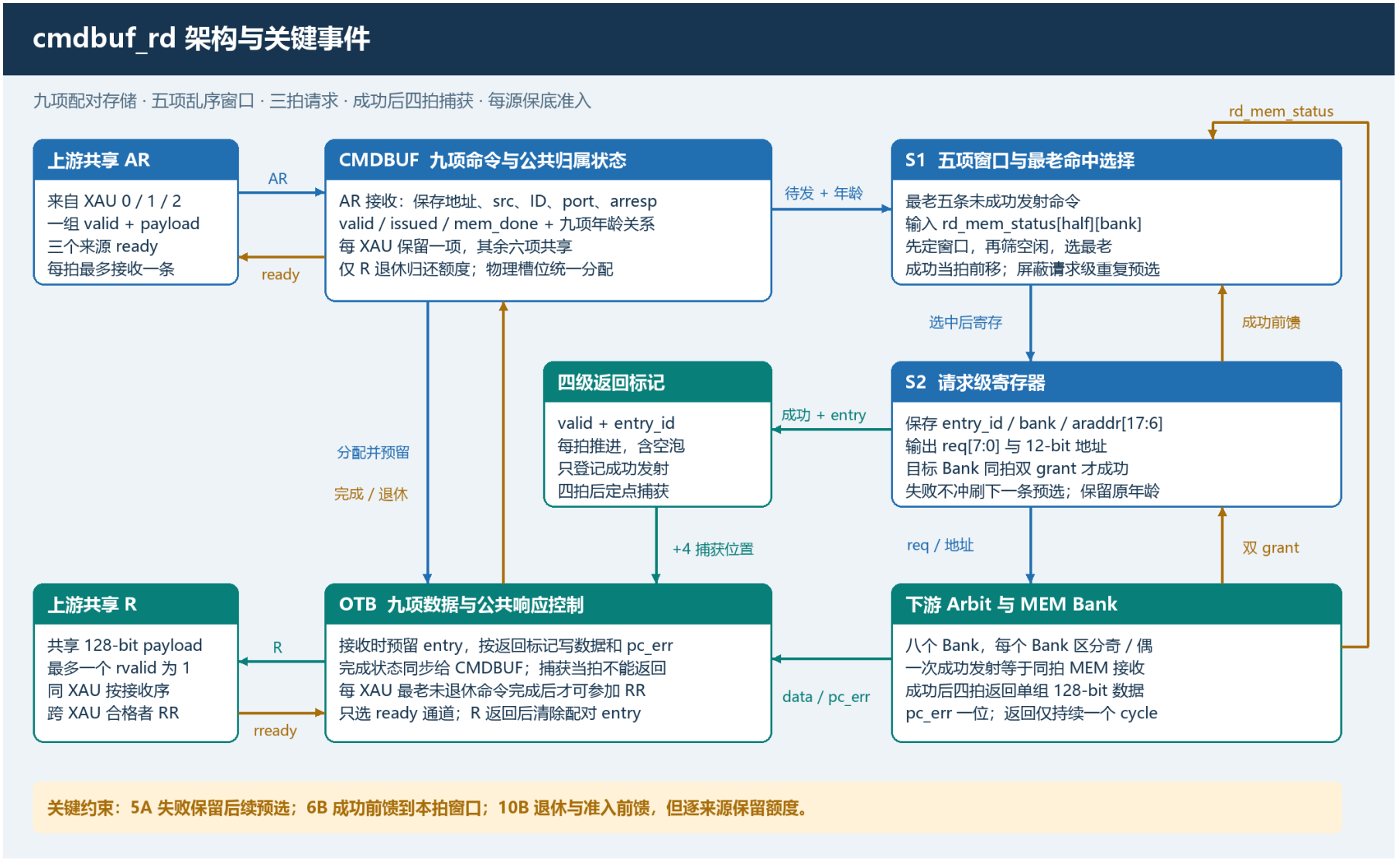
信号	方向	位宽	含义
rd_mem_status	输入	[15:0]	[7:0] 为偶 Bank，[15:8] 为奇 Bank；0 空闲、1 忙
mem_rdata	输入	128	合并后的单组返回数据
pc_err	输入	1	当前有效返回的奇偶校验异常
rw_rd_gnt	输入	8	目标 Bank 的读仲裁许可

```
bank_id   = saved_araddr[5:3]       // Bank 0~7
bank_half = saved_araddr[6]         // 0 偶、1 奇
bank_idle = !rd_mem_status[{bank_half, bank_id}]
arb_addr  = issue_addr_q            // 保存的 araddr[17:6]
```

arb_addr[0] 已携带原地址 bit 6，无须再添加 bank_half 外部端口。没有独立 MEM 返回 valid 的已定义端口；本模块依据成功发射事件固定延迟四拍，产生内部捕获有效信号。pc_err 不能充当返回 valid。

当前协议是一条 AR 对应一次 128-bit 返回，不额外引入 burst、rlast、AXI 重排规则或未约定的地址对齐限制。

3 总体架构　cmdbuf_rd 架构



cmdbuf_rd 架构

顶层仍由 CMDBUF 和 OTB 两个主要部分组成。控制职责建议如下：

功能	归属建议	作用
AR 接收与配对分配	CMDBUF 侧公共控制	计算每源 ready，原子分配 CMDBUF/OTB
命令元数据与年龄	CMDBUF	保存字段、维护先后关系
五项窗口和 Bank 判断	CMDBUF	从合格窗口选择最老命令
请求级寄存器	CMDBUF	下一拍输出 req/address，取得同拍 grant
成功发射标记流水	OTB 返回控制	将内部 entry 标识延迟四拍
数据与 pc_err 存储	OTB	无背压捕获 MEM 单拍返回
每源队头、RR、共享 R mux	公共响应控制	按同源接收顺序返回
完成和退休广播	公共控制	MEM 完成通知 CMDBUF；R 返回释放两侧

AR 接收时立即为 OTB 保留位置。请求级发出命令时，同步将命令归属传给 OTB；只有成功发射事件才能启动返回标记流水。MEM 数据回来时无需再次争抢容量。

4 存储组织与内部状态

4 1 固定九项 entry

本方案采用固定 entry 编号 0～8，内部 entry_id 为 4 bit，9～15 为无效编码。CMDBUF 与 OTB 使用相同 entry_id，使分配、返回和释放容易对应。

相同物理编号是实现选择；外部要求是一一对应。若实现为不同物理编号，必须增加可靠映射并保持本文的行为。

命令不会因成功发射而搬移到另一物理槽，槽位只在 R 返回后成为空闲。乱序、新老关系由年龄结构表达，不由槽位编号表达。

4 2 每项字段

字段	位宽	归属	更新时机
valid	1	公共归属状态	AR 接收置位，R 退休清除
issued	1	CMDBUF	分配置零，成功发射置位
mem_done	1	公共完成状态	分配置零，MEM 捕获置位
cmd_addr	18	CMDBUF	AR 接收保存
cmd_src	2	CMDBUF	AR 接收保存
cmd_id	8	CMDBUF	AR 接收保存
cmd_port_id	5	CMDBUF	AR 接收保存
cmd_arresp	2	CMDBUF	AR 接收保存
data	128	OTB	MEM 捕获保存
saved_pc_err	1	OTB	与 data 同拍保存

valid 与 mem_done 可以作为两部分共享的权威状态。CMDBUF 直接使用 mem_done 感知返回，OTB 以相同状态表示数据有效，避免维护两个独立完成位而发生失配。若为了模块边界而镜像状态，两侧必须在同一事件下更新并检查一致性。

本方案建议将 OTB 数据用寄存器阵列实现。九项共 1152 个数据 bit，能够为不同 entry 同拍执行捕获和清除；不能未经端口分析就替换成单写口 RAM。

4 3 派生状态

状态	条件	是否占容量	是否计入待发

正在请求级尝试的命令仍属于 WAIT，直至取得成功 grant；额外的 issue_valid_q/issue_entry_q 仅用于标识流水占用、屏蔽重复预选。失败不需要释放、重新分配或更改年龄。

```
FREE --AR 接收--> WAIT --成功发射--> INFLIGHT --MEM 捕获--> DONE --R 返回--> FREE
                    |
                    +--仲裁失败--> WAIT，年龄不变
```

4 4 全局寄存器

状态	建议宽度	用途
older_set	9 × 9	每项有哪些更老的有效命令
issue_valid_q	1	请求级有效
issue_entry_q	4	请求级 entry
issue_bank_q	3	请求级目标 Bank
issue_addr_q	12	请求级下游地址
ret_valid_q	4	固定四拍返回标记
ret_entry_q	4 × 4	每级返回标记对应 entry
occ_q	3 × 4	各 XAU 已接收、未退休总占用
used_cnt_q	4	总占用，0～9；可由 occ 求和派生
pending_cnt_q	4	总待发数，0～9
rr_ptr_q	2	RR 扫描起点，合法值 0～2

5 AR 准入和容量隔离

5 1 逻辑保留与共享额度

设计采用每 XAU 逻辑保留一项、其余六项共享。只有一个 XAU 活动时最多占七项，另两项继续为其他来源保留，不因其暂时没有请求而借出。

occ[x] 统计该 XAU 全部 valid 命令。成功发射以及 MEM 返回均不会减少它；只有 R 退休才减少。

```
sum(max(occ[x], 1)) <= 9

等价于：sum(max(occ[x] - 1, 0)) <= 6
```

只限制每 XAU 最多七项不够，因为 (4,5,0) 仍会占满九项并挡住第三路。上述不变量同时保护每个零占用来源的首条接收名额。

5 2 考虑当拍退休后的 ready

```
retire[x] = rvalid[x] && rready[x]
occ_after[x] = occ_q[x] - retire[x]
free_after = 9 - sum(occ_after)
shared_after = sum(max(occ_after[x] - 1, 0))

arready[x] = (free_after > 0)
          && ((occ_after[x] == 0) || (shared_after < 6))

accept[x] = ar_fire && (src == x)
occ_next[x] = occ_after[x] + accept[x]
```

减法、比较和加法使用足够宽的无符号中间值；occ=0 时的 max(occ-1,0) 必须先判断，不能发生下溢。reset 期间的 ready 输出另外受复位策略控制。

原占用 (XAU0,XAU1,XAU2)	当拍退休	arready[2:0]
(7,0,0)	无	110
(7,1,0)	无	100

原占用 (XAU0,XAU1,XAU2)	当拍退休	arready[2:0]
(7,1,1)	无	000
(4,4,0)	无	100
(7,1,1)	XAU1 唯一一条	010
(7,1,1)	XAU0 一条	111

若某来源 occ_after[x]=0，保留不变量保证至少还有一个空槽；它接收第一条不增加 max(occ_after[x],1)，因此不依赖其他来源退休即可准入。

5 3 物理 entry 分配

```
free_mask = ~valid_q | retire_mask
alloc_valid = ar_fire
alloc_entry = select_one(free_mask)
```

可以用固定最低空闲编号选择器，分配顺序不参与年龄优先级。不存在两个 AR 同拍竞争分配的问题，因为总线每拍最多一条。

接收沿原子完成：置 valid、保存所有 AR 字段、清 issued/mem_done、初始化 OTB 状态、把新命令登记为最年轻。新命令只能从下一周期开始参与窗口判断。

6 接收年龄管理

本方案采用九项年龄矩阵，避免有限宽度时间戳溢出后比较新老的问题。

定义 older_set[i][j]=1 表示“有效 entry j 比有效 entry i 更老”。对角线恒为零；任意两个不同有效 entry 的先后关系唯一。

```
oldest(M)[i] = M[i] && !(|(older_set[i] & M))
```

其中 M 是待比较集合。例如 M 为窗口内可发集合时，返回最老可发命令；M 为某 XAU 的所有 valid 项时，返回其最老未退休命令。

6 1 更新规则

只在分配或退休改变年龄关系；发射、失败、MEM 完成不改变年龄。

在先求出 valid_next 后，对每对 i、j 使用下列优先级：

```
if (!valid_next[i] || !valid_next[j] || i == j)
    older_next[i][j] = 0;
else if (alloc_valid && i == alloc_entry)
    older_next[i][j] = 1;  // 所有其他存活项都比新 i 老
else if (alloc_valid && j == alloc_entry)
    older_next[i][j] = 0;  // 新 j 不可能比旧 i 老
else
    older_next[i][j] = older_q[i][j];
```

退休后无复用时，该项所在行列清零；同拍退休并复用同一槽时，新分配规则重建该行列，不能保留旧命令年龄。

九项矩阵只有 81 个 bit，易于实现与检查。若实现优化为三角矩阵，行为与比较方向应保持不变。

7 三拍前向流水

周期	阶段	工作
C1	S0 接收	AR 握手，配对分配，保存命令
C2	S1 判断	构建五项窗口，匹配 Bank 空闲，选最老合格项；拍末寄存
C3	S2 请求	输出 req/address，当拍获得两种 grant，拍末提交成功或失败

S0、S1、S2 针对不同命令并行运行。C3 可以同时接收 C、判断 B、请求 A。不能将三阶段做成等待前一命令完成后才处理下一条的串行状态机。

S1 没有候选则向 S2 注入空泡。S2 的每次尝试在该周期结束，不因失败而保留原请求级内容；下一周期请求级接收的是本周期 S1 的结果。

8 五项窗口和最老匹配选择

8 1 候选集合

```
pending_mask = valid_q & ~issued_q
current_issue_mask = issue_valid_q ? onehot(issue_entry_q) : 0
issue_success = |(req & ar_rd_gnt & rw_rd_gnt)
success_mask = issue_success ? current_issue_mask : 0


rank_pool = pending_mask & ~success_mask


older_count[i] = popcount(older_set[i] & rank_pool)
window_mask[i] = rank_pool[i] && (older_count[i] < 5)


bank_idle_mask[i] = !rd_mem_status[cmd_addr[i][6:3]]
candidate_mask = window_mask & bank_idle_mask & ~current_issue_mask
select_onehot = oldest(candidate_mask)
```

顺序必须是先决定最老五条，再筛 Bank 空闲。若先在全部九项中过滤空闲 Bank，会把窗口外命令错误引入候选。

所有 entry 的 Bank 状态可以并行读取。窗口 rank 最大为八，计数比较使用四位即可。若候选存在，年龄关系保证 oldest 返回唯一命令。

8 2 6B 成功当拍前移

当前请求级 A 成功时，success_mask 在本拍 S1 计算中将 A 排除。原第六条 F 可当拍进入窗口，并在下一拍发出请求。

```
N：请求 A 成功；B～E 忙；F 空闲 → S1 当拍选 F
N+1：请求 F
```

如果等 issued 寄存器下一拍更新后才前移，会多一个气泡，与所选 6B 不符。

8 3 失败时的窗口位置

当前请求级 A 失败时，A 仍留在 rank_pool 和最老五条窗口中，只在最后候选掩码中被屏蔽，避免同拍重复预选。

不能在构窗前无条件移除 current_issue_mask，否则 A 失败也会让 F 错误进入窗口。

8 4 五条全部阻塞

A ~ F 从老到新，A ~ E 对应 Bank 均忙、F 空闲且当前请求级为空时，candidate_mask=0。

rd_mem_status 若持续不变，窗口不会自动滑动，F 不发射。

这一等待是规格行为。每 XAU 容量保留只能让其他来源的命令进入 CMDBUF，不能授权窗口外命令绕过发射规则。

9 请求级和 5A 失败恢复

9 1 请求级寄存器

在每个周期结束时：

```
issue_valid_q <= select_valid
if (select_valid):
    issue_entry_q <= select_entry
    issue_bank_q  <= cmd_addr[select_entry][5:3]
    issue_addr_q  <= cmd_addr[select_entry][17:6]
```

下一周期输出：

```
req = issue_valid_q ? (8'b1 << issue_bank_q) : 8'b0
arb_addr = issue_addr_q
issue_success = issue_valid_q
             && ar_rd_gnt[issue_bank_q]
             && rw_rd_gnt[issue_bank_q]
```

req 必须来自请求级寄存器，不能由本拍 grant 决定是否产生，否则会引入仲裁组合环。无请求时的 arb_addr 可输出寄存值或零；本方案建议对外置零便于观察，此行为不是已确认功能要求。

9 2 5A 的恢复过程

周期	S1 判断	S2 请求
C2	A	空
C3	B	A 失败
C4	A	B
C5	后续命令	A 重试

A 失败不冲刷 B，也不在失败当拍重新选择 A 替换 B。A 的 valid 保留、issued 仍为零、年龄不变；下一拍自动恢复候选资格。

如果只有 A 一条命令，C3 失败后，C4 重判、C5 重试。这一重试间隔来自已经选择的流水行为。

9 3 请求拍 Bank 状态变化

已确认的成功判据始终是目标 Bank 的同拍双 grant。判断拍空闲、请求拍转忙时，是否额外撤销请求尚未独立对齐。

本方案建议锁定 S1 的选择，在 S2 正常输出请求，由双 grant 决定是否接受，避免加入第二套隐含选择规则。该建议列为待确认实现边界。

9 4 同步给 OTB 的信息

CMDBUF 在请求级同时向 OTB 提供 attempt_valid、entry_id 及 issue_success。最小功能信息是命令归属与成功提交；完整地址和元数据已由接收时保存，不必重复存入 OTB。

attempt_valid 便于观察请求尝试；只有 issue_success 才允许 OTB 产生返回标记。失败不能造成 OTB 完成计时、重复分配或提前清除。

10 固定四拍返回标记

下游成功接收后不能反压返回，因此使用四级 valid 加 entry_id 标记流水，按时钟无条件推进。

```
每个时钟沿：
    ret_valid_q[0] <= issue_success
  成功时 ret_entry_q[0] <= issue_entry_q


  对 k=1..3：
      ret_valid_q[k] <= ret_valid_q[k-1]
    前一级有效时 ret_entry_q[k] <= ret_entry_q[k-1]


  若本沿前的 ret_valid_q[3] 为 1：
      capture_entry = 本沿前的 ret_entry_q[3]
      otb_data[capture_entry] <= mem_rdata
      otb_pc_err[capture_entry] <= pc_err
      mem_done[capture_entry] <= 1
```

在 Verilog 中用同一沿的非阻塞赋值读取旧末级即可。不要将 capture_valid 再额外寄存一拍，也不要在捕获时使用刚移入末级的下一状态。

时钟沿	A 标记和数据的动作
C3 末	A 成功，写入第 0 级
C4 末	A 移入第 1 级
C5 末	A 移入第 2 级
C6 末	A 移入第 3 级
C7 末	读取旧第 3 级，捕获 A 数据和 pc_err
C8 周期	A 的已寄存完成状态可参与 R 选择

空泡、仲裁失败写入无效标记；已有标记继续前进。AR 满、没有新请求、rready 全零均不能暂停该流水。

外部无返回事务标识，本方案依赖“每个成功命令恰好四拍后返回，且返回合并总线保持该对应关系”的接口约束。若下游将来改为可变延迟，必须增加可匹配的返回标签或改变协议，当前标记流水不能继续直接使用。

11 MEM 捕获和完成通知

捕获当拍将完整 128-bit 数据和 pc_err 写到标记指定的 OTB entry，并使该 entry 的 mem_done 置位。CMDBUF 使用同一完成状态或接收同拍完成通知。

捕获条件只由 ret_valid_q[3] 决定。数据全零、pc_err=0 仍是合法返回；不能以数据非零或错误有效代替捕获有效。

捕获时 entry 必须 valid、issued 且尚未 mem_done。它仍占用容量，后续 rready 不允许响应时，数据和错误保持直到退休。

当前 MEM 捕获不能组合旁路进入本拍 R 候选。R 只能读取本周期开始前已经寄存的 mem_done 和 OTB 数据，从而保证最早下一 cycle 返回。

12 每 XAU 保序及 RR 响应

12 1 每源队头

```
src_mask[x][i] = valid_q[i] && (cmd_src[i] == x)
head_onehot[x] = oldest(src_mask[x])
head_valid[x] = |head_onehot[x]
head_done[x] = mem_done[head_entry[x]]，仅在 head_valid 时读取


r_candidate[x] = head_valid[x] && head_done[x] && rready[x]
```

先找该 XAU 最老的未退休命令，再检查它是否完成。不能在该 XAU 的所有完成项里找最老，否则会跳过尚未完成的真正队头。

不同 arid 的同源请求也必须遵循同一接收序列。不同 XAU 不施加全局返回顺序：XAU0 的队头未完成时，XAU1 的合格队头仍可响应。

12 2 RR 实现建议

当前已确认的是在合格通道间 RR。为了给出可编码方案，建议 rr_ptr 表示扫描起点，按 0→1→2 循环，从起点扫描三个通道，选第一个 r_candidate=1 的通道。

```
winner = rr_first(r_candidate, rr_ptr_q)
rvalid = winner_valid ? onehot(winner) : 0
```

```
若 retire_fire: rr_ptr_next = (winner == 2) ? 0 : winner + 1
否则: rr_ptr_next = rr_ptr_q
```

图 08 使用上述方向和空闲保持；初始起点及精确推进策略尚未独立确认。本方案建议初始指针为 0，但不能把这一建议当作既定系统复位协议。

若三个通道持续有完成队头且持续 ready，上述 RR 方案会循环服务三个通道。它不为非 ready 或队头未完成的通道预留 R 总线。

12 3 共享负载选择

```
response_entry = head_entry[winner]
rdata      = otb_data[response_entry]
rid        = cmd_id[response_entry]
r_port_id = cmd_port_id[response_entry]
raddr      = cmd_addr[response_entry][2:0]
rresp      = error_encode(cmd_arresp[response_entry], otb_pc_err[response_entry])
```

winner 无效时建议将共享负载置零，便于波形和降低无意义翻转；协议检查只在对应 rvalid 有效时比较负载。

不能用实时 AR 字段或实时 MEM 数据构造 R 负载。R 被选中时对应 rready 已经为 1，因此该周期结束沿同时完成传输和退休。

13 错误处理

保存的 arresp	保存的 pc_err	rresp
00	0	00
00	1	01
11	0	11
11	1	11

地址越界优先于奇偶校验异常。arresp=11 的命令仍完整经历分配、窗口选择、请求、MEM 返回和 R 返回，不做本地立即完成。

```
if saved_arresp == 2'b11: rresp = 2'b11
else if saved_pc_err:    rresp = 2'b01
else:                    rresp = 2'b00
```

该编码逻辑仅对已定义的 arresp=00/11 给出协议承诺。错误响应的数据内容尚未独立约定；本方案建议仍输出捕获值，同时以 rresp 指示数据状态，最终需确认。

14 R 退休和 10B 同拍复用

R 成功传输产生唯一 retire_entry，同时清除旧命令在 CMDBUF、OTB 的有效归属、完成状态和年龄关系，并扣除所属 XAU 的占用。

本方案建议分配及退休时将 OTB data/pc_err 清零。功能正确性至少要求旧数据失效且不能暴露给新事务；若为功耗或 RAM 实现改为只清有效位，是否符合“清除”的最终要求需确认。

14 1 满表复用

free_mask 包含当拍 retire_mask。因此即使沿前九项均 valid，仍能选中本拍即将退休的槽位分配新请求。每来源额度也必须先扣除当拍退休再计算。

例如 (7,1,1) 中 XAU1 唯一命令退休，本拍只能对 XAU1 给出 ready=1；若退休的是 XAU0 的一条，则释放的是共享额度，三路均可具备准入资格。

14 2 同一槽位更新优先级

R 负载由沿前旧状态输出；时钟沿上同槽新分配优先于旧退休清除：

```
if reset_active:
  清控制状态
else if alloc_valid && alloc_entry == i:
    valid=1; issued=0; mem_done=0
  保存新 AR 元数据；清新 OTB 数据状态；重新登记最年轻年龄
else if retire_fire && retire_entry == i:
    valid=0; issued=0; mem_done=0
  清旧归属及对应存储
else:
  若成功发射命中 i，issued=1
```

若 MEM 捕获命中 i，保存数据和 pc_err，mem_done=1

这是每项的逻辑优先级，不代表将所有数组写成多个冲突的时钟沿 always 过程。每个寄存器应有唯一写入过程；并行事件分别作用于不同 entry，不能用顶层全局 if/else 误删其他事件。

14 3 合法并发关系

同拍事件	可否同 entry	说明
R 退休与 AR 分配	可以	10B，输出旧负载后建立新归属
MEM 捕获与 R 退休	不可以	本拍捕获项此前尚未完成，不能本拍返回
成功发射与 R 退休	不可以	待发与已完成状态互斥
成功发射与 MEM 捕获	不可以	同一命令相隔四拍；出现同项通常是重复发射错误
AR 分配与旧返回标记命中	不可以	未捕获命令不能先退休，其槽位不能被重分配
上述事件作用于不同 entry	可以	可同时接收、成功发射、捕获、返回

正常协议下，旧命令退休时已不存在其未到达的返回标记，因此 entry 重用无需额外 generation tag。复位打断未完成事务是另一边界，见第 21 节。

15 计数及状态一致性

```
used_next = used_q + ar_fire - retire_fire
pending_next = pending_q + ar_fire - issue_success
occ_next[x] = occ_q[x] + accept[x] - retire[x]


sum(occ_q) == used_q == popcount(valid_q)
pending_q == popcount(valid_q & ~issued_q)
0 <= pending_q <= used_q <= 9
sum(max(occ_q[x],1)) <= 9
```

预选、失败、MEM 捕获不改变 pending_cnt。R 退休的命令早已 issued，退休本身也不再减少 pending_cnt。

6B 可以在本拍窗口计算中提前排除成功项，但 pending_cnt 对外仍表示本周期寄存状态，并在本周期末更新。这样不把下游 grant 通过计数绕回下游仲裁逻辑。

实现可以选择寄存计数或由位向量计算，但保留额度计算、对外指示和验证采样必须使用一致的周期定义。

16 周期内组合依赖

建议按以下依赖形成组合逻辑，并统一在周期结束沿提交：

```
当前命令/OTB/年龄状态 + rready
  → 各 XAU 队头和合格候选 → RR → R 负载/retire
  → 扣除 retire 后的占用 → 每来源 arready → AR 接收/分配

当前请求级寄存器
    → req/address → 下游两组 grant → issue_success
  → 成功前馈后的窗口 → Bank 匹配/最老选择 → 下一周期请求级

返回标记旧末级 + 当前 MEM 返回
  → 本沿捕获 → 下一周期可见的 mem_done/data
```

当前 R 选择不依赖新 AR，当前 req 不依赖本拍选择结果，当前 R 候选不依赖本拍 capture_next。上述隔离是保持三拍路径、禁止同拍 MEM 旁路及避免本模块内部组合环的关键。

17 性能与时序实现考虑

17 1 理想吞吐

当上游每拍供给、窗口中持续存在可发项、每次双 grant 成功、R 持续可返回时，模块可每拍接收一条、成功发射一条、捕获一条和返回一条。

单 XAU 最多七项仍能支撑这一理想吞吐：C1～C7 接收七条，第一条 C3 成功、C7 捕获、C8 返回；C8 同时接收第八条，随后每拍释放与接收相抵。这里必须对容量额度做当拍退休前馈，即使物理池中还剩另两路保留空槽也一样。

若没有 10B，单来源达到七项时会额外停一拍。若增加额外 R 流水级，或 MEM 延迟不再是四拍，则这一七项支撑一条每拍的结论也需重新评估。

17 2 关键组合路径

路径	关注点
grant → 成功前馈 → 窗口 → 最老选择 → 请求级寄存器	6B 明确要求这条路径在一拍内完成
mem_done/年龄 → 每源队头 → RR → 共享 128-bit mux	影响 R 返回周期
rready → RR/retire → 每源额度 → arready	10B 和容量保留引入的前馈路径
AR 握手 → 元数据写入及年龄更新	不能越过 S0 寄存直接影响本拍判断

九项年龄比较、Bank 空闲解码和每源队头可并行计算。可预先计算“当前请求成功/不成功”两套窗口，再由成功信号选择，以缩短 late grant 路径；必须证明与统一窗口公式等价。

为了频率随意再插一拍会改变 6B、三拍请求或最早 R 返回时序，不能在保持原规格的名义下增加延迟。目标频率、工艺、综合约束尚未给出，本方案不承诺具体 MHz。

17 3 存储量估算

按本文字段直接实现，未计组合逻辑及实现相关冗余：

项目	bit 数
命令元数据与 valid/issued/mem_done	9 × 38 = 342
OTB 数据及 pc_err	9 × 129 = 1161
年龄矩阵	81
请求级	20
返回标记流水	20
三路占用、总占用、待发数、RR 指针	22
合计	约 1646

这是寄存状态规模估算，不是综合面积结果。共享状态无需在两个模块中重复实现；是否复制、编码或优化由综合和模块边界决定。

18 典型时序

18 1 连续无阻塞

周期	AR 接收	S1 选中	S2 请求成功	MEM 捕获	R 返回
C1	A	—	—	—	—
C2	B	A	—	—	—
C3	C	B	A	—	—
C4	—	C	B	—	—
C5	—	—	C	—	—
C6	—	—	—	—	—
C7	—	—	—	A	—
C8	—	—	—	B	A
C9	—	—	—	C	B
C10	—	—	—	—	C

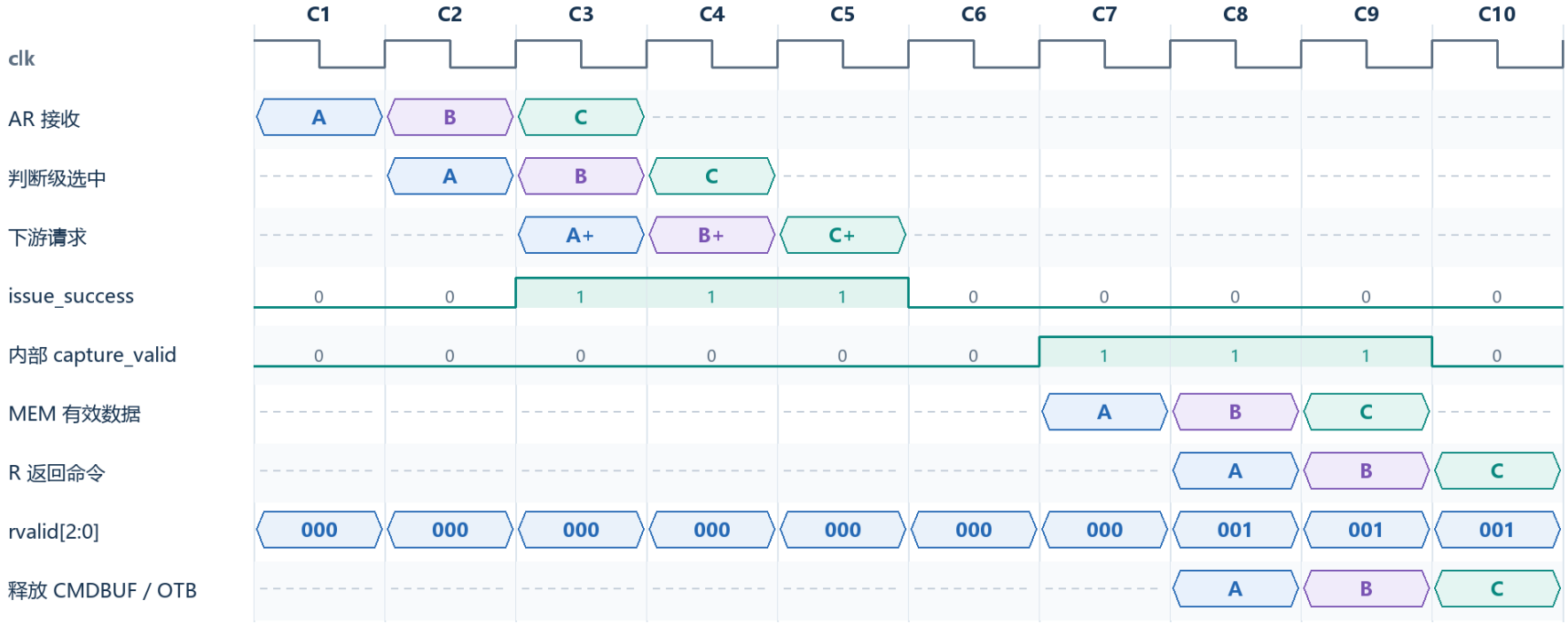
连续流水时序

01　连续正常流水

已确认 无旁路

每列一个周期，右边界上升沿提交；+ 成功，× 失败，虚线表示无对应有效数据。

A、B、C 均来自 XAU 0；仲裁成功；rready[0] 始终为 1



C3 同时接收 C、判断 B、请求 A。A 在 C3 发射成功、C7 捕获返回、C8 响应成功后释放。

已确认：MEM 返回当拍写 OTB，最早下一拍参与 R 仲裁；仍须满足源内保序、rready 和 RR。

连续流水时序

18 2 已对齐场景索引

图	核心行为
01 连续流水	三拍请求、固定四拍捕获、下一拍可返回
02 同 XAU 保序	后接收先完成的命令等待前序返回
03 窗口前移	第六条通过成功前馈进入窗口
04 双 grant 与空泡	失败没有返回标记
05 选择 5A	A 失败保留已选 B
06 选择 6B	成功当拍选入后续项
07 单拍返回	R 不 ready 时仍捕获并保持
08 RR 示例	只选择 ready 的完成队头；精确环方向见待定项
09 字段与错误	地址切片、字段透传、异常优先级
10 选择 10B	满表同拍退休和接收
11 窗口全阻塞	不因第六条可用而越窗发射
12 容量保留	一路占七项后其他来源仍可准入
13 保底复用	释放唯一项时仅恢复该来源资格

19 当前 RTL 划分

当前文件	主要内容
cmdbuf_rd.v	顶层端口、公共事件、模块连接

当前文件	主要内容
cmdbuf_rd_cmd.v	九项元数据、状态、年龄、窗口、请求级、容量准入和响应 RR
cmdbuf_rd_otb.v	九项数据、pc_err、固定四拍返回标记与捕获
当前实现将分配、容量保护、来源队头和响应 RR 合并在 cmdbuf_rd_cmd.v 内。状态保持单一归属、每个寄存器唯一写入者，分配/完成/退休事件同步传递给 OTB。	

常量建议统一定义：XAU_NUM=3、ENTRY_NUM=9、WINDOW_SIZE=5、BANK_NUM=8、ADDR_WIDTH=18、DATA_WIDTH=128、ID_WIDTH=8、PORT_ID_WIDTH=5、MEM_LATENCY=4。参数化不代表任意组合均已验证；首先固定这一基线实现。

20 验证和关键检查

20 1 基准模型

每次合法 AR 握手分配独立测试序号 accept_seq，保存所有输入字段。为每个 XAU 建接收队列，并维护全局占用、待发集合、成功发射四拍日历以及完成记录。

参考模型按本方案窗口规则选下一拍请求，不使用 arid 数值决定新老。MEM 在对应日历拍提供有效数据；无效拍可以改变总线，暴露晚采样和实时总线误用。

R 只从此前已完成的各 XAU 队头选择。检查完整负载属于同一命令；错误数据值尚未确定时，错误子例优先核对状态与归属，正常子例逐位比较 128-bit 数据。

20 2 安全性检查

以下是断言意图，实际 SVA 需绑定 RTL 信号及采样语义：

- req 与 rvalid 均 onehot0；rvalid 不得指向当拍 rready=0 的通道。

- ar_fire 必须命中合法 src 的 ready；一次 AR 最多分配一个配对 entry。
- mem_done 必须蕴含 valid 和 issued；issued 不得在 grant 失败时置位。
- 每条命令最多成功发射一次、MEM 捕获一次、R 返回一次。
- 捕获 entry 必须有效、已成功发射且未完成；同条命令本拍捕获时不能本拍退休。
- 每次成功发射恰好在四拍后对应一次捕获，失败不得生成返回标记。
- 预选命令必须属于成功前馈后的最老五条集合，目标 half 空闲，且不是当前请求级项。
- 候选非空时，选中者不存在同集合内更老命令；候选为空则不产生选择有效。
- 同 XAU 响应顺序与实际 AR 握手顺序一致；所有元数据和数据归属一致。
- valid、计数、每源占用和年龄结构一致，保留容量不变量一直保持。
- 同拍复用输出旧命令，新命令在下一状态有效、最年轻、未发射、未完成。

20 3 吞吐检查

5A、6B、10B 需要检查精确相邻周期行为，不能只检查最终数据正确。额外空泡也可能是规格偏离。

重点覆盖单 XAU 达七项后的持续接收、全表九项的跨源复用、失败时保留 B、成功当拍引入 F、同时接收/发射/捕获/响应四事件。

20 4 进展条件

无条件安全性检查与有环境前提的活性检查分开。rd_mem_status 持续阻塞、上游公平性未定、rready 永久为零或仲裁持续失败时，不要求每条命令最终完成。

现有 49 组注错清单 可作为验证覆盖来源。容量准入模型已对 190 个合法占用状态及 1716 个接收状态转换进行穷举检查；这是抽象容量模型结果，不是完整 RTL 形式验证或时序收敛结果。

21 复位与异常边界

复位建议将 valid、issued、mem_done、请求级 valid、四拍返回 valid、年龄矩阵、计数和 RR 控制清零。复位期间 req/rvalid/arready 建议全零，具体极性和生效方式待系统确认。

数据阵列可在无效状态下不做复位，由分配和退休清零；外部负载在无 valid 时按实现建议置零。若要求所有存储 bit 复位为零，可调整，但会影响复位网络负担。

运行中复位不能只清本地标记而默认下游旧返回自动消失。需要系统明确同步冲刷 MEM，或停发并排空旧事务等协议，才能定义何时恢复接收。当前方案不自行选定恢复流程。

项目	已知行为	本方案建议或仍待确定内容
容量隔离	不允许 XAU 独占导致其他来源没有 entry	已纳入每源一项保底、六项共享的实现方案
复位	当前 rst_n 低有效异步复位，初始 RR 为 0	运行中 MEM 返回与复位的协调仍未定义
非法输入	src 0～2、arresp 00/11 有定义	src=3、arresp=01/10 的恢复行为未定义
重复 ID	arid 为透传标签	本实现能按内部归属区分；系统是否另有限制待说明
数据清除	退休后旧归属和数据不能再次有效	当前 OTB 在复位、分配与退休时物理清零，属于实现选择
错误响应数据	rresp 优先级已确认	当前实现仍输出捕获值，属于实现选择
RR 细节	合格通道间 RR	当前实现初始 0，按 0→1→2 扫描，服务后推进，空闲保持
Bank 状态变化	双 grant 决定最终成功	当前实现请求级不因 status 改变撤销，属于实现选择
无效周期负载	无有效请求或响应时不作功能比较	建议输出零，尚无强制值约定
上游公平性	已确认可切换 src	公平性未定，不建立无条件最终接收保证

22 实施交付边界

本方案完成了接口、状态、选择算法、返回匹配、容量准入、并发更新优先级和验证判据的设计定义。可先据此搭建 RTL 主体与参考模型；将第 21 节中会影响外部行为的选择记录为可追踪配置，最终冻结前按系统要求确定。

现有时序图是规格示意，容量检查是独立抽象模型。当前已经交付三份可综合 Verilog-2001 RTL，并完成重新编译、23 项仿真、22 项 SBY 任务与顺序等价检查；详细范围见 Formal 验证分册。尚未完成物理综合与静态时序签核，不能据此宣称目标频率或面积达标。

CMDBUF RD 时序图集

每列表示一个周期，右边界采样提交。最短路径为 C1 接收、C2 判断、C3 成功发射、C7 捕获 MEM 单拍数据、C8 返回 R 并退休。不同命令各级可以重叠。

已确认采用 5A、6B、10B：失败保留后续预选；成功当拍前移待发窗口；满表退休当拍允许合规来源复用。MEM 捕获当拍不能将该数据直接返回 R。

严格窗口包含最老五条未成功发射命令。五条都不匹配 Bank 状态时，第六条即使匹配也不能越窗发射。容量使用每来源一项逻辑保底及六项共享；上游可以切换 src，但没有已定义的公平性保证。

图 01　连续正常流水

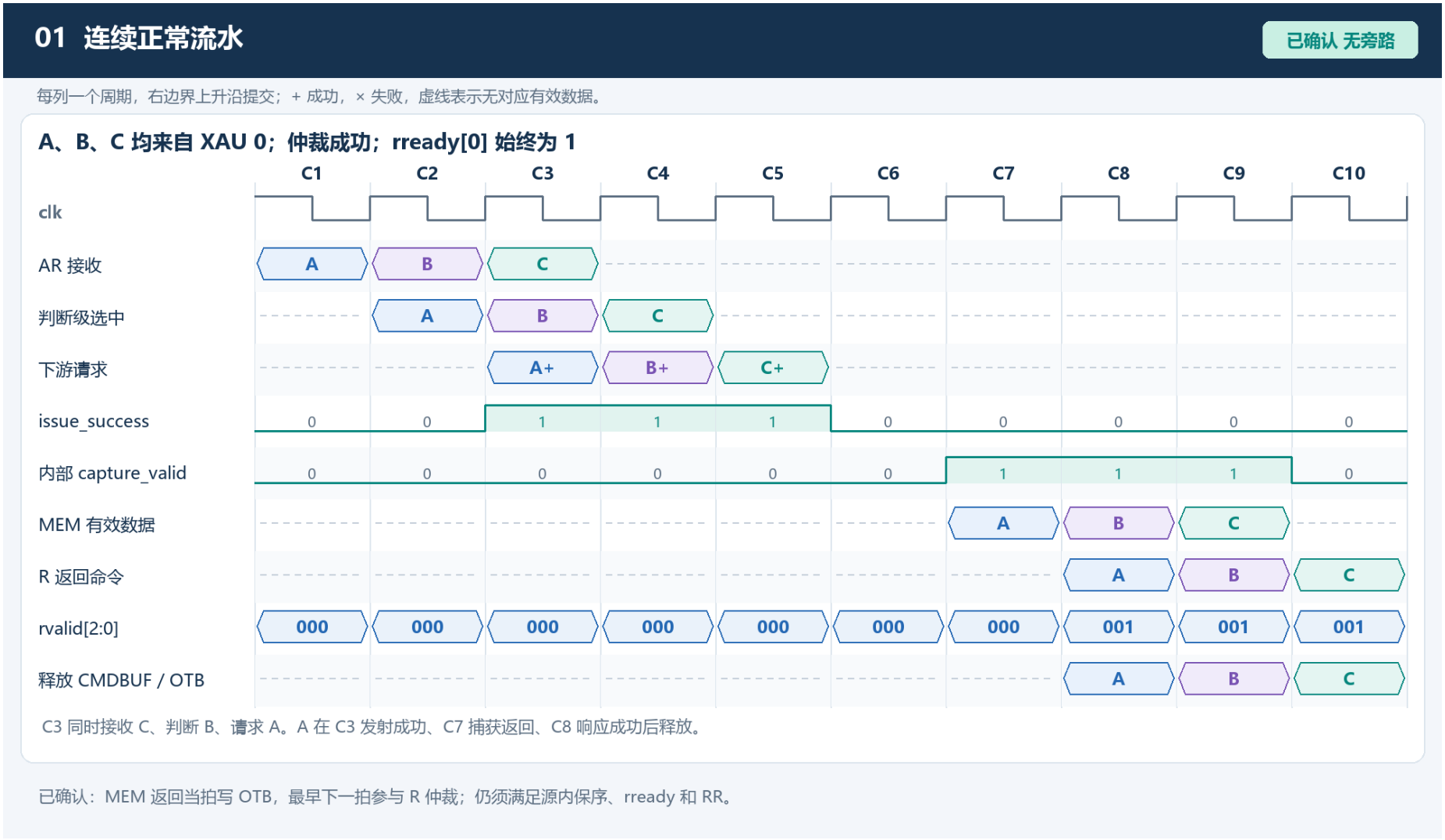


图 01　连续正常流水

图 02　乱序发射与单 XAU 保序返回

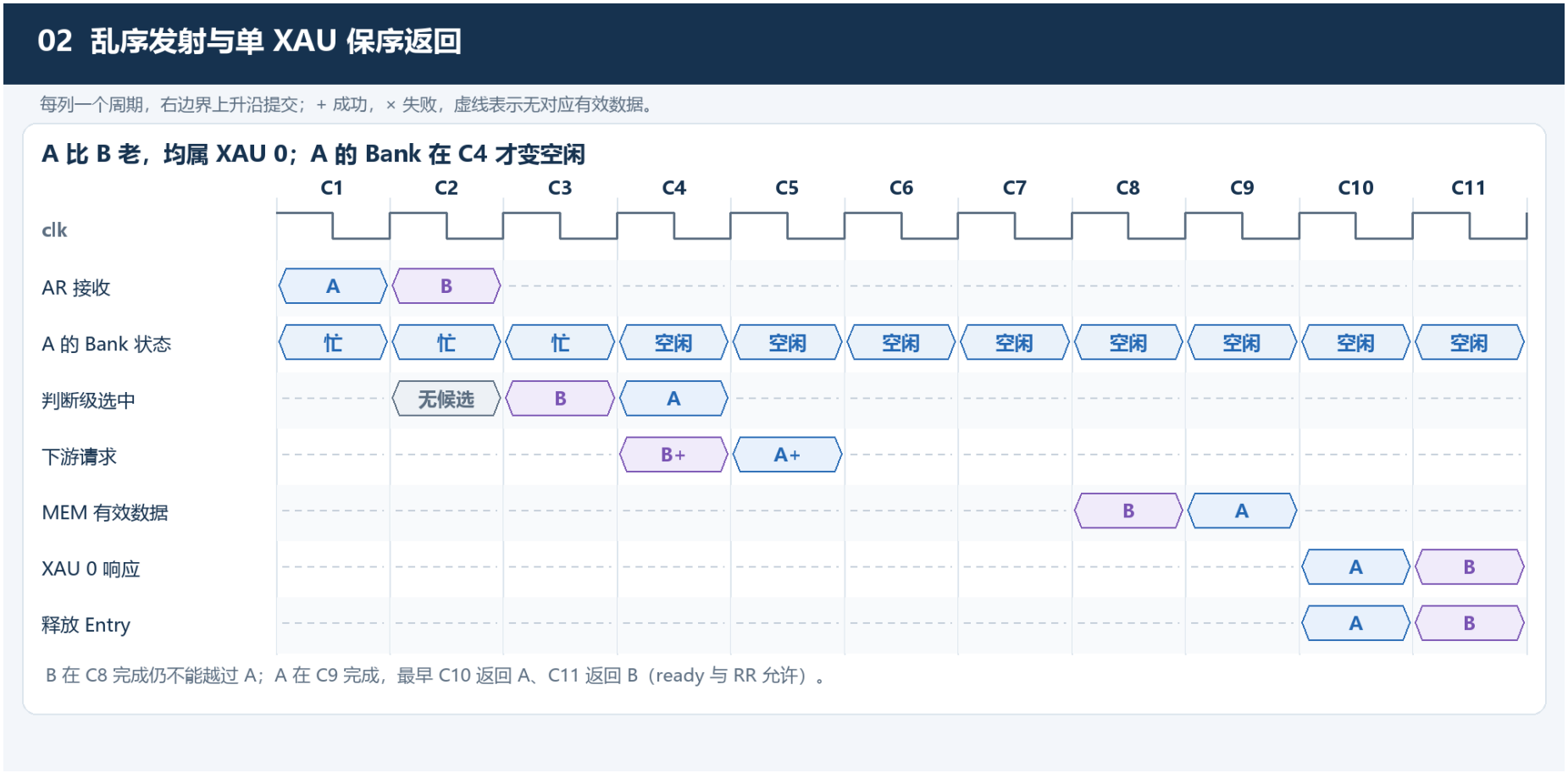


图 02　乱序发射与单 XAU 保序返回

图 03　窗口外命令等待成功前馈后进入窗口



图 03　窗口外命令等待成功前馈后进入窗口

图 04　双 grant 判定与返回气泡

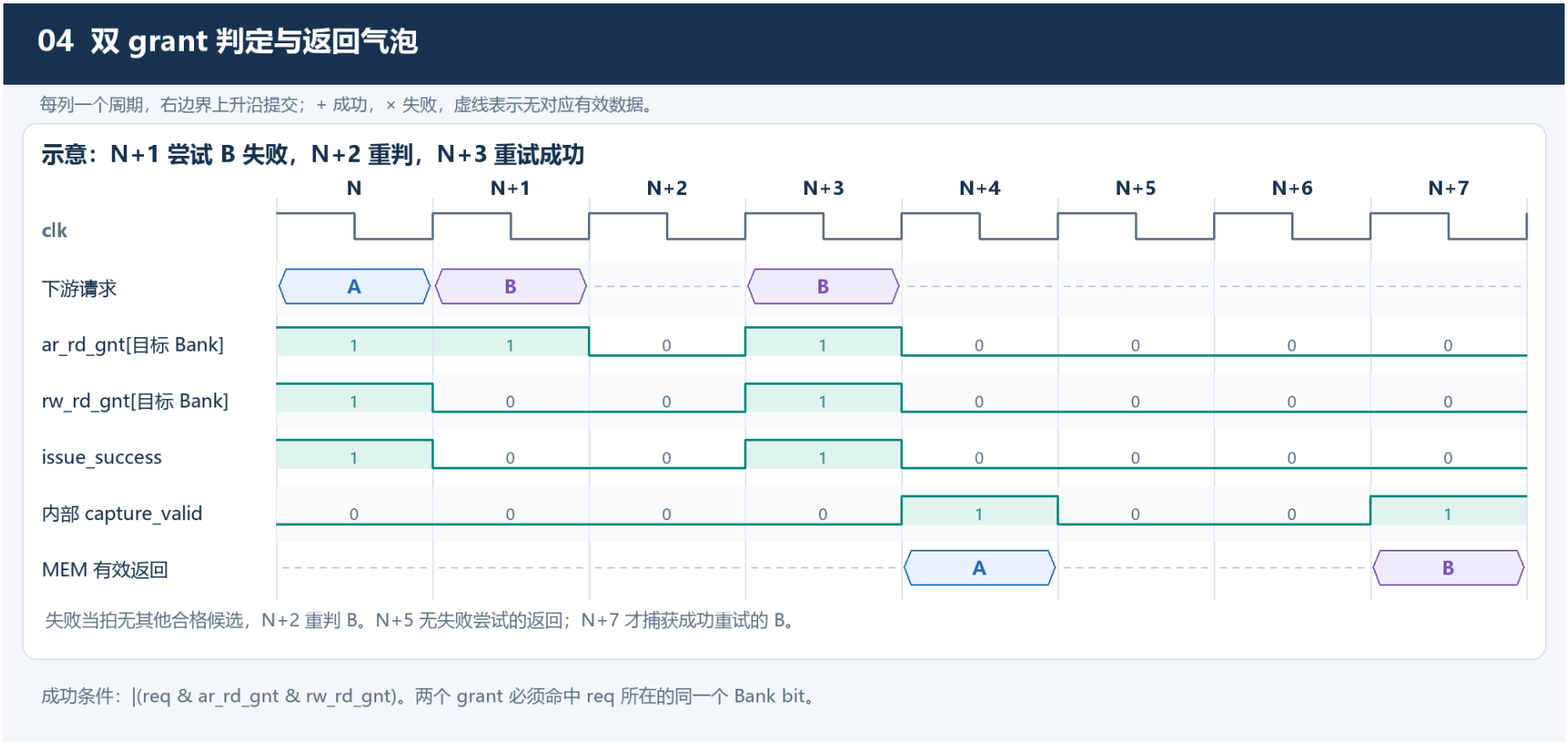


图 04　双 grant 判定与返回气泡

图 05　失败不取消后续预选 采用 5A



图 05　失败不取消后续预选：采用 5A

图 06　成功当拍前移窗口 采用 6B



图 06　成功当拍前移窗口：采用 6B

图 07　MEM 单拍返回必须捕获 不受 R ready 影响



图 07　MEM 单拍返回必须捕获，不受 R ready 影响

图 08　响应 RR 仅在当拍 ready 的完成队头间选择



图 08　响应 RR 仅在当拍 ready 的完成队头间选择

图 09　地址拆分 字段透传与异常优先级

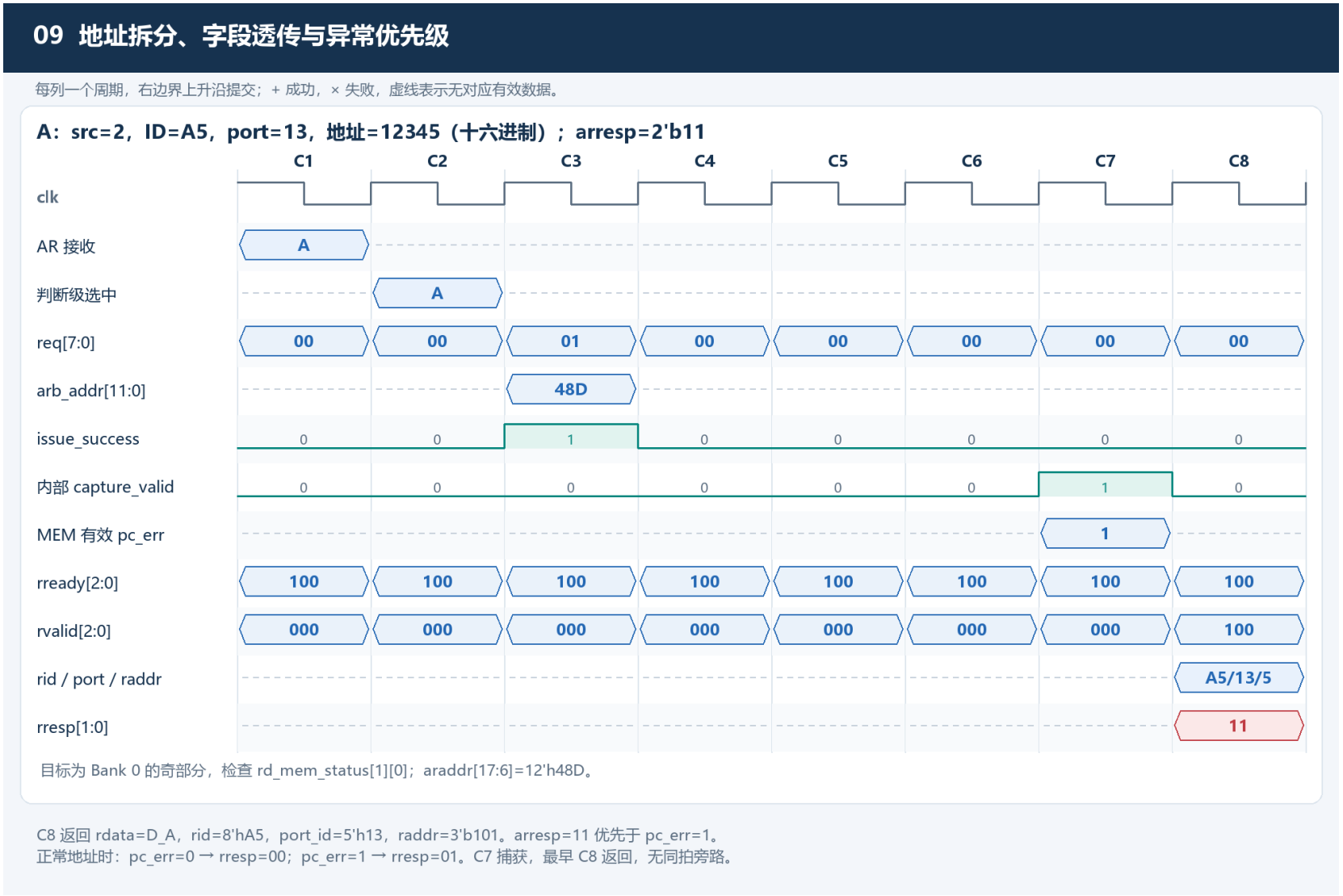


图 09　地址拆分、字段透传与异常优先级

图 10　满表同拍释放复用 采用 10B

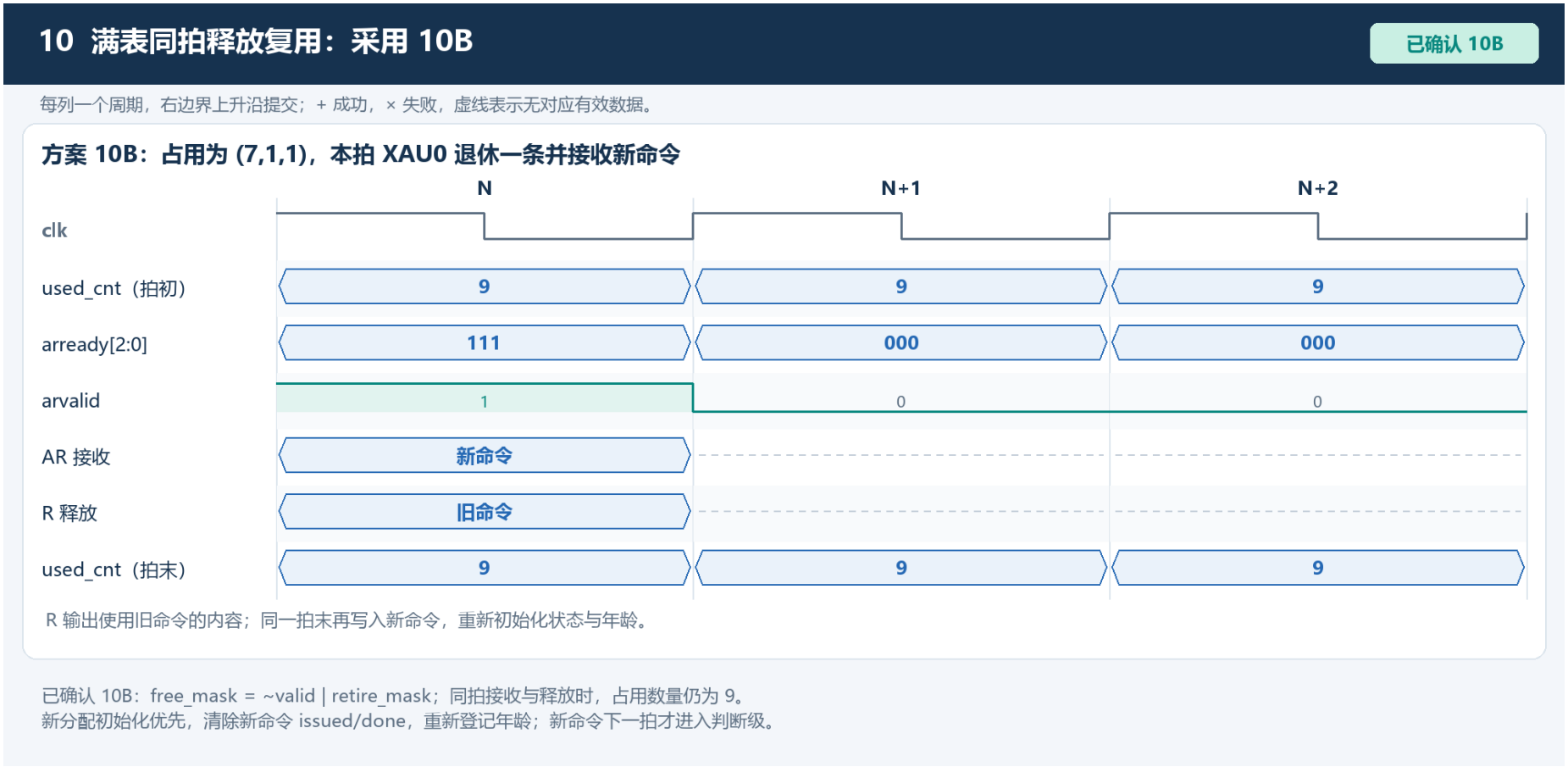


图 10　满表同拍释放复用：采用 10B

图 11　窗口内全忙 第六条空闲命令仍被阻塞



图 11　窗口内全忙：第六条空闲命令仍被阻塞

图 12　容量保留 一路占七项后仍允许其他来源接收

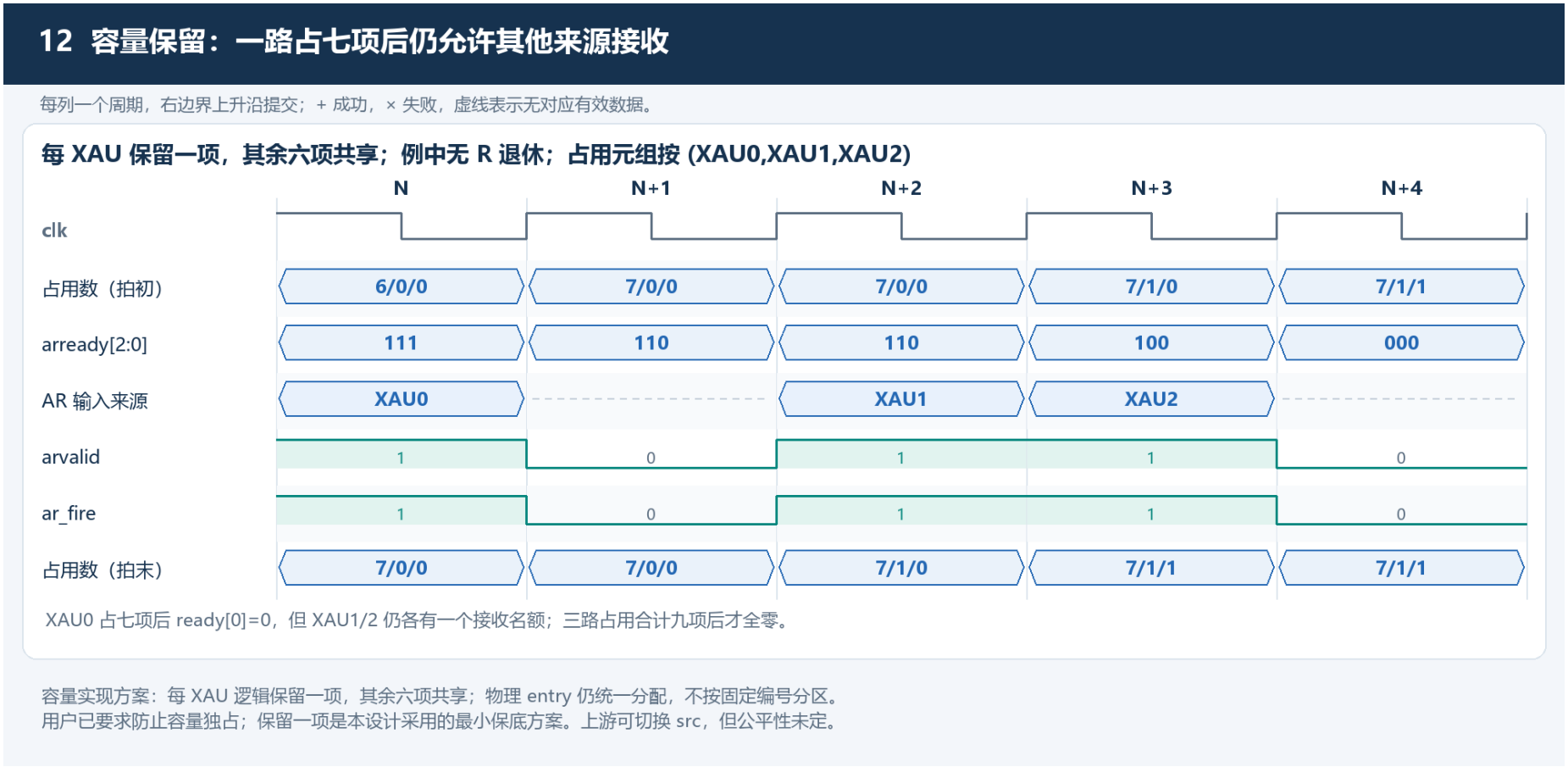


图 12　容量保留：一路占七项后仍允许其他来源接收

图 13　满表复用仍需保留刚释放来源的最低名额

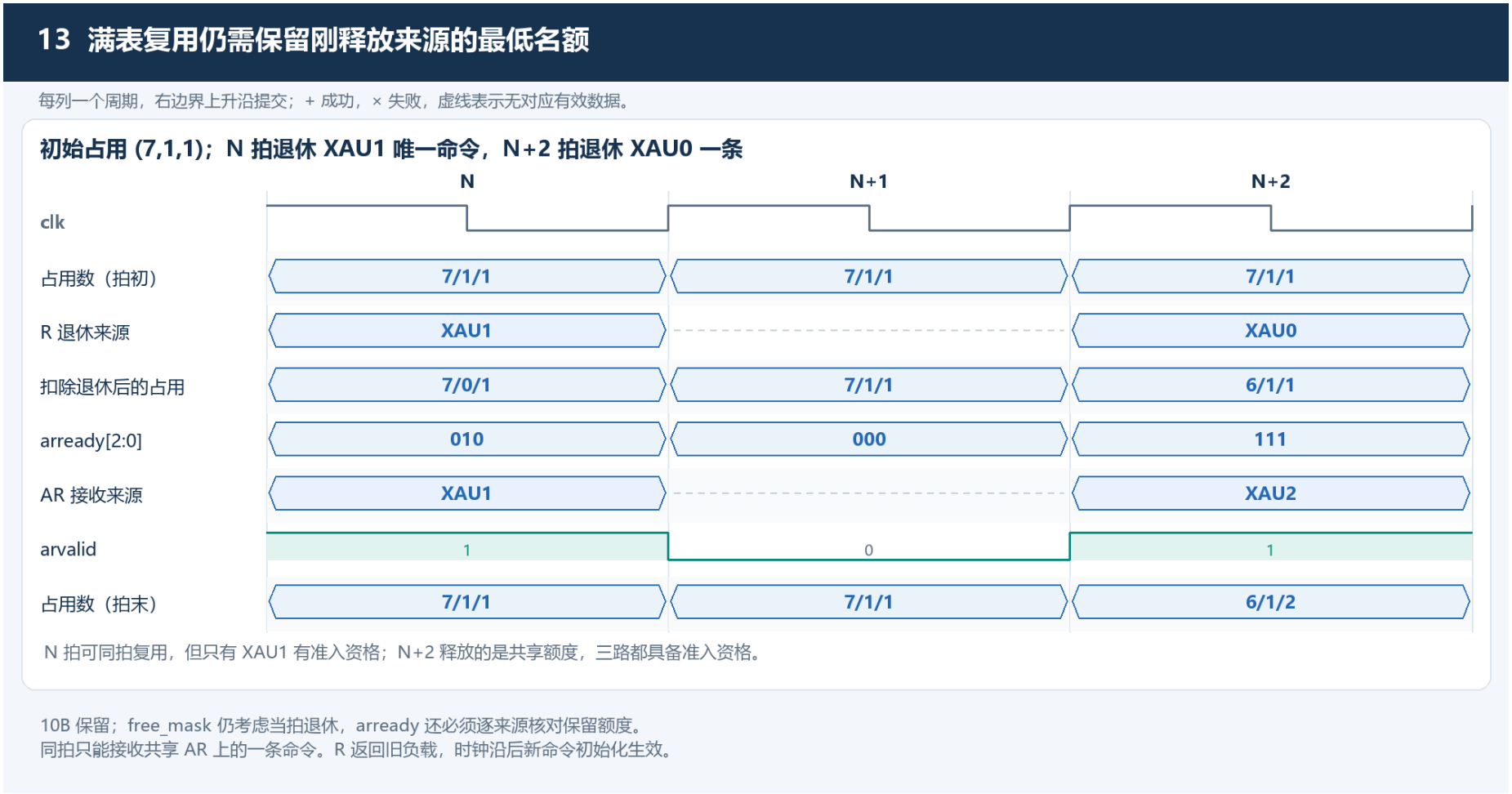


图 13　满表复用仍需保留刚释放来源的最低名额

CMDBUF RD 注错与容量保护清单

内容来源　docs/fault_catalog/cmdbuf_rd_fault_catalog.md

cmdbuf_rd 历史对齐与注错清单

将历史歧义和时序分歧转成可复用的 RTL 注错计划；保留最终规则、错误变体、触发条件和检查点。

版本日期 2026-10-10。包含七项历史问题、49 组注错点、13 张时序图索引，以及 9 项暂不作为强制判据的边界。

容量规则已修订：禁止单个 XAU 独占九项。实现采用每 XAU 保留一项、六项共享，原 FI008 的单路占满测试已被替换。

容量隔离设计与准入公式 · 容量模型检查结果

场景 11 的固定窗口阻塞是正确行为。注错对象是“允许第六条越窗发射”，而不是“第六条持续等待”。

最终基准

- 一个 cycle 内最多接收一条共享 AR、发射一条下游读请求、返回一条共享 R；三个动作可作用于不同命令。
- 地址 18 bit；arid/rid 各 8 bit；port_id 各 5 bit；R raddr 为保存地址低 3 bit。
- Bank=araddr[5:3]、half=araddr[6]；状态接口 [1:0][7:0]，0 表示空闲；Arbit 地址为 [17:6] 共 12 bit。
- CMDBUF 和 OTB 各九项共享、一一对应；接收分配、R 退休释放；成功发射及 MEM 完成不释放。最新要求禁止一路容量独占；本设计用每 XAU 一项保底和六项共享额度实现。
- 最老五条未成功发射命令构窗；成功项以 6B 当拍前馈移出窗口；当前请求项不能重复预选。
- 采用 5A、6B、10B。其错误替代行为分别是取消已预选 B、成功后晚一拍前移窗口、满表释放后晚一拍才允许新接收。
- MEM 返回为单组 128-bit 数据及一位 pc_err；从双 grant 成功拍起固定 +4 拍，仅有效一拍，必须及时捕获。

- MEM 捕获的该条命令最早下一 cycle 参与 R 返回；同 XAU 保序，跨 XAU 在 ready 的完成队头间 RR。
- arresp=11 仍访问 MEM；rresp 优先级为地址越界 11，其次 pc_err 对应 01，正常 00。
- 截图七项均已有后续结论，其中 H06 已被最新防独占要求修订；U01 ~ U09 列出尚不能建立完整错误判据的边界。具体一项保底是设计选择，并非用户指定的配额数字。

原截图七项与最终结论

编号	历史问题	最终规则	注错编号
H01	乱序窗口范围：最老五条未成功发射，还是最老五条未释放。	最老五条未成功发射；先定窗口再匹配 Bank。	FI010、FI011、FI012、FI018、FI019
H02	奇偶 Bank 信息如何下传：原 11-bit [17:7] 丢失 bit 6，曾提出额外 bank_half。	地址改为 12-bit araddr[17:6]，最低位携带奇偶信息。	FI002、FI003、FI004
H03	R 负载是否共享：共享一组，还是每个 XAU 一组。	共享一组；rvalid[2:0] 选择接收对象。	FI005、FI007
H04	MEM 返回几组：合并一组、按八个 Bank，或按十六个 half 分组。	合并后单组 128-bit 数据及一位 pc_err。	FI006、FI028、FI029、FI030
H05	固定四拍起算点：双 grant 成功是否等同于 MEM 同拍接收，中间是否还有流水级。	成功与 MEM 接收同拍，成功后四拍捕获。	FI020、FI025、FI026、FI027

编号	历史问题	最终规则	注错编号
H06	OTB 容量与分配：原先确认九项共享、一一对应、无配额；最新要求禁止 XAU 独占容量。	九项物理池仍共享；本设计采用每 XAU 一项保底、六项共享额度，覆盖原无准入保护结论。	FI008、FI009、FI023、FI039、FI040、FI041、FI043、FI044、FI045、FI046、FI047、FI048、FI049
H07	RR 遇到反压：先选再锁住，还是只在 ready 通道间选。	只在当前 ready 且完成队头可返回的通道中 RR；rvalid 不等待 ready。	FI005、FI032、FI033、FI034、FI035

原始截图

时序场景对照

图	场景	当前行为	注错编号
图 01	连续正常流水	C1 接收、C2 判断、C3 请求；不同命令重叠，C3 成功则 C7 捕获、最早 C8 返回。	FI013、FI014、FI025、FI031
图 02	乱序发射与单 XAU 保序	较新命令可以先完成，但不能越过同 XAU 更老的未返回命令。	FI012、FI030、FI032
图 03	窗口外命令进入时机	第六条等待；只有窗口内成功发射触发 6B 当拍前移，才可补入。	FI011、FI018、FI019
图 04	双 grant 与返回气泡	目标 Bank 两个 grant 同拍为 1 才成功，失败不产生返回标记，四拍日历保留气泡。	FI020、FI025、FI026、FI027
图 05	已选择 5A	A 失败不冲刷同拍选中的 B；B 下一拍发送，A 下一拍按原年龄重新判断。	FI015、FI016、FI017

图	场景	当前行为	注错编号
图 06	已选择 6B	成功信息前馈本拍窗口；不能等下一拍才补入第六条。失败则不前移。	FI018、FI019
图 07	MEM 单拍返回与 R 等待	返回当拍必须保存数据及错误，不受 rready 影响；之后等合格响应机会。	FI028、FI029、FI038
图 08	响应 RR	只在 ready 且完成队头可返回的 XAU 中轮询；图 08 参考模型的环方向与空闲保持判据见 U06。	FI005、FI033、FI034、FI035
图 09	字段透传与错误	12-bit 下游地址、保存字段透传、越界仍访 MEM、越界优先于 parity。	FI002、FI004、FI007、FI036、FI037
图 10	已选择 10B	满表允许同拍 R 退休及 AR 接收；当拍输出旧负载，沿后新分配初始化生效。	FI039、FI040、FI041、FI042
图 11	固定窗口全部阻塞	A ~ E 均忙、F 空闲且状态不变时 F 仍不能发射；这是合法等待，越窗发射才是错误。	FI011、FI022
图 12	入口容量保留	单路占七项后保留其他两路首条接收名额；九项物理池统一分配。	FI008、FI043、FI044、FI045、FI046、FI049
图 13	满表保底复用	释放某来源唯一项时，只为该来源恢复接收资格；释放共享额度时各路均可竞争。	FI039、FI047、FI048

周期约定与判错边界

- 所有周期沿用图集约定：每列一个周期，右边界采样提交。例如 C3 成功、C7 捕获、最早 C8 返回。

- arready==000 时 arvalid==0 是上游环境约束。注错基准驱动器应遵守；违反它的协议负向激励需另定义 DUT 预期。
- 容量保护要求按来源产生 arready，三位可以不同且不要求 one-hot；真正接收看 arready[src]。
- 接口名 arb_addr、ar_port_id、r_port_id 等仅用于区分通道，最终 RTL 命名尚未绑定。
- 窗口公式、free_mask 和内部 entry 标记是等价行为参考，不强制使用年龄矩阵、特定位编码或某种队列结构。
- 每次只植入一个 mutation_variants 子项，并先在无注错基准上通过同一激励。这里记录的是待实施注错计划，不是已执行 RTL 或已测出的缺陷。
- 吞吐类变异可能不产生错误数据，需检查请求间隔、窗口前馈和接收机会；内部一致性类还需可观测内部完成及分配事件。

行为参考逻辑

以下为伪代码，需要按最终 RTL 的状态与采样时刻绑定，不能直接视为可编译 SVA。

```
bank_id = saved_araddr[5:3]
half = saved_araddr[6]
bank_idle = !rd_mem_status[half][bank_id]
issue_success = |(req & ar_rd_gnt & rw_rd_gnt)
pending_mask = valid & ~issued
success_mask = issue_success ? onehot(current_issue_entry) : 0
window_mask = oldest_5(pending_mask & ~success_mask)
candidate_mask = window_mask & bank_idle_mask & ~current_issue_mask
select = oldest(candidate_mask)


free_mask = ~valid | retire_mask
occ_after[x] = occ[x] - retire_for_xau[x]
free_after = 9 - sum(occ_after)
shared_after = sum(max(occ_after[x] - 1, 0))
arready[x] = (free_after > 0) && ((occ_after[x] == 0) || (shared_after < 6))
ar_fire = arvalid && src_is_0_to_2 && arready[src]
occ_next[x] = occ_after[x] + (ar_fire && src == x)
r_candidate[xau] = oldest_unretired_done_before_this_cycle[xau] && rready[xau]
retire_fire = |(rvalid & rready)
```

注错目录

编号	注错主题	类别	对应图
FI001	共享 AR 的接收与源归属	接口与字段 / 功能	01, 09
FI002	Arbit 地址保留奇偶位	接口与字段 / 功能	09
FI003	八个 Bank 的编号解码	接口与字段 / 功能	09
FI004	rd_mem_status 维度与空闲极性	接口与字段 / 功能	03, 09, 11
FI005	共享 R 负载与唯一接收对象	接口与字段 / 功能	08, 09
FI006	单组 MEM 返回接口	接口与字段 / 功能	01, 07, 09
FI007	事务元数据完整保存与透传	接口与字段 / 功能	09
FI008	共享池防止单个 XAU 独占容量	容量与生命周期 / 功能	12
FI009	CMDBUF 与 OTB 原子分配和一一对应	容量与生命周期 / 功能	01, 07, 10
FI010	窗口排除已成功发射但未退休命令	窗口与选择 / 功能	01, 03, 06
FI011	五项全部阻塞时不得越窗选择第六项	窗口与选择 / 功能	11
FI012	窗口内选择最老可用命令	窗口与选择 / 功能	02, 03
FI013	接收判断请求的三拍路径	请求流水与仲裁 / 功能	01, 09
FI014	不同命令之间的流水重叠	请求流水与仲裁 / 吞吐与时序	01

编号	注错主题	类别	对应图
FI015	5A 仲裁失败保留下一条预选	请求流水与仲裁 / 功能	05
FI016	失败重试保留原年龄	请求流水与仲裁 / 功能	05
FI017	屏蔽当前请求级命令的重复预选	窗口与选择 / 功能	05
FI018	6B 成功当拍前移窗口	窗口与选择 / 吞吐与时序	03, 06
FI019	请求失败不能提前腾出窗口名额	窗口与选择 / 功能	05, 06, 11
FI020	目标 Bank 双 grant 同拍成功	请求流水与仲裁 / 功能	04
FI021	无请求时 grant 不得制造成功	请求流水与仲裁 / 功能	04
FI022	待发计数统计全部未成功命令	容量与生命周期 / 功能	01, 04, 11
FI023	命令保留到 R 返回	容量与生命周期 / 功能	07, 10
FI024	MEM 完成同时通知 CMDBUF	容量与生命周期 / 内部一致性	01, 07
FI025	固定四拍从成功接收起算	MEM 返回与捕获 / 功能	01, 04, 09
FI026	失败请求不启动返回计时	MEM 返回与捕获 / 功能	04
FI027	返回标记每拍前进且保留空泡	MEM 返回与捕获 / 功能	04, 07
FI028	MEM 单拍返回必须立即捕获	MEM 返回与捕获 / 功能	07

编号	注错主题	类别	对应图
FI029	pc_err 与零数据不能充当有效信号	MEM 返回与捕获 / 功能	07, 09
FI030	返回标记绑定原命令	MEM 返回与捕获 / 功能	02, 07
FI031	MEM 捕获当拍禁止该命令 R 旁路	R 返回与保序 / 功能	01, 07, 09
FI032	同 XAU 按接收年龄保序	R 返回与保序 / 功能	02
FI033	不同 XAU 不施加全局返回顺序	R 返回与保序 / 功能	08
FI034	RR 只选 ready 的完成队头	R 返回与保序 / 功能	07, 08
FI035	RR 推进与空闲保持	R 返回与保序 / 功能	08
FI036	地址越界命令仍访问 MEM	错误状态 / 功能	09
FI037	越界异常优先于奇偶异常	错误状态 / 功能	09
FI038	pc_err 与所属 OTB 命令一起保存	错误状态 / 功能	07, 09, 10
FI039	10B 满表同拍退休与接收	容量与生命周期 / 吞吐与时序	10
FI040	同槽复用保留旧响应并初始化新命令	容量与生命周期 / 功能	10
FI041	R 退休同时清除两侧归属	容量与生命周期 / 功能	01, 10
FI042	复用新命令仍遵守三拍路径	请求流水与仲裁 / 功能	01, 10

编号	注错主题	类别	对应图
FI043	两个 XAU 联合占用仍为第三路保留名额	容量隔离 / 功能	12
FI044	占用额度统计到真正 R 退休	容量隔离 / 功能	07, 12
FI045	逐来源 ready 判定	容量隔离 / 功能	12
FI046	空闲来源的保底项不可借出	容量隔离 / 功能	12
FI047	满表退休最后一项只恢复该来源保底	容量隔离 / 功能	13
FI048	同拍退休接收的每来源计数合成	容量隔离 / 功能	10, 13
FI049	容量保护不改变五项发射窗口	容量隔离 / 功能	11, 12

注错用例

FI001 共享 AR 的接收与源归属

正确行为：AR 只有一组 arvalid、地址和元数据，每拍最多接收一条；合法 src 为 0～2，接收事件为 arvalid && arready[src]。

错误变体（每次独立植入一项）：

- 把三个 arready 当作三路同时接收许可，为一次 AR 分配多个 entry。
- 使用错误的 src 位或忽略目标 arready 判定握手。

最小触发：连续三拍分别输入 src=0、2、1，各带不同地址和 ID；按实际 arready[src] 记录握手。

检查点：

- 每个实际握手只产生一次分配，归属保存为当拍 src。
- 容量保留策略使 aready 可以为 110、100 或 010，必须检查目标位，不能用 |aready 替代。三位同时为 1 也只接收一条。

关联：图 01、图 09。依据：后续 AR 接口澄清。类型：功能。状态：待实施。

FI002 Arbit 地址保留奇偶位

正确行为：Arbit 地址为 12 bit，内容为 araddr[17:6]，其中最低位就是 araddr[6]。

错误变体（每次独立植入一项）：

- 沿用 11-bit araddr[17:7]。
- 错取 [16:5] 或丢掉 bit 6。
- 地址取自实时 AR 或另一条命令，与 req 指向的命令不一致。

最小触发：发送两条仅 araddr[6] 不同的命令，Bank 编号相同、两个 half 均空闲；再改变 bit 17 检查最高位；发射期间继续输入其他地址。

检查点：

- req 的 Bank bit 应相同，arb_addr[0] 应不同；逐位比较 arb_addr 与该请求级命令保存地址的 [17:6]。无需新增 bank_half 外部端口。

关联：图 09。依据：H02；地址由 [17:7] 修正为 [17:6]。类型：功能。状态：待实施。

FI003 八个 Bank 的编号解码

正确行为：Bank 编号为 araddr[5:3]，范围 0～7；req 为对应 Bank 的 8-bit one-hot 或全零。

错误变体（每次独立植入一项）：

- 将 Bank 编号取成 [6:4]。
- Bank bit 左右反转或偏移一位。
- 沿用 bank0～8 的九 Bank 解释。

最小触发：分别覆盖 Bank 0 和 Bank 7，再覆盖所有中间 Bank；两个 half 各测一次。

检查点：

- 有请求时 req == (8'b1 << saved_araddr[5:3])，且 $onehot0(req)；编号由地址决定而不是物理 entry 下标。

关联：图 09。依据：Bank 0～7 修正。类型：功能。状态：待实施。

FI004 rd_mem_status 维度与空闲极性

正确行为：rd_mem_status 为 [1:0][7:0]；[0][bank] 偶、[1][bank] 奇；0 空闲、1 忙。

错误变体（每次独立植入一项）：

- 按 [bank][half] 或旧端口布局索引。
- 交换奇偶 half。
- 把 1 判为空闲。

最小触发：使用非对称状态：只令 rd_mem_status[0][1]=1，再只令 [1][6]=1；分别提交对应地址及相邻空闲地址。

检查点：

- bank_idle = !rd_mem_status[saved_araddr[6]][saved_araddr[5:3]]；非对称模式必须能区分转置、half 交换与极性翻转。

关联：图 03、图 09、图 11。依据：rd_mem_status 端口修正；H02。类型：功能。状态：待实施。

FI005 共享 R 负载与唯一接收对象

正确行为：R 共享一组 128-bit 数据及 rid、port_id、raddr、rresp，通过 rvalid[2:0] 指示唯一接收 XAU。

错误变体（每次独立植入一项）：

- 同拍对多个 XAU 拉高 valid。
- valid 选中 XAU1，但负载 mux 取 XAU0。

最小触发：至少两个 XAU 同时有完成队头且 ready=1，给各命令不同的全部元数据与数据。

检查点：

- $onehot0(rvalid)。
- 所有共享 R 字段必须属于唯一获选命令；不能只核对 rdata 而漏掉 rid、port_id、raddr、rresp。

关联：图 08、图 09。依据：H03。类型：功能。状态：待实施。

FI006 单组 MEM 返回接口

正确行为：MEM 返回是合并后的一组 128-bit 数据和一位 pc_err；cmdbuf_rd 不接收每 Bank 独立的返回负载。

错误变体（每次独立植入一项）：

- 按 Bank 或 half 再错误选择一组不存在的返回总线。
- 只存储或返回 128-bit 数据的一部分。
- 将 pc_err 当作多位状态。

最小触发：从不同 Bank 发射，按成功发射 +4 拍驱动共享返回；数据高低 64 bit 使用不同且非零的图案。

检查点：

- 按内部返回标记写入对应 OTB，正常响应逐位比较 128-bit 数据；pc_err 仅按一位保存和判错。

关联：图 01、图 07、图 09。依据：H04；数据口 128 bit 与 pc_err 1 bit。类型：功能。状态：待实施。

FI007 事务元数据完整保存与透传

正确行为：rid=原 arid[7:0]，R port_id=原 AR port_id[4:0]，raddr=原 araddr[2:0]；都从命令保存值读取。

错误变体（每次独立植入一项）：

- 漏存 ID 或 port_id。
- 位宽截断或 raddr 切片错误。
- 响应使用实时 AR 输入而非保存字段。

最小触发：使用 ID=8'hA5、port_id=5'h13、araddr=18'h12345；响应前持续输入不同元数据。

检查点：

- 该命令返回 rid=A5、port_id=13、raddr=3'b101，且不受后续 AR 改变影响。

关联：图 09。依据：arid/rid 8 bit；port_id 5 bit；raddr 透传低 3 bit。类型：功能。状态：待实施。

FI008 共享池防止单个 XAU 独占容量

正确行为：最新要求禁止一路占满并挡住其他来源。本设计采用每 XAU 保留一项、其余六项共享；单路最多七项，物理槽位仍统一分配。

错误变体（每次独立植入一项）：

- 沿用无保护的全局共享，允许单 XAU 占九项。
- 有保留空位却将所有 aready 同时压低。

最小触发：不允许退休，仅由 XAU0 接收七条；然后检查 XAU1 和 XAU2 的接收资格，并在上游调度给出相应 src 时各接收一条。

检查点：

- 占用为 (7,0,0) 时 aready=110；(7,1,0) 时为 100；(7,1,1) 时为 000。不能用单路占满九项作为正常容量测试。

关联：图 12。依据：H06 新要求覆盖旧无配额结论；防止 XAU 容量独占。类型：功能。状态：待实施。

FI009 CMDBUF 与 OTB 原子分配和一一对应

正确行为：每次 AR 接收同时保留一个 CMDBUF 项和对应 OTB 项，整个生命周期绑定同一命令。

错误变体（每次独立植入一项）：

- 只分配 CMDBUF，等 MEM 返回时才申请 OTB。
- 两边独立选空位而丢失对应关系。

最小触发：并发发生新接收、旧命令返回、另一命令 R 退休，覆盖表尾与复用槽位。

检查点：

- 每条有效命令始终具有唯一对应的 CMDBUF 与 OTB 存储；返回时已有位置，不受当拍容量影响。物理编号是否相同属于实现方式，必须保持可验证的一一映射。

关联：图 01、图 07、图 10。依据：H06；接收时申请 OTB。类型：功能。状态：待实施。

FI010 窗口排除已成功发射但未退休命令

正确行为：窗口是最老五条未成功发射命令，而不是最老五条未释放命令。

错误变体（每次独立植入一项）：

- 已发射或已完成、尚未 R 返回的命令继续占乱序窗口名额。

最小触发：较老的五条已成功发射且仍驻留，较新待发命令的 Bank 空闲。

检查点：

- 较新待发命令必须进入待发窗口；不能被已成功发射项阻挡。

关联：图 01、图 03、图 06。依据：H01。类型：功能。状态：待实施。

FI011 五项全部阻塞时不得越窗选择第六项

正确行为：先取最老五条待发命令，再在窗口中筛空闲 Bank；窗口不因空命中或等待时间而自动滑动。

错误变体（每次独立植入一项）：

- 先筛全九项的空闲 Bank，再取最老。
- 五项均忙时滑动窗口或扩大到六项。
- 超时后强行发射 F。

最小触发：A～F 按年龄排列，请求级为空；A～E 均访问 Bank0 偶，F 访问 Bank1 偶；rd_mem_status 恒为 16'h0001。

检查点：

- F 必须一直留在窗口外；无其他请求级成功时，req=0。

- 这是正确的窗口阻塞，不得以 F 未最终返回判定 DUT 错误。若继续接收且无退休，容量达到九项后背压。

关联：图 11。依据：H01；最新第六条命令场景。类型：功能。状态：待实施。

FI012 窗口内选择最老可用命令

正确行为：从窗口内满足 Bank 空闲且未被当前请求级占用的命令中选接收年龄最老的一条。

错误变体（每次独立植入一项）：

- 只看全局队头，队头忙则停止。
- 按物理 entry 下标、ID 数值或 Bank 编号优先。

最小触发：窗口内 A 忙，B、C 空闲，A 比 B 老、B 比 C 老；通过槽位复用让 B 的物理编号大于 C，ID 使用非单调值。

检查点：

- 应选择 B；更改 entry 位置或外部 ID 不改变接收年龄优先级。

关联：图 02、图 03。依据：最老匹配选择规则；ID 与接收年龄区分。类型：功能。状态：待实施。

FI013 接收判断请求的三拍路径

正确行为：最早 C1 接收、C2 判断、C3 请求下游；当拍 grant 决定 C3 是否成功。

错误变体（每次独立植入一项）：

- 新 AR 直接进入当拍选择。
- 选择结果同拍组合发 req。

最小触发：空表、目标 Bank 空闲、下游始终允许成功，C1 输入唯一命令 A。

检查点：

- C1/C2 不得为 A 发 req；正常无阻塞流水在 C3 发 req，不能把 C3 尝试等同于无条件成功。

关联：图 01、图 09。依据：固定三 cycle 要求。类型：功能。状态：待实施。

FI014 不同命令之间的流水重叠

正确行为：请求 A 的同拍可判断 B、接收 C；无阻塞时每拍可成功发射一条。

错误变体（每次独立植入一项）：

- 将接收、判断、请求做成整机串行状态机，或等待上一命令 MEM 返回才处理下一条。

最小触发：C1/C2/C3 连续接收 A/B/C，相关 Bank 空闲且每次双 grant 成功。

检查点：

- 应能形成 C3/C4/C5 连续请求；记录多余气泡，即使最终数据都正确也要报告吞吐偏离。

关联：图 01。依据：跨命令流水要求。类型：吞吐与时序。状态：待实施。

FI015 5A 仲裁失败保留下一条预选

正确行为：C3 请求 A 失败而同拍选中 B 时，C4 仍请求 B；A 在 C4 重新参加判断。

错误变体（每次独立植入一项）：

- A 失败冲刷 B。
- A 在失败当拍重选并替换 B。

最小触发：A、B、C 均可匹配；C3 只让 A 的请求失败，其他请求成功。

检查点：

- 请求序列为 C3 A 失败、C4 B、C5 A 重试；A 的恢复不得取消已选 B。

关联：图 05。依据：用户选择 5A。类型：功能。状态：待实施。

FI016 失败重试保留原年龄

正确行为：失败命令重新成为候选，但相对接收年龄不变。

错误变体（每次独立植入一项）：

- 重试时把 A 插到队尾。
- 更新 A 的年龄为当前时间。

最小触发：A 比 B、C 老，A 失败时 B 已预选；下一拍 B 请求，A/C 都可用。

检查点：

- 判断级应先重新选 A 而不是 C；已预选 B 仍按 5A 发送。

关联：图 05。依据：失败后新老相对属性不变。类型：功能。状态：待实施。

FI017 屏蔽当前请求级命令的重复预选

正确行为：当前请求级命令不能在同拍再次被判断级选中。

错误变体（每次独立植入一项）：

- 候选掩码漏掉 current_issue_entry 排除项。

最小触发：A 正在请求且仍为最老可用项，让本次请求失败以避免 issued 成功前馈掩盖错误；记录下一拍 req，随后重试成功。

检查点：

- 失败当拍不能再选 A，最早下一拍才重判；同一命令最多成功发射一次，不得生成重复返回标记。

关联：图 05。依据：5A 流水占用语义。类型：功能。状态：待实施。

FI018 6B 成功当拍前移窗口

正确行为：A 当拍成功时，把成功信息前馈给本拍窗口计算；第六条 F 可以当拍进入窗口并被选中。

错误变体（每次独立植入一项）：

- 只等下一拍 issued 寄存器更新才前移窗口。

最小触发：A 正在请求，A～F 为最老六条待发命令；B～E 忙、F 空闲；A 在 N 拍成功。

检查点：

- N 拍选 F，N+1 发 F；等到 N+1 才选 F 会产生额外一拍气泡。

关联：图 03、图 06。依据：用户选择 6B。类型：吞吐与时序。状态：待实施。

FI019 请求失败不能提前腾出窗口名额

正确行为：未成功的当前请求级命令仍计入最老五条窗口；排除重复选择不等于将其从待发窗口移除。

错误变体（每次独立植入一项）：

- 不论 grant 成败，在构窗之前移除请求级命令。
- 预选就置 issued。

最小触发：A 正在请求且失败；A ~ F 按年龄排列，B ~ E 忙、F 空闲。

检查点：

- 失败当拍不得选 F；应先构 A ~ E 窗口，再屏蔽 A 的重复预选。

关联：图 05、图 06、图 11。依据：5A 与 6B 组合语义。类型：功能。状态：待实施。

FI020 目标 Bank 双 grant 同拍成功

正确行为：issue_success = |(req & ar_rd_gnt & rw_rd_gnt)，三者必须命中同一个 Bank bit。

错误变体（每次独立植入一项）：

- 两个 grant 用 OR。
- 分别归约两个 grant 后再 AND。
- 记住上一拍的一个 grant 与本拍另一个拼成成功。

最小触发：req[0]=1；分别覆盖目标 grant=00、01、10、11，两个 grant 分居 Bank0/Bank1，以及两个 grant 分在相邻周期。

检查点：

- 只有同拍目标位 11 成功；其余情况均不置 issued、不减少待发数、不生成返回标记。

关联：图 04。依据：双 grant 成功定义。类型：功能。状态：待实施。

FI021 无请求时 grant 不得制造成功

正确行为：成功事件受本模块当前有效 req 限定；无 req 时不会产生本模块的成功命令。

错误变体（每次独立植入一项）：

- 成功逻辑只看两个 grant 向量，忽略 req。

最小触发：req=0 时驱动 grant 非零。若集成环境明确保证此时 grant 必为零，则改为内部组合逻辑变异检查，不把违反环境保证的激励当正常用例。

检查点：

- 不得出现 issued 改变、待发数减少或 +4 拍虚假捕获。

关联：图 04。依据：双 grant 成功定义。类型：功能。状态：待实施。

FI022 待发计数统计全部未成功命令

正确行为：pending_cnt 为 4 bit、范围 0～9；接收 +1，成功发射 −1；预选、失败、MEM 返回和 R 退休不直接改变待发数。

错误变体（每次独立植入一项）：

- 计数封顶为窗口大小 5。
- 预选或 req 就减计数。
- 已发射驻留项仍计入待发。

最小触发：先积累六条未发射命令，再分别制造预选、失败、成功，以及同拍接收与成功。

检查点：

- pending_next=pending+ar_fire-issue_success；同拍接收与成功净变化为零，六条待发时值为 6。计数采样统一为时钟沿前或沿后。

关联：图 01、图 04、图 11。依据：Arbit 待发指示口定义。类型：功能。状态：待实施。

FI023 命令保留到 R 返回

正确行为：成功发射后及 MEM 完成后，命令仍占 CMDBUF 和 OTB；真正 R 返回才退休。

错误变体（每次独立植入一项）：

- 发射成功即清 CMDBUF。
- MEM 返回即释放 CMDBUF 或 OTB。

最小触发：A 发射成功并收到 MEM 数据，但令所属 XAU 的 rready=0，延后多拍才允许返回。

检查点：

- 等待期间 A 两侧存储均保持有效归属；不能把它的槽位当空位接收新命令。

关联：图 07、图 10。依据：完整命令生命周期。类型：功能。状态：待实施。

FI024 MEM 完成同时通知 CMDBUF

正确行为：MEM 返回写 OTB 时，CMDBUF 同时感知对应命令完成；具体完成标志命名和编码由实现决定。

错误变体（每次独立植入一项）：

- 只写 OTB 而丢失 CMDBUF 完成通知。
- 通知错误的 entry。

最小触发：乱序成功发射两条不同命令，按 +4 拍顺序返回，响应先暂缓。

检查点：

- 内部监视对应命令的完成事件或等价状态变化。仅看最终 R 波形可能区分不了此变异，需要内部观测点。

关联：图 01、图 07。依据：MEM 返回也被 CMDBUF 感知。类型：内部一致性。状态：待实施。

FI025 固定四拍从成功接收起算

正确行为：成功发射与 MEM 接收同拍；在 C3 成功则 C7 捕获该命令返回。

错误变体（每次独立植入一项）：

- 从 AR 或预选拍开始计时。
- 延迟写成 3 或 5 拍。
- 在 Arbit 与 MEM 之间再虚构一拍。

最小触发：唯一命令 C3 成功，C7 提供唯一有效图案；C6/C8 总线使用不同无关值。

检查点：

- 仅 C7 对该 entry 发生有效捕获；时间以统一采样沿计，不把包含首尾的五列误认为延迟五拍。

关联：图 01、图 04、图 09。依据：H05。类型：功能。状态：待实施。

FI026 失败请求不启动返回计时

正确行为：返回标记只由成功发射产生，失败尝试没有对应 MEM 返回。

错误变体（每次独立植入一项）：

- 每次 req 或预选均压入四拍标记流水。

最小触发：A 在 N 拍失败、N+2 拍重试成功；MEM 只在 N+6 返回 A。

检查点：

- N+4 不得因失败尝试而捕获或置完成，N+6 才捕获一次。

关联：图 04。依据：双 grant 与固定四拍关系。类型：功能。状态：待实施。

FI027 返回标记每拍前进且保留空泡

正确行为：固定延迟流水每个时钟推进，空泡也推进，不受新请求、表满或 R ready 控制。

错误变体（每次独立植入一项）：

- 只在有成功发射时推进。
- 表满或 rready=0 时暂停计时。
- 将空泡压缩掉。

最小触发：C3 A 成功、C4/C5 无成功、C6 B 成功；期间安排表满或全部 R 不 ready。

检查点：

- C7 捕获 A，C10 捕获 B；C8/C9 不得挤入 B 的完成标记。

关联：图 04、图 07。依据：固定四拍及单拍返回。类型：功能。状态：待实施。

FI028 MEM 单拍返回必须立即捕获

正确行为：返回数据和 pc_err 只保证一个 cycle 有效；该拍写入 OTB，与 R 是否 ready 无关。

错误变体（每次独立植入一项）：

- 只在 rready=1 时写入。
- 晚一拍再读 MEM 数据。
- 响应直接读取届时的实时 MEM 总线。

最小触发：C7 返回 A，C7/C8 rready=000，C9 允许目标通道响应；C8 起 MEM 总线改变。

检查点：

- C9 响应使用 C7 保存的内容；正常 rresp=00 的子例精确比较数据，错误子例至少比较错误状态，错误时 rdata 语义见 U05。

关联：图 07。依据：MEM 返回只持续一拍。类型：功能。状态：待实施。

FI029 pc_err 与零数据不能充当有效信号

正确行为：捕获有效性来自成功发射延迟四拍的标记；pc_err=0 和全零数据也可能是合法返回。

错误变体（每次独立植入一项）：

- pc_err=1 才捕获。
- rdata 非零才置完成。

最小触发：成功命令返回 128'b0 且 pc_err=0；目标 XAU 队首并 ready。

检查点：

- 命令必须正常完成，并在后续允许响应时返回零数据和 rresp=00。

关联：图 07、图 09。依据：pc_err 仅为一位奇偶校验异常。类型：功能。状态：待实施。

FI030 返回标记绑定原命令

正确行为：每个成功发射事件保存可定位原 CMDBUF/OTB 命令的内部标识，四拍后数据及错误写回该命令。

错误变体（每次独立植入一项）：

- 使用当前请求级下标或当前 AR 下标。

- 按接收 FIFO 次序而非实际发射次序匹配。
- 直接把外部 arid 当物理槽号。

最小触发：同 XAU 的较新 B 先于 A 发射；返回期间继续接收 C/D，并给各正常返回不同数据图案。

检查点：

- B 的返回写 B、A 的返回写 A；最终 R 仍 A 后 B，不能将交换的数据掩盖为正确顺序。

关联：图 02、图 07。依据：乱序发射与 OTB 对应关系。类型：功能。状态：待实施。

FI031 MEM 捕获当拍禁止该命令 R 旁路

正确行为：C7 捕获 A，A 最早 C8 才能向 R 返回；该限制针对 A，不禁止 C7 返回其他此前已完成命令。

错误变体（每次独立植入一项）：

- 用本拍 capture_valid 或 MEM 数据组合直通 R 仲裁和负载。

最小触发：A 为唯一未返回命令，位于所属 XAU 队首，rready 始终为 1，C7 MEM 返回。

检查点：

- C7 不得返回 A；C8 可具备响应资格，覆盖最早下一拍路径。
- 不能把 C8 必然返回作为所有场景的断言；前序命令、ready 和 RR 可使响应延后。额外固定响应级是否允许不由本条单独裁定。

关联：图 01、图 07、图 09。依据：用户确认 MEM 最早下一 cycle 返回 R。类型：功能。状态：待实施。

FI032 同 XAU 按接收年龄保序

正确行为：同一 XAU 的 R 按 AR 接收顺序返回，跨 arid 也保序；ID 是透传标签，不能当年龄。

错误变体（每次独立植入一项）：

- 按完成顺序返回。
- 仅同 ID 保序。

- 按 ID 数值升序返回。

最小触发：同 XAU 先接收 A(id=A5)、后接收 B(id=03)，但使 B 先完成；另用反向 ID 配置重复该年龄关系。

检查点：

- A 未返回前不能返回 B；基准队列以握手次序建模，不以 ID 排序。重复 ID 的合法性尚未明确，基础用例使用不同 ID。

关联：图 02。依据：单 XAU 保序；arid/rid 补充。类型：功能。状态：待实施。

FI033 不同 XAU 不施加全局返回顺序

正确行为：每个 XAU 独立维护最老未返回命令；某通道队头未完成不妨碍另一通道完成队头参加 RR。

错误变体（每次独立植入一项）：

- 只允许全局最老未返回命令响应。

最小触发：较老 A 属于 XAU0 且未完成；较新 B 属于 XAU1 已完成，rready[1]=1。

检查点：

- B 是合格 RR 候选；不能因 A 阻塞而禁止 XAU1 响应。若有其他合格通道，按 RR 决定获选者。

关联：图 08。依据：按单 XAU 保序并 RR 返回。类型：功能。状态：待实施。

FI034 RR 只选 ready 的完成队头

正确行为：通道候选 = 该 XAU 最老未返回命令已完成 && rready[src]；rvalid 只在对应 ready=1 时拉高。

错误变体（每次独立植入一项）：

- 先选不 ready 通道并锁定共享 R。
- valid 先拉高再等 ready。
- 同通道跳过未完成队头，选较新的完成项。

最小触发：XAU0 有完成队头但不 ready，XAU1 队头完成且 ready；另测 XAU0 队头未完成、第二条已完成。

检查点：

- (rvalid & ~rready)==0。
- 第一子例应服务 XAU1；第二子例 XAU0 不得因后项已完成而成为候选。

关联：图 07、图 08。依据：H07；ready 门控 rvalid 约束。类型：功能。状态：待实施。

FI035 RR 推进与空闲保持

正确行为：已确认合格通道间采用 RR；图 08 的参考模型约定沿固定环在服务后推进、无响应时保持后续优先级。精确环方向与空闲推进策略见 U06。

错误变体（每次独立植入一项）：

- 以固定优先级替代 RR，持续压制其他合格通道。
- 每拍推进并改变后续可见服务顺序；此变异按图 08 参考模型判定前需确认 U06。
- 按错误的获选通道推进，造成合格通道被跳过或长期偏置。

最小触发：先观察一次已知响应建立指针，再让三个 XAU 持续具有 ready 的完成队头；中间插入全无候选拍。

检查点：

- 持续三个合格通道时检查轮转服务而非固定优先级。图 08 按 XAU0→1→2 及空闲保持绘制，精确序列检查需先确认 U06；比较可见服务顺序，不仅凭内部指针编码变化判错。

关联：图 08。依据：H07；RR 返回规则。类型：功能。状态：待实施。

FI036 地址越界命令仍访问 MEM

正确行为：arresp=2'b11 的请求仍缓存、参与窗口、仲裁及 MEM 返回，最终返回地址越界状态。

错误变体（每次独立植入一项）：

- 接收后本地立即完成越界响应。
- 丢弃越界命令，不发送 MEM。

最小触发：输入 arresp=11、目标 Bank 空闲的命令，允许双 grant，按 +4 拍返回。

检查点：

- 应有正常下游请求和返回捕获；R 不得绕过 MEM 生命周期提前返回，最终 rresp=11。错误响应数据内容见 U05。

关联：图 09。依据：用户确认越界仍发 MEM。类型：功能。状态：待实施。

FI037 越界异常优先于奇偶异常

正确行为：arresp=11 时 rresp=11；否则已定义的 arresp=00 下，pc_err=1 返回 01，pc_err=0 返回 00。

错误变体（每次独立植入一项）：

- pc_err 覆盖越界状态。
- 用位运算拼接成未定义编码。
- 忽略任一错误来源。

最小触发：覆盖 (arresp,pc_err)=(00,0)、(00,1)、(11,0)、(11,1)。

检查点：

- 期望 rresp 分别为 00、01、11、11。不为 arresp=01/10 自行定义结果。

关联：图 09。依据：错误优先级最终确认。类型：功能。状态：待实施。

FI038 pc_err 与所属 OTB 命令一起保存

正确行为：每条命令保存其返回拍的 pc_err，R 使用该保存状态。

错误变体（每次独立植入一项）：

- 使用实时 pc_err。
- 所有 entry 共用一个最后错误寄存器。
- 只清数据不清复用项的错误归属。

最小触发：A 返回 pc_err=1 后暂缓响应，B 随后返回 pc_err=0；两条 arresp 都为 00。

检查点：

- A 的 rresp=01、B 的 rresp=00，响应时实时 MEM pc_err 的变化不影响结果。

关联：图 07、图 09、图 10。依据：OTB 保存 pc_err。类型：功能。状态：待实施。

FI039 10B 满表同拍退休与接收

正确行为：九项全占用时，当拍 R 退休可使符合来源保留额度的新 AR 同拍接收；不能无条件对三个来源都拉高 ready。

错误变体（每次独立植入一项）：

- free 判断只看拍初 valid，不包含当拍 retire，导致额外停一拍。

最小触发：占用为 (7,1,1)，XAU0 当拍退休一条，释放一个共享额度；上游按目标 ready 发送新命令。另测 XAU1 唯一命令退休，参见 FI047。

检查点：

- 应允许该拍接收；used_next=9+1-1=9。若变异错误压低 ready，应记录服务机会丢失，不能强行违反 arready=000 时 arvalid=0 的环境约束。

关联：图 10。依据：用户选择 10B。类型：吞吐与时序。状态：待实施。

FI040 同槽复用保留旧响应并初始化新命令

正确行为：复用拍 R 输出旧命令；时钟沿后新分配有效，初始化优先于退休清除，具有新元数据、最年轻年龄、未发射未完成状态。

错误变体（每次独立植入一项）：

- 新写入提前覆盖旧 R 负载。
- 退休清除覆盖新 valid。
- 沿用旧 issued、done、pc_err 有效归属或年龄。

最小触发：10B 满表复用同一逻辑槽，旧 A 与新 B 的 src、地址、ID、port 和错误情况均不同。

检查点：

- 当拍只返回 A；随后 B 仍有效且须经历自己的发射和捕获；B 不得凭旧完成状态直接响应，也不能继承 A 的年龄。

关联：图 10。依据：10B 同拍事件优先级。类型：功能。状态：待实施。

FI041 R 退休同时清除两侧归属

正确行为：实际 R 返回后，旧命令从 CMDBUF 和 OTB 同时移除，旧数据不再有效；同拍新分配仍按 10B 生效。

错误变体（每次独立植入一项）：

- 只清 CMDBUF 或只清 OTB。
- 清错 entry。
- 退休后旧命令再次参加 R。

最小触发：两个不同命令驻留，只返回其中一条；分别覆盖无复用和同拍复用。

检查点：

- 只有获选命令的旧归属失效，每次 AR 接收最多有一次 R 返回；另一命令保持。物理数据寄存器是否必须写零见 U04，不以残留无效位判错。

关联：图 01、图 10。依据：用户定义完整生命周期。类型：功能。状态：待实施。

FI042 复用新命令仍遵守三拍路径

正确行为：10B 释放复用只提前资源可用性，不让新 AR 越过输入寄存；新命令下一拍才判断。

错误变体（每次独立植入一项）：

- 将退休槽位的新 AR 组合直送当拍判断级。

最小触发：N 拍满表退休 A 并接收 B，B 的目标 Bank 空闲；保持后续 grant 可用。

检查点：

- B 最早 N+1 判断、N+2 请求；不得在 N 或 N+1 提前发 B。

关联：图 01、图 10。依据：10B 与固定三拍组合。类型：功能。状态：待实施。

FI043 两个 XAU 联合占用仍为第三路保留名额

正确行为：每个零占用 XAU 都保留一个物理空位；sum(max(occ[x],1))<=9。只设单路上限七项不足。

错误变体（每次独立植入一项）：

- 仅检查每路 occ<=7，允许到达 (4,5,0)。

最小触发：无退休，将占用推进至 (4,4,0)，分别尝试再接收 XAU0/1，并检查 XAU2 首条请求。

检查点：

- 在 (4,4,0) 时 arready=100；只能由 XAU2 使用最后一项，禁止进入 (5,4,0) 或 (4,5,0)。

关联：图 12。依据：每 XAU 一项逻辑保留方案。类型：功能。状态：待实施。

FI044 占用额度统计到真正 R 退休

正确行为：occ[x] 统计该来源全部已接收且未退休命令，覆盖未发射、已发射及已完成状态。

错误变体（每次独立植入一项）：

- 使用 pending_cnt 代替总占用。
- 在发射成功或 MEM 完成时归还额度。

最小触发：XAU0 占七项后，让部分命令发射或返回 MEM，但保持 rready[0]=0。

检查点：

- occ[0] 仍为七；没有 R 退休时不能重新给 XAU0 准入，也不能侵占 XAU1/2 的保底空位。

关联：图 07、图 12。依据：容量保留与完整生命周期。类型：功能。状态：待实施。

FI045 逐来源 ready 判定

正确行为：arready 每位表示该 XAU 的准入资格，实际接收只认 arready[src]。

错误变体（每次独立植入一项）：

- 将有物理空位广播成 ready=111。

- 用 arvalid && |arready 接收。

最小触发：占用为 (7,0,0)，ready 应为 110；输入 src=0、arvalid=1 时不握手。该刺激满足 ready 非全零的既有约束；再切换 src=1 并发送。

检查点：

- 不得为被禁止的 XAU0 分配 entry；切换到 XAU1 后可接收。上游已确认允许切换，但尚不保证公平性。

关联：图 12。依据：共享 AR 与容量隔离。类型：功能。状态：待实施。

FI046 空闲来源的保底项不可借出

正确行为：其他来源暂时无请求时也不借出其最后一个保底名额，否则长期不退休时无法回收。

错误变体（每次独立植入一项）：

- 根据当前 src 或短期未观察到其他来源请求，允许一路借满九项。

最小触发：只有 XAU0 长时间请求且不退休，稍后 XAU1 才开始请求。

检查点：

- XAU0 在七项处停止准入；XAU1 首条请求无需等待 XAU0 退休即可拥有 entry。

关联：图 12。依据：最小保底方案。类型：功能。状态：待实施。

FI047 满表退休最后一项只恢复该来源保底

正确行为：满表 (7,1,1) 中 XAU1 唯一命令退休后，扣除退休占用为 (7,0,1)，仅 XAU1 可使用空位。

错误变体（每次独立植入一项）：

- 发现 retire 就令 ready=111，允许其他来源抢走刚恢复的保底。

最小触发：在该满表分布下退休 XAU1，并分别观察或尝试 src=0、1、2。

检查点：

- 该拍 aready=010；XAU1 可按 10B 同拍复用，其他来源不能抢占。

关联：图 13。依据：10B 与容量保留组合。类型：功能。状态：待实施。

FI048 同拍退休接收的每来源计数合成

正确行为：先扣除当拍退休再算准入，最终 occ_next[x]=occ[x]-retire[x]+accept[x]；同源两个事件净值为零。

错误变体（每次独立植入一项）：

- 用 if/else 丢失其中一个事件。
- 退休与接收属于不同 src 时却更新同一计数。
- ready 反过来依赖本拍新分配，形成自依赖。

最小触发：分别覆盖 (7,1,1) 下 XAU1 退休并接收 XAU1，以及 XAU0 退休并接收 XAU2。

检查点：

- 结果依次为 (7,1,1) 和 (6,1,2)；总占用均九，保留不变量保持。退休只由已有命令的 R 响应决定。

关联：图 10、图 13。依据：每来源计数与 10B。类型：功能。状态：待实施。

FI049 容量保护不改变五项发射窗口

正确行为：为其他 XAU 保留的命令可进入池，但仍必须遵守最老五条未成功发射窗口。入口可用不等于下游一定有进展。

错误变体（每次独立植入一项）：

- 因为新命令使用保底项就允许其越过五项窗口。
- 将跨 XAU 公平当作发射窗口的额外绕过条件。

最小触发：XAU0 的 A～E 占最老五条且 Bank 均忙；XAU1 的 F 经保底接收且 Bank 空闲，状态保持。

检查点：

- XAU1 可以完成 AR 接收，但 F 仍不能越窗发射；不得把这一正常窗口等待判为容量独占错误。

关联：图 11、图 12。依据：容量隔离与既有窗口规则。类型：功能。状态：待实施。

暂不作为强制判据的边界

编号	主题	判据限制
U01	复位协议	复位极性、同步或异步、途中未完成请求与四拍返回标记的处置均未定义。不能要求自行选定的恢复行为。
U02	非法输入编码	src=3、arresp=01/10 的处理未定义；基础环境限定 src 为 0～2、arresp 为 00/11。
U03	重复 ID 与唯一性	重复 arid、跨 XAU 相同 ID 及 port_id 约束未明确；基准不以 ID 作为唯一键，基础激励可采用不同且非单调 ID。
U04	清除是否要求物理写零	已确认退休后移除命令及数据有效归属；是否要求 128-bit 存储内容实际清零尚未区分。暂不据无效存储残值判错。
U05	错误响应的数据内容	rresp 的优先级已确认；越界或 pc_err=1 时 rdata 是否仍透传捕获值、强制清零或不关心，尚无独立明确约束。正常响应精确比数据，错误响应先判状态及归属。
U06	RR 精确推进规则	合格通道间 RR 已确认；图 08 的 0→1→2、服务后推进和空闲保持属于参考模型约定，初始起点及精确序列尚未单独确认。可先检查轮转与合法候选；以空闲后首个获选者判错前需确认策略，内部指针编码变化本身不构成错误。
U07	判断后 Bank 状态变化	若判断拍空闲、请求拍转忙，是否再次检查状态并撤销 req 未独立对齐。双 grant 是成功的最终判据；状态变化子例不先设定撤销策略。
U08	无效输出与进展时限	req/rvalid 无效拍的 payload 值、grant 和 ready 的公平性、最长等待上限未定义。场景 11、永久不 ready 或长期仲裁失败均不能无条件要求最终响应。

编号	主题	判据限制
U09	共享 AR 上游公平性	用户已确认目标 ready=0 时可以切换 src，但公平性未定。cmdbuf_rd 保证保留空位和对应 ready，不能强迫上游选择另一来源；最终一定接收仍需公平调度前提。

后续基准模型与注错记录

1. 每次合法 AR 握手分配单调递增的测试序号 accept_seq，保存 src、地址、ID、port、arresp；以 accept_seq 标识命令，而不是以外部 ID 标识。
2. 为每个 XAU 保存接收序列及全部未退休占用 occ；共享池保存占用和待发集合，检查 sum(max(occ[x],1))<=9；根据年龄与窗口规则算判断候选，再寄存供下一拍请求。
3. 从当前有效 req 与两组目标位 grant 判成功；成功事件携带命令序号进入固定四拍日历，空拍仍推进。失败只恢复候选资格，不改原年龄。
4. MEM 模型只在日历指定拍提供有效数据和 pc_err；无有效返回拍可主动改变总线，以暴露晚采样及实时总线误用。
5. R 期望只能查看本拍开始前已捕获的完成集合；MEM 本拍捕获的命令在下一拍才可用。R 按每 XAU 队头及 ready、RR 选择。
6. 按同拍原子事件更新状态：旧 R payload 来自旧命令；退休释放、新接收分配可复用同槽；新分配元数据及年龄覆盖旧归属清除。
7. 数量不变量：used_next=used+ar_fire-retire_fire；pending_next=pending+ar_fire-issue_success；pending<=used<=9。退休的命令必已成功发射。
8. 一次成功发射对应一次 +4 捕获，一次接收最多对应一次退休。若环境允许永久阻塞，只检查安全性与有条件的进展，不设置无条件超时故障。

实施记录建议字段：case_id、mutation_index、RTL 版本、修改位置、触发种子、是否真正激活变异、首个分歧周期、命中的检查器、结果与波形路径。

若变异未被触发、没有可观测影响或缺少环境前提，应分别记为未激活、等价候选或环境约束不足，不能直接记成验证通过。

机器可读用例 · 时序图集 · 详细设计方案

当前为注错设计资料；尚未实施 DUT 变异或运行 RTL 验证。

内容来源 docs/fault_catalog/capacity_policy.md

cmdbuf_rd 容量隔离修订

用户新增约束：在满足输入输出约束时，不能让一个 XAU 占满所有 entry，导致其他 XAU 无法发送命令。该要求覆盖原“九项无准入保护共享”的规则；CMDBUF 和 OTB 的九项物理池、一一对应和 R 返回后释放规则保持。

采用每个 XAU 逻辑保留一项、其余六项共享的设计。保底一项是满足入口资源可用性的最小方案，并非用户另行指定的配额数字。不按物理编号将存储切成三块；同一分配器选择可用 entry。

占用与准入

occ[x] 统计来源 x 的全部已接收、未完成 R 返回的命令，包含已发射和 MEM 已完成但尚未返回上游的命令。只有 R 退休才扣除占用。

保持不变量：

```
sum(max(occ[x], 1)) <= 9
```

等价于 `sum(max(occ[x] - 1, 0)) <= 6`

单个 XAU 因而最多使用七项。仅添加“每路至多七项”仍不够，(4,5,0) 会把第三路挡住；上述不变量同时禁止两路用满全部容量。

兼容 10B 的准入伪代码：

```
retire[x] = rvalid[x] && rready[x]
occ_after[x] = occ[x] - retire[x]
free_after = 9 - sum(occ_after)
shared_after = sum(max(occ_after[x] - 1, 0))
arready[x] = (free_after > 0)
          && ((occ_after[x] == 0) || (shared_after < 6))
accept[x] = arvalid && (src == x) && arready[x]
occ_next[x] = occ_after[x] + accept[x]
```

retire 由已有命令的 R 响应产生，不能反过来依赖新 AR 分配。物理空位仍使用 `~valid | retire_mask`。同拍退休复用时 R 读取旧负载，沿后新分配初始化覆盖旧归属清除。

当来源 x 为零占用时，不变量必然为它留有空槽；它接收首条命令不增加 max(occ[x],1)。因此即使其他来源长期不退休，它仍有至少一条接收资格。

占用 (XAU0,XAU1,XAU2)	当拍退休	aready[2:0]
(7,0,0)	无	110
(7,1,0)	无	100
(7,1,1)	无	000
(4,4,0)	无	100
(7,1,1)	XAU1 的唯一命令	010
(7,1,1)	XAU0 的一条命令	111

保证范围

用户已确认上游可以切换 src，但公平性未定。cmdbuf_rd 保证空位及对应 ready；一组共享 AR 每拍仍只能接收一条，模块不能强迫上游为另一来源提供发送机会。因此目前不建立“任意持续请求最终一定被接收”的无条件活性断言。

零占用来源有首条准入资格，不等于每个来源都始终有资格追加第二条。五项乱序窗口也保持：其他来源使用保底接收的第六条命令仍可能在窗口外等待，不能以容量公平为由越窗发射。

本策略会使只有一个来源活动时，最多占用七项，另外两项保留；这是保证其他来源后来到达时仍有空位的容量代价。

资料与检查

时序图 12 展示逐来源准入；时序图 13 展示满表退休与保底复用。

注错清单 已替换 FI008，并新增 FI043 ~ FI049。历史无配额规则仅作为已被覆盖的背景保留。

容量模型检查结果 穷举占用和同拍退休接收组合，检查保留不变量和每来源 ready；它不是 RTL 仿真或 RTL 形式验证结果。

UCAgent Tk GUI 详细操作说明

内容来源　docs/gui/README.md

CMDBUF RD　Tk GUI 详细操作图示

本文对应仓库内已交付的 **UCAgent 原生 Tkinter/ttk 桌面界面**，操作图来自实际窗口。日期：2026-10-12。HTML 版本是阅读说明的页面；实际验证操作在 Tk 窗口中完成。

本分册提供 PDF 阅读版和 Word 编辑版。截图与操作步骤逐项对应，并增加关键控件的放大视图。原 HTML 作为历史阅读源保留。

重跑当前项目的操作顺序：打开桌面 → 连接执行服务 → 进入 cmdbuf_rd → 选择 SBY 或 FormalMC → 填写并保存配置 → 环境检查 → FormalMC 能力自检 → 运行完整验证 → 查看结果和日志。

本次拍图只读取现有任务和运行记录，没有启动新的求解任务、修改服务配置或推进验收阶段。图中的 SBY 最新记录是历史 `quota_math` 单任务通过，FormalMC 最新记录是历史环境检查发现工具缺失。两者都不能当成本次完整验证通过。截图源代码和像素校验见 拍图记录。

阅读导航

- 01 启动桌面与连接服务
- 02 看懂主导航和两个项目入口
- 03 打开原 SBY 任务，一键进入 FormalMC
- 04 从“双后端验证”直接进入项目
- 05 配置 SBY
- 06 配置 FormalMC
- 07 查看 22 个任务与 177 个检查点
- 08 执行环境检查、自检和完整验证
- 09 读懂结果与运行日志
- 10 取消、退出和恢复
- 11 原 14 阶段工作台的按钮和页签

- 12 新设计如何建立阶段任务
- 13 常见问题与截图中的能力提示
- 14 截图和证据索引

01 启动桌面与连接服务

1 1 当前电脑的入口

下载或克隆仓库后，进入 `gui/`。已经建立到本次虚拟机服务的 SSH 转发时，运行 `Start-UCAgent-CMDBUF.pyw`，会直接定位到原 SBY 阶段任务。

Windows 的 `.pyw` 需要安装带 Tk 的 Python 并正确关联。双击无窗口时，用 Python 显式启动：

```
python gui/Start-UCAgent-CMDBUF.pyw
```

如果电脑没有文件关联或想直接指定服务，使用通用入口：

```
python gui/UCAgent-Desktop.pyz --server http://127.0.0.1:18814
```

当前默认 `18814` 是 Windows 电脑的本地转发端口，转发到虚拟机的 `8814` 服务。可在终端保持以下连接：

```
ssh -N -L 18814:127.0.0.1:8814 root@192.168.31.116
```

登录使用已有 SSH 凭据，GUI 不要求填写 SSH 密码。终端关闭或电脑重启后，需要重新建立转发。

1 2 在其他设备启动

桌面客户端连接该设备实际可访问的 UCAgent 服务地址。若客户端和服务都在执行设备本机，可使用：

```
python3 gui/UCAgent-Desktop.pyz --server http://127.0.0.1:8814
```

Linux 桌面要提供 Tk 和图形显示环境。执行服务需使用配套的更新源码、登记完整验证包，并提供实际工具环境；具体登记见 GitHub 使用说明 和 登记格式。

1 3 在窗口中连接　图 02 连接执行服务

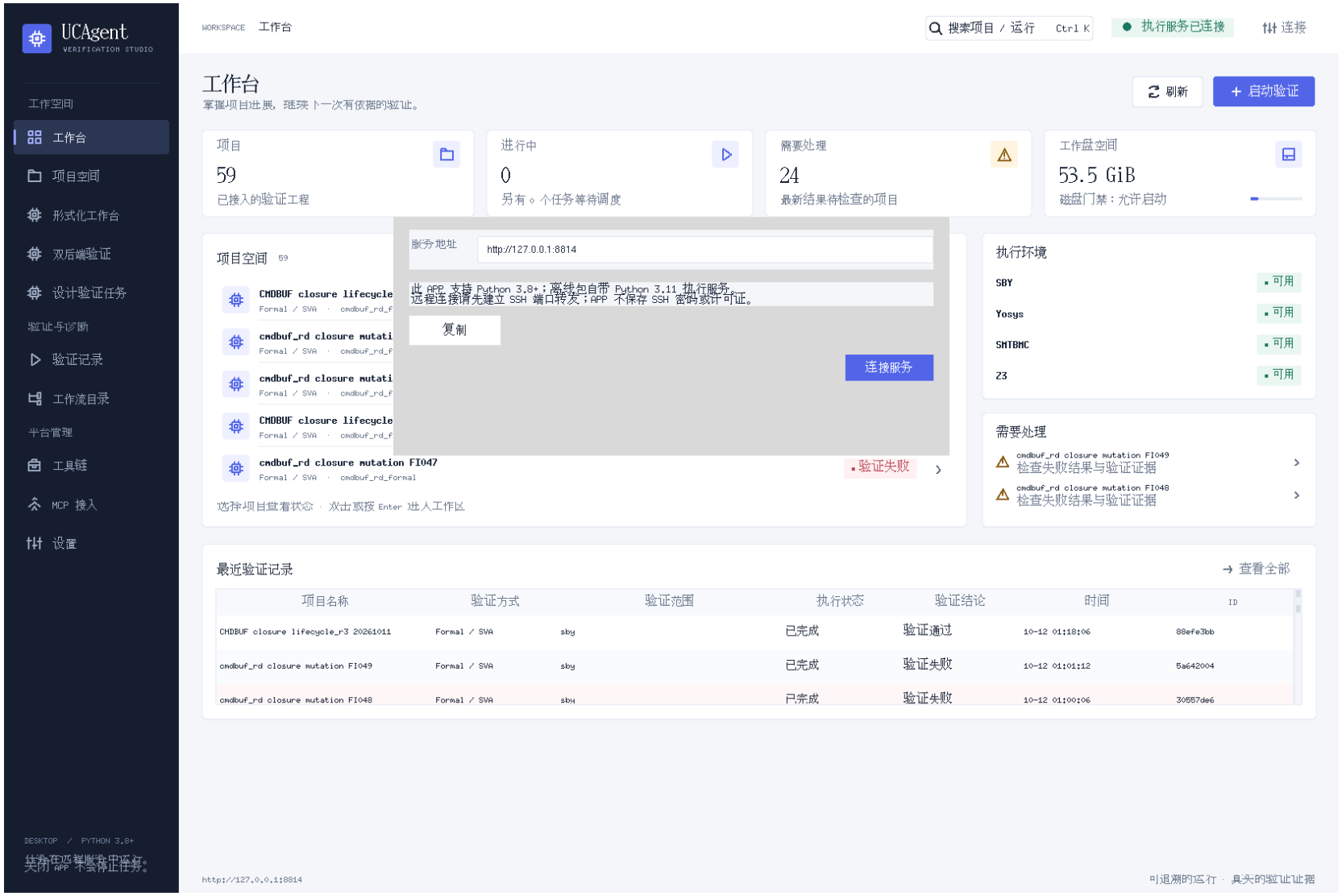


图 02：连接执行服务

真实窗口截图。下页按操作位置放大，文字序号对应下方控件说明。

操作位置放大 图 02 连接执行服务

1 顶部连接入口



2 服务地址



3 连接服务



操作位置：①窗口右上角“连接” → ②弹窗“服务地址” → ③“连接服务”。

1. 输入能从客户端访问的 HTTP 服务地址。当前 Windows 电脑填写 `http://127.0.0.1:18814`。
2. 点击“连接服务”。
3. 确认右上角显示绿色 **“执行服务已连接”**。
4. 如果没有连接成功，先确认服务和 SSH 转发正常，再点击“刷新”或重新连接。

图 02 是在虚拟机本机拍摄，因此显示 `8814`；它不是 Windows 端必须照抄的地址。连接服务成功只表示可以读取平台，不能说明 SBY / FormalMC 工具或设计验证已经通过。

02 看懂主导航和两个项目入口　图 01 工作台总览

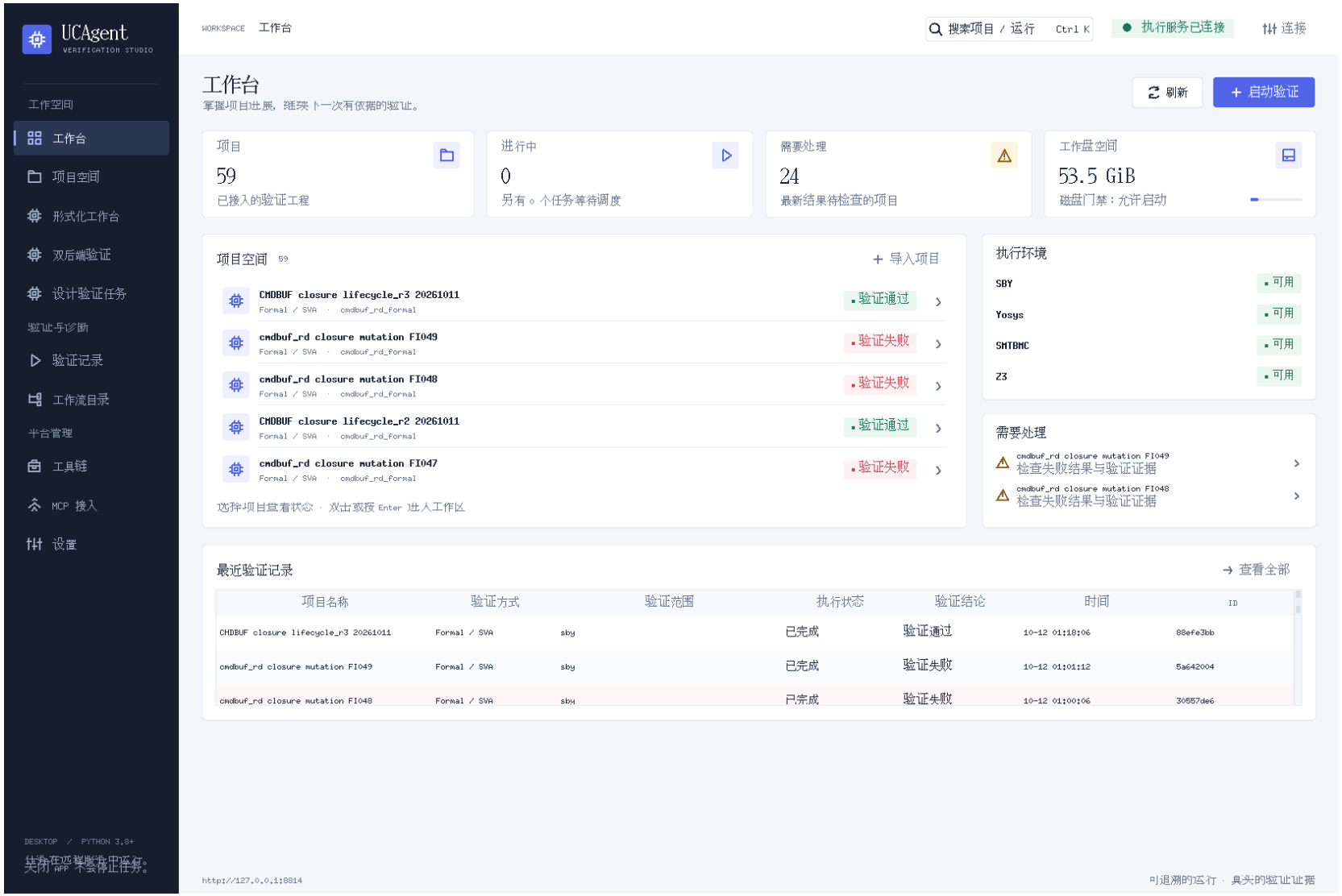


图 01：工作台总览

真实窗口截图。下页按操作位置放大，文字序号对应下方控件说明。

操作位置放大 图 01 工作台总览

1 形式化工作台



2 双后端验证



3 设置



位置	用途	当前项目怎么使用
工作台	总览、项目、运行情况和工具状态	检查连接状态，了解是否有正在执行的任务
项目空间	管理已导入的 RTL / 验证工程	新设计先导入工程；当前 cmdbuf_rd 已经有正式任务
形式化工作台	原 UCAgent 阶段任务及 Check / Complete 流程	查看原 14 阶段、原验收结论和一键切换按钮
双后端验证	已登记完整验证包的 SBY / FormalMC 操作台	重跑当前封存项目的 22 个任务
验证记录	平台通用运行记录	根据项目名称和运行 ID 查历史记录
工具链	执行服务上的工具登记	新建设计任务时核对真实工具链 ID
设置 / 顶部连接	当前客户端与服务配置	连接正确的执行服务

当前包的独立新运行主要在“**双后端操作台 → 本后端运行结果 / 运行日志**”查看；原阶段记录在“**形式化工作台**”查看。它们的证据格式与验收状态分别保留。

总览图中的 `FIxxx` 是注错工程。故障版产生“验证失败”可能正是注错检测的预期结果，应结合项目名称和注错报告判断，不能把它自动归为原版 cmdbuf_rd 的缺陷。

03 打开原 SBY 任务 一键进入 FormalMC

3 1 选择原任务　图 03 形式化工作台中的原 SBY 任务



图 03：形式化工作台中的原 SBY 任务

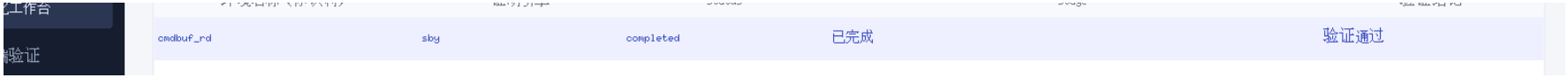
真实窗口截图。下页按操作位置放大，文字序号对应下方控件说明。

操作位置放大 图 03 形式化工作台中的原 SBY 任务

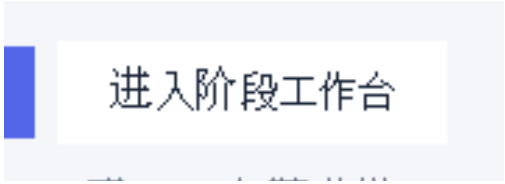
1 形式化工作台



2 选中 cmdbuf_rd / sby 原任务



3 进入阶段工作台



1. 点击左侧“**形式化工作台**”。
2. 在列表中选中 DUT 为 `cmdbuf_rd`、引擎为 `sby` 的原任务。
3. 双击该行，或点击“**进入阶段工作台**”。

本次原会话 ID 为 `90855753a21f4dc7a0dc29b7892e38f3`。新设备使用自己的任务 ID；仅复制源码和验证包不会自动复制这个会话数据库。

3 2 进入原验收页　图 04 原 SBY 阶段树与切换按钮

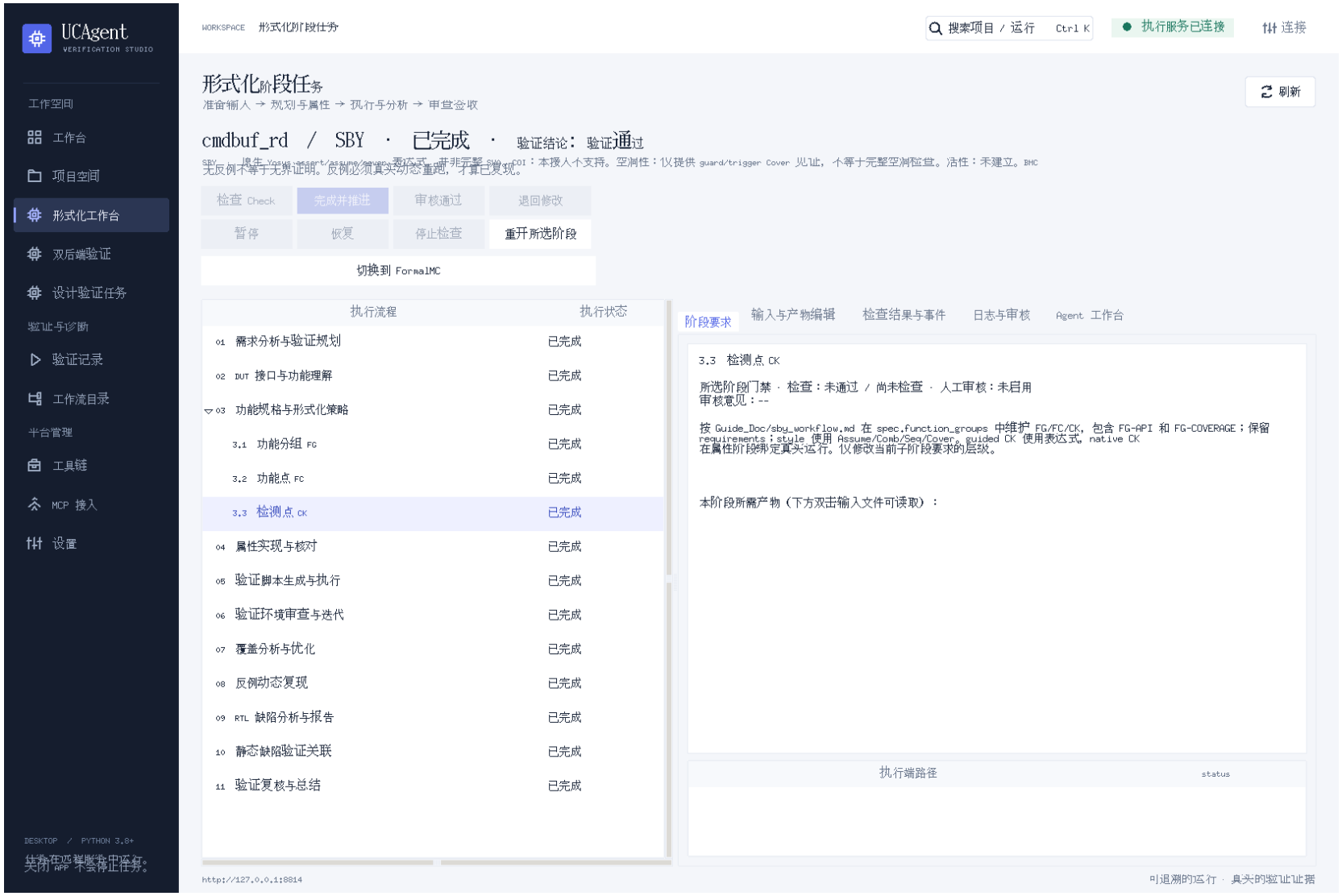


图 04：原 SBY 阶段树与切换按钮

真实窗口截图。下页按操作位置放大，文字序号对应下方控件说明。

操作位置放大 图 04 原 SBY 阶段树与切换按钮

1 整任务名称、引擎和结论



2 阶段树



真实窗口局部放大。此图保留当时的任务、结果和运行状态。

3 切换到 FormalMC



识别顺序：①顶部 DUT / 引擎 / 结论 → ②左侧阶段树 → ③“切换到 FormalMC”。

顶部应显示 `cmdbuf_rd / SBY`，当前原任务为“**已完成**”，验证结论为“**验证通过**”。左侧包含需求、FG、FC、CK、属性、环境、覆盖、回放及总结等原流程节点。

单击“**3.3 检测点 CK**”等节点，右侧“阶段要求”显示对应要求和产物路径。单击阶段只改变查看位置，不推进流程。已完成任务中的 Check / Complete 等按钮处于灰色是正常状态。

图 04 右侧的“所选阶段门禁”是选中节点的检查投影，截图中显示“尚未检查”；它与顶部整任务结论、左侧阶段完成状态不是同一个字段。审查历史结论时，继续查看图 05 的已存检查结果和正式验收报告，不能只凭其中一行判断全部证明完成。

点击“**切换到 FormalMC**”，会直接进入下图的 FormalMC 操作台。

图 07 由原 SBY 任务进入 FormalMC 操作台

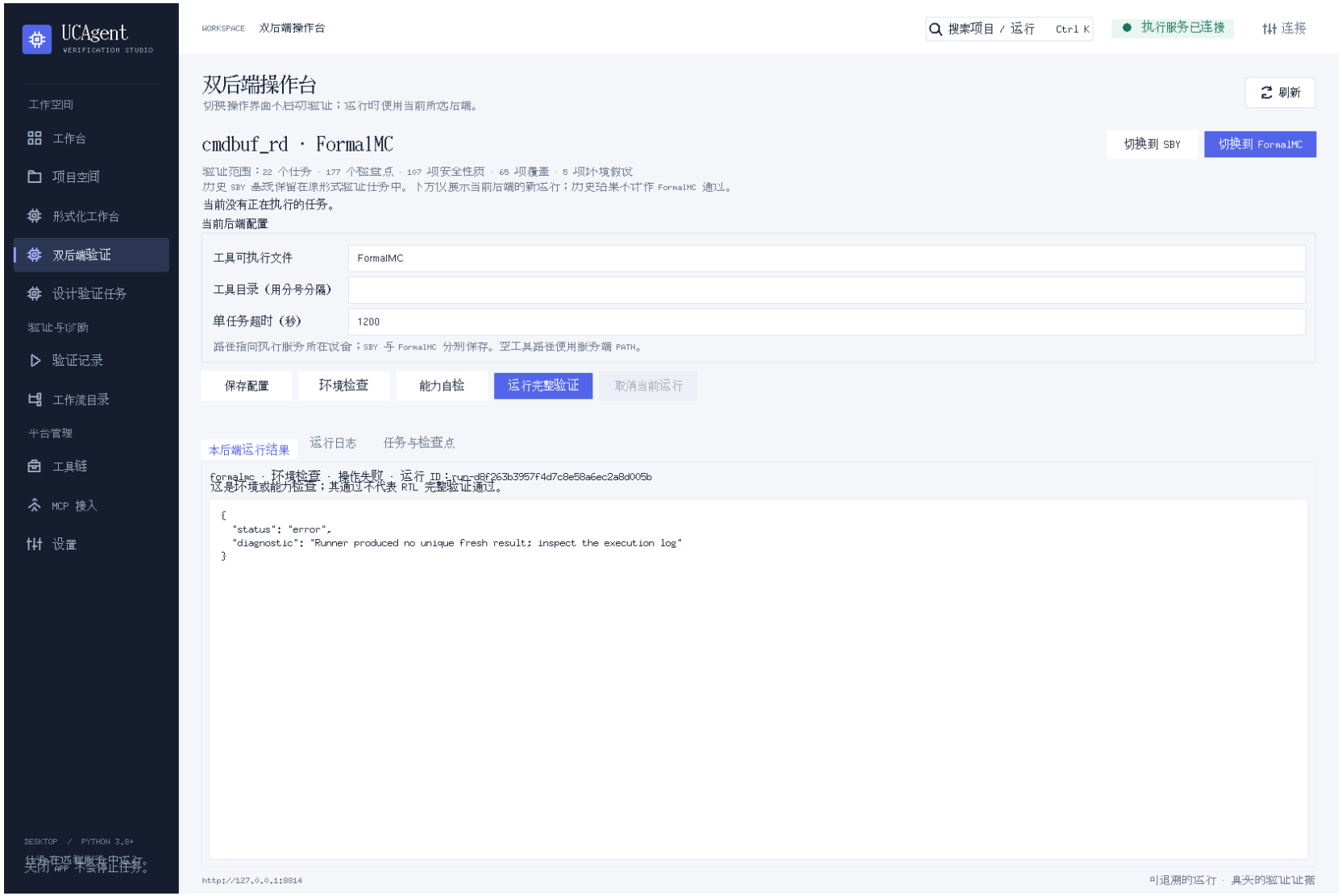


图 07：由原 SBY 任务进入 FormalMC 操作台

真实窗口截图。下页按操作位置放大，文字序号对应下方控件说明。

操作位置放大 图 07 由原 SBY 任务进入 FormalMC 操作台

1 切换 SBY / FormalMC



2 当前 FormalMC 的工具配置



3 保存配置



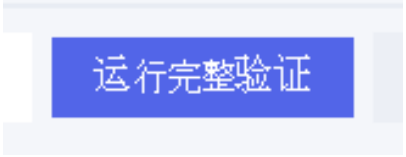
4 环境检查



5 FormalMC 能力自检



6 运行完整验证



这一点击只切换操作界面。它不启动 FormalMC、不改写原 SBY 结论，也不将原 SBY 结果算成 FormalMC 通过。

04 从 双后端验证 直接进入项目　图 08 完整验证包列表

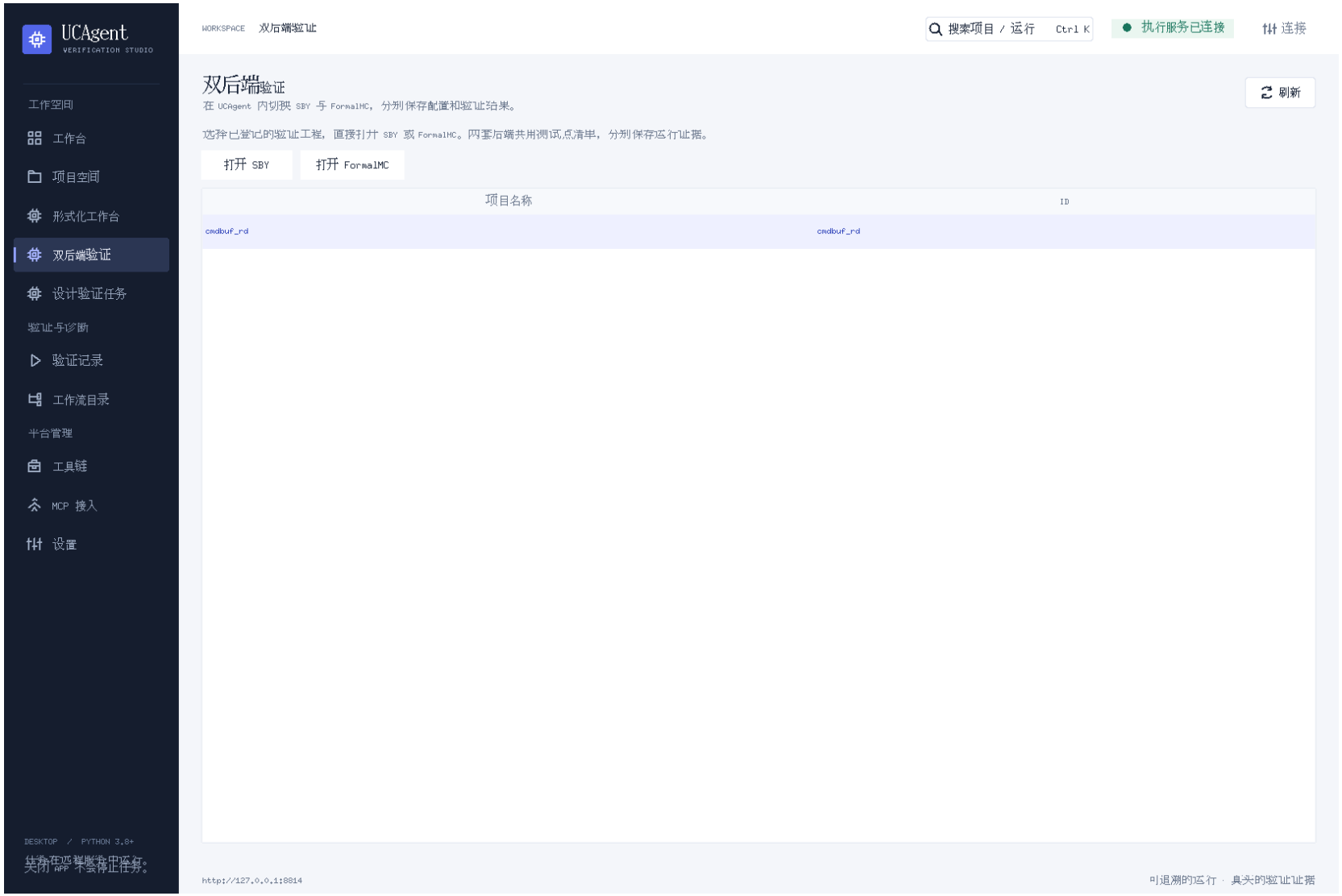


图 08：完整验证包列表

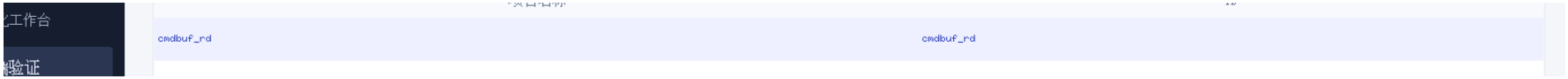
真实窗口截图。下页按操作位置放大，文字序号对应下方控件说明。

操作位置放大 图 08 完整验证包列表

1 双后端验证



2 选中 cmdbuf_rd 完整包



3 打开 SBY / FormalMC



操作位置：①左侧“双后端验证” → ②选中 `cmdbuf_rd` → ③“打开 SBY”或“打开 FormalMC”。

新设备没有原 SBY 会话时，只要执行服务已经登记完整验证包，就可以使用这个入口。它不要求先建立或完成一个新的 14 阶段任务。

如果列表为空，执行服务尚未登记包或客户端连接了另一台服务。管理员需在登记文件中填写 `formal/cmdbuf_rd_dual_backend` 的本机绝对路径，再按 服务登记格式启动匹配版本的服务。

05 配置 SBY　图 09 SBY 配置和操作按钮

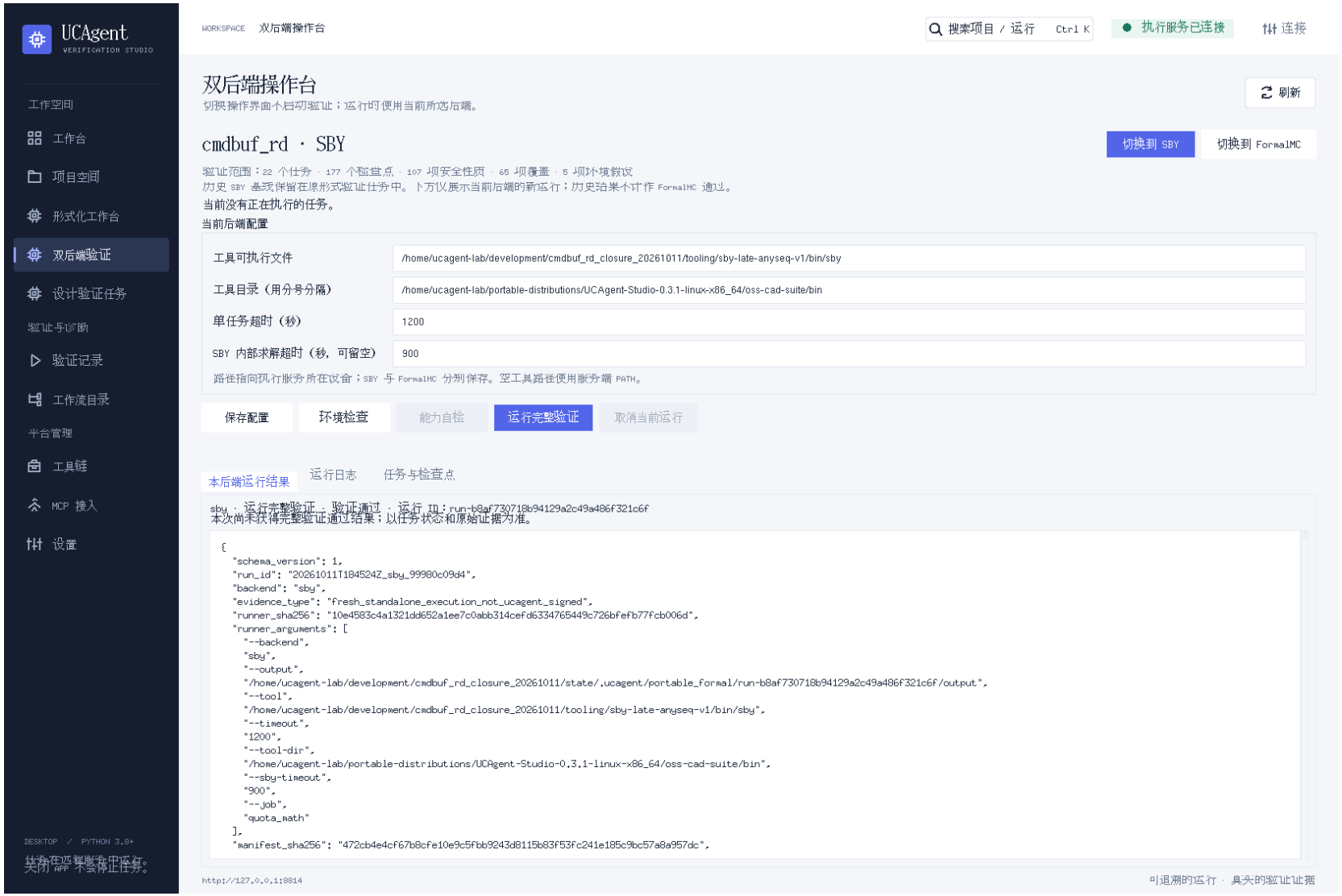


图 09：SBY 配置和操作按钮

真实窗口截图。下页按操作位置放大，文字序号对应下方控件说明。

操作位置放大 图 09 SBY 配置和操作按钮

1 当前 SBY 的工具配置



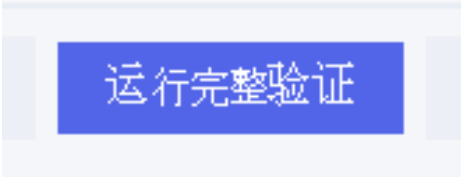
2 保存配置



3 环境检查



4 运行完整验证



5 取消当前运行：当前无活动任务



确认标题是 **`cmdbuf_rd · SBY`**，右上角 SBY 按钮为选中状态。以下路径均指向**执行服务所在设备**，不是 Windows 客户端。

字段	怎么填	示例 / 本次值
工具可执行文件	填 sby 的实际文件路径；填命令名时需能在服务进程 PATH 中找到	新设备示例 `/opt/oss-cad-suite/bin/sby`
工具目录（用分号分隔）	提供 Yosys、SMTBMC 及所需求解器的目录；多个目录用英文 ; 分隔	`/opt/oss-cad-suite/bin;/opt/solvers/bin`
单任务超时（秒）	一个任务的外部等待预算，不是全部 22 任务的总时间	本次配置 `1200`
SBY 内部求解超时（秒，可留空）	SBY 自身的求解预算；留空使用任务原配置	本次配置 `900`

截图中的 `/home/ucagent-lab/...` 是本次虚拟机的实际安装路径。其他设备应填写自己的路径，不直接照搬。

1. 填写或调整字段。
2. 点击“**保存配置**”，等待“已保存 sby 配置”提示。
3. 点击“**环境检查**”。
4. 环境检查通过后，点击“**运行完整验证**”。

SBY 的“**能力自检**”按钮在当前界面中为灰色；该按钮提供给 FormalMC 使用。正常重跑当前 SBY 项目不需要点击它。

配置变更后没有保存，启动会显示下图的提示，不会提交检查或验证任务：

图 13 配置未保存提示



图 13：配置未保存提示

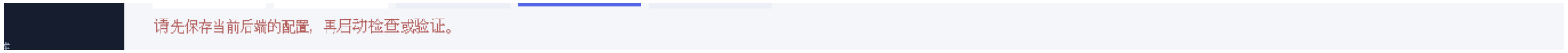
真实窗口截图。下页按操作位置放大，文字序号对应下方控件说明。

操作位置放大 图 13 配置未保存提示

1 保存配置



2 提示



图中为拍图临时把超时从 1200 改为 1201 产生的真实客户端提示，随后已恢复原值，没有修改服务配置。编辑草稿可以在同一操作台来回切换后端时保留；需要长期保存时必须点击“保存配置”。

06 配置 FormalMC　图 07 FormalMC 工具配置

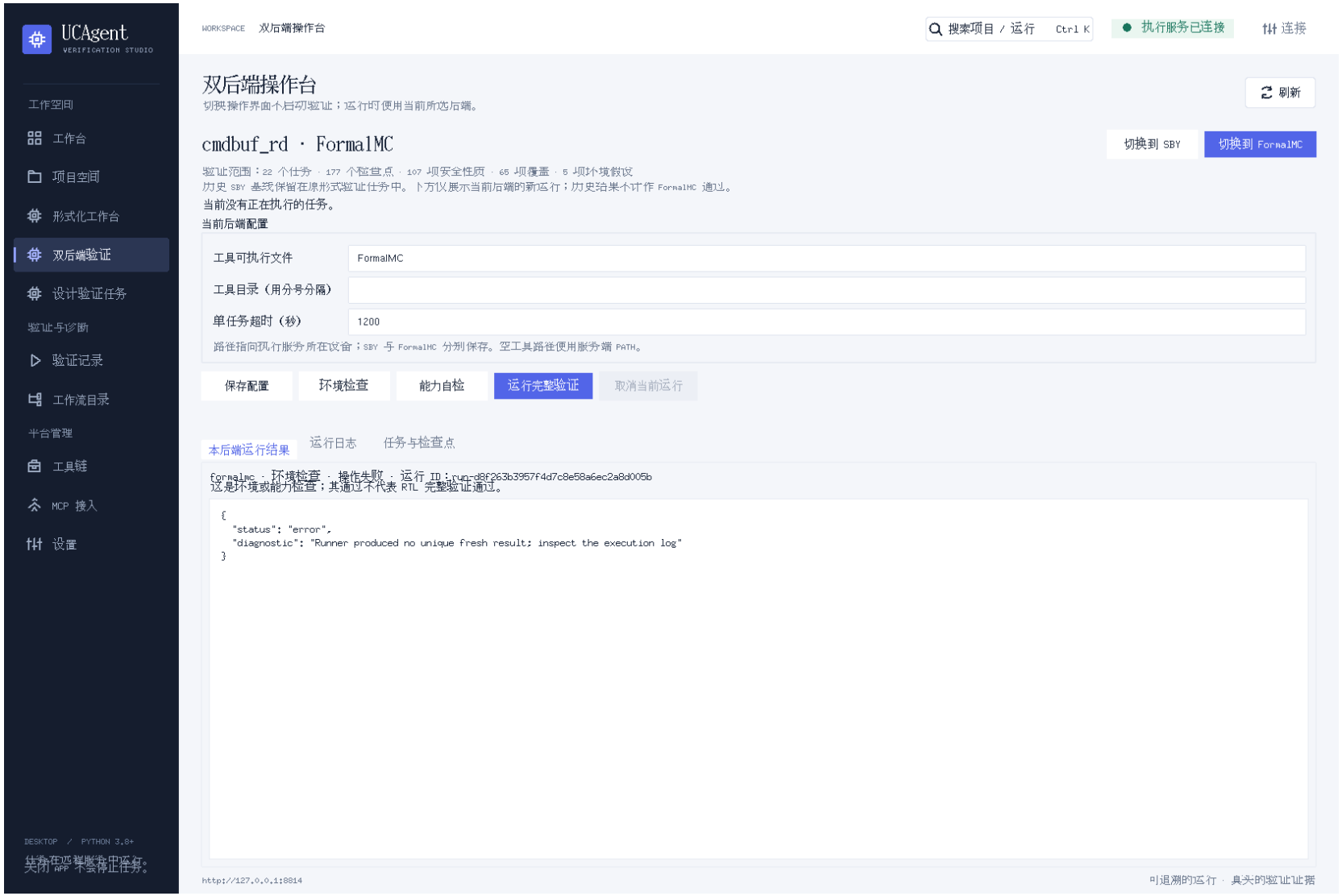


图 07：FormalMC 工具配置

真实窗口截图。下页按操作位置放大，文字序号对应下方控件说明。

操作位置放大 图 07 FormalMC 工具配置

1 切换 SBY / FormalMC



2 当前 FormalMC 的工具配置



3 保存配置



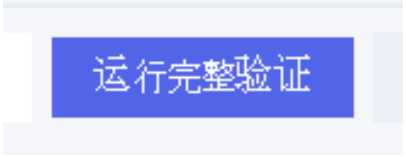
4 环境检查



5 FormalMC 能力自检



6 运行完整验证



1. 点击右上角“**切换到 FormalMC**”。
2. 确认标题变为 **`cmdbuf_rd · FormalMC`**。
3. 填写执行设备实际安装的 FormalMC 可执行文件，例如 `/opt/formalmc/bin/FormalMC`。这只是路径格式示例，不表示该文件在当前机器上存在。
4. 如厂商工具依赖额外目录，在“工具目录”填写对应目录；许可证和厂商初始化环境由服务进程所在设备提供。
5. 设置单任务超时，点击“**保存配置**”。
6. 先做“**环境检查**”，再做“**能力自检**”；通过后点击“**运行完整验证**”。

FormalMC 模式没有 SBY 内部超时字段。SBY 和 FormalMC 分别保存工具路径、超时和运行结果；切回 SBY 后，其配置仍是 SBY 配置。

能力自检使用运行包内的小模型检查工具是否支持所需初始状态、符号输入、覆盖目标和故意失败的断言。自检通过只证明运行入口满足这些检查，不表示 cmdbuf_rd 已证明。完整 FormalMC 运行也会自动执行这项自检。

当前虚拟机未安装真实 FormalMC，工具缺失截图见 第 09 节。未发现工具时，应在实际有工具和许可证的执行设备上配置并重新运行。

07 查看 22 个任务与 177 个检查点　图 10 任务与检查点清单

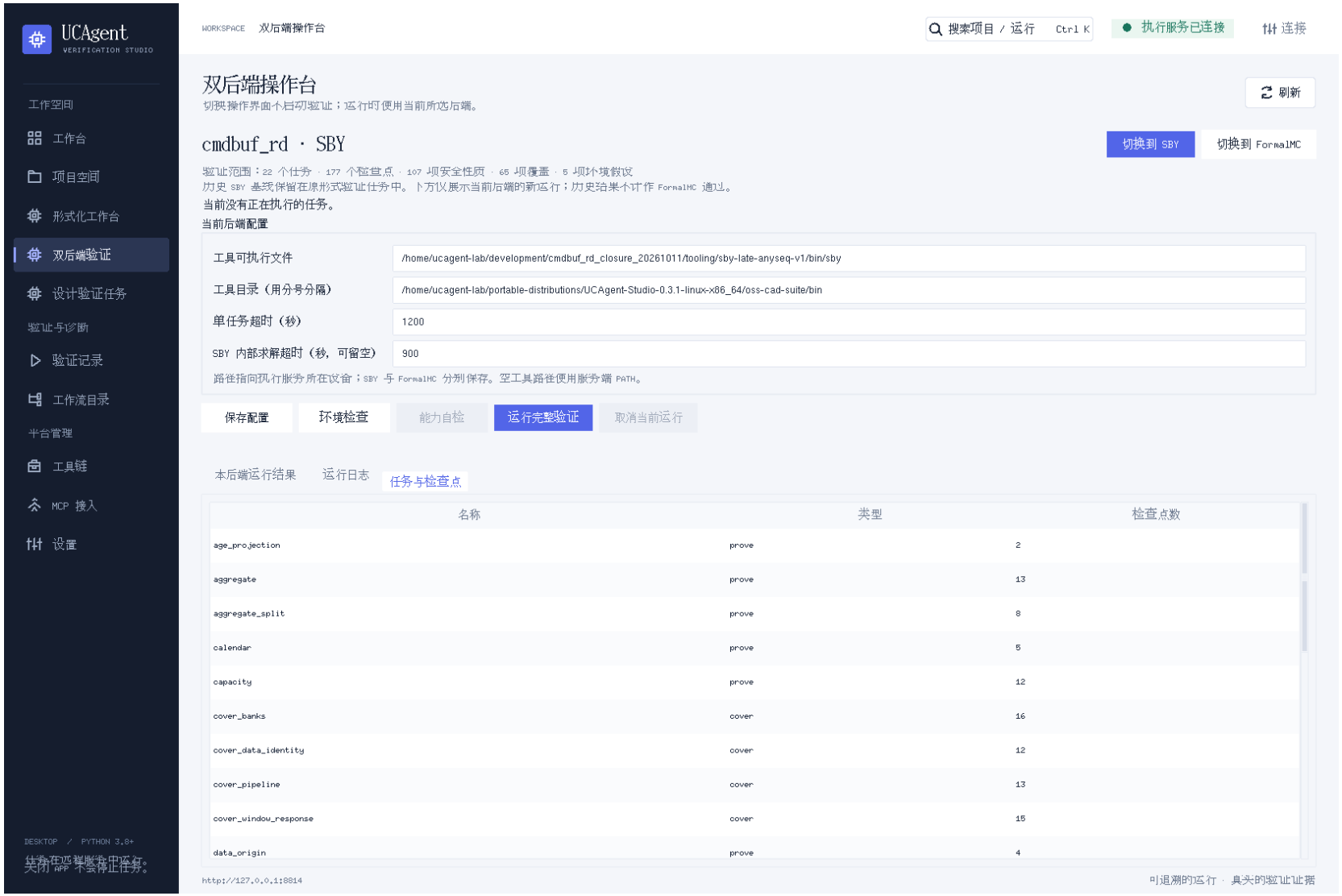
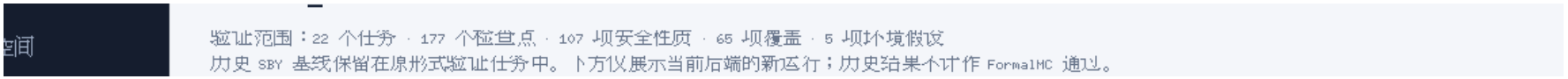


图 10：任务与检查点清单

真实窗口截图。下页按操作位置放大，文字序号对应下方控件说明。

操作位置放大 图 10 任务与检查点清单

1　核对验证范围



2　任务与检查点页签



3 任务与检查点

名称	类型	检查点数
age_projection	prove	2
aggregate	prove	13
aggregate_split	prove	8
calendar	prove	5
capacity	prove	12
cover_banks	cover	16
cover_data_identity	cover	12
cover_pipeline	cover	13
cover_window_response	cover	15
data_origin	prove	4

真实窗口局部放大。此图保留当时的任务、结果和运行状态。

点击下方“**任务与检查点**”页签，核对顶部范围：

项目	当前封存包
任务总数	22
规格检查点 CK	177
安全检查点	107
业务覆盖检查点	65
环境假设	5

表格显示任务 ID、证明 / 覆盖模式和关联的 CK 数量；向下滚动可查看剩余任务。数学供应证明、辅助不变量等任务可能关联 0 个直接 CK，这不表示它们没有参与依赖证明。

完整测试点分解是 **15 FG → 35 FC → 177 CK**，见 规格分解 和 机器可读清单。该页是任务清单，不是逐 CK 的编辑器；逐项定义和运行结果在规格文件及结果 JSON 中查看。

当前项目的“运行完整验证”会提交全套任务，运行器按真实依赖关系安排供应证明和依赖任务。只修改仓库顶层 `rtl/` 不会更新封存包的 `jobs/` 输入；设计变更后需重新生成输入并重新验证。

08 执行环境检查 自检和完整验证

8 1 按钮的准确含义

按钮	实际操作	何时使用
保存配置	保存当前后端工具路径和超时	修改字段后先执行
环境检查	检查运行包完整性和对应工具能否启动	第一次连接新设备、修改路径或工具环境后
能力自检	执行 FormalMC 兼容性小模型	FormalMC 环境检查通过后

按钮	实际操作	何时使用
运行完整验证	用当前后端执行 cmdbuf_rd 全套任务	配置和工具条件就绪后
取消当前运行	请求停止该项目真正处于活动状态的运行	需要停止当前求解时

8 2 完整运行的实际步骤

1. 确认标题和后端正确。
2. 核对范围为 **22 个任务 / 177 个检查点**。
3. 修改配置后点击“保存配置”。
4. 点击“环境检查”，到“运行日志”查看实际输出。
5. FormalMC 再执行“能力自检”。
6. 点击“运行完整验证”一次。
7. 观察“正在执行”提示，记录项目、后端、动作与运行 ID。
8. 在“本后端运行结果”和“运行日志”查看进度与最终结果。

运行中可以切换 SBY / FormalMC 页面；已经启动的任务继续使用启动时的后端。切换页面不会把正在运行的 SBY 改成 FormalMC，也不会启动第二个任务。活动任务存在时，保存和新启动按钮受控；服务同时只接受一个新的活动运行。

环境检查、能力自检及完整运行都有各自的运行记录。当前操作台主要展示当前后端的最近一次记录；“刷新”或 F5 重新读取实际状态，不会重新求解。

09 读懂结果与运行日志

9 1 结果页 先看运行范围 再看通过状态　图 11 历史 SBY 单任务结果

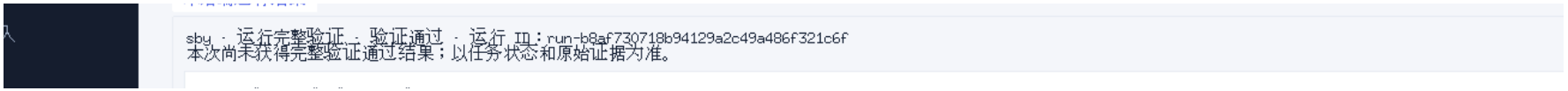


图 11：历史 SBY 单任务结果

真实窗口截图。下页按操作位置放大，文字序号对应下方控件说明。

操作位置放大 图 11 历史 SBY 单任务结果

1 单任务通过，但未完整通过



2 未运行计数与完整套件标记


```
  },
  "counts": {
    "assumption": 5,
    "not_run": 172
  },
  "completed_utc": "2026-10-11T18:45:25.895752+00:00",
  "full_suite_passed": false
}
```

图 11 是真实的 `quota_math` 单任务运行。它的工具求解通过，但 **`full_suite_passed` 为 `false`**，所以正文明确提示“本次尚未获得完整验证通过结果”。这个运行仅验证辅助数学任务，结果里其余 172 个规格检测点为未运行，5 个环境假设单列。

结果第一行的“运行完整验证”是动作名称，**不能只凭该名称或“验证通过”文字判断全套完成。**应同时核对结果中的范围和完整套件标记。

看到的结果	应如何理解
环境检查通过	包和工具启动检查通过，尚未证明 DUT
能力自检通过	运行入口的兼容性检查通过，尚未证明 DUT
单任务 `passed`，`full_suite_passed=false`	本次是部分运行，不是全套通过
正文显示“本次完整验证通过”，结果 `full_suite_passed=true`	当前后端的全套运行满足检查点和依赖要求
`error` / 工具缺失 / 许可证错误	执行条件或工具发生错误，不能当作 DUT 证明通过
`timeout` / `cancelled` / 未识别结果	本次没有获得完整证明结论
安全性质失败并有反例	检查具体性质、假设和波形；动态重跑确认问题

全套通过时核对当前后端、22 任务的运行范围和 **107 安全已证明 / 65 覆盖已命中 / 5 假设单列。**环境假设不计为证明成功数量。

9 2 日志页 查看真实工具输出　图 12 实际 SBY 运行日志



图 12：实际 SBY 运行日志

真实窗口截图。下页按操作位置放大，文字序号对应下方控件说明。

操作位置放大 图 12 实际 SBY 运行日志

1 运行日志



真实窗口局部放大。此图保留当时的任务、结果和运行状态。

点击“**运行日志**”。这里保留真实命令、工具诊断、求解输出和退出情况。右侧滚动条用于查看前后内容。排查时记录运行 ID、后端、任务 ID、错误文字及相关日志，不只截图一个 PASS / FAIL。

操作台屏幕上的原始结果内容可以滚动查看。完整证据文件保存在执行服务状态目录下；需要复制完整目录时，在执行设备按实际工作空间和运行 ID 找到对应文件。当前界面没有为完整包提供一个“导出全部证据”的按钮。

9 3 FormalMC 工具缺失示例 图 14 真实 FormalMC 缺工具日志

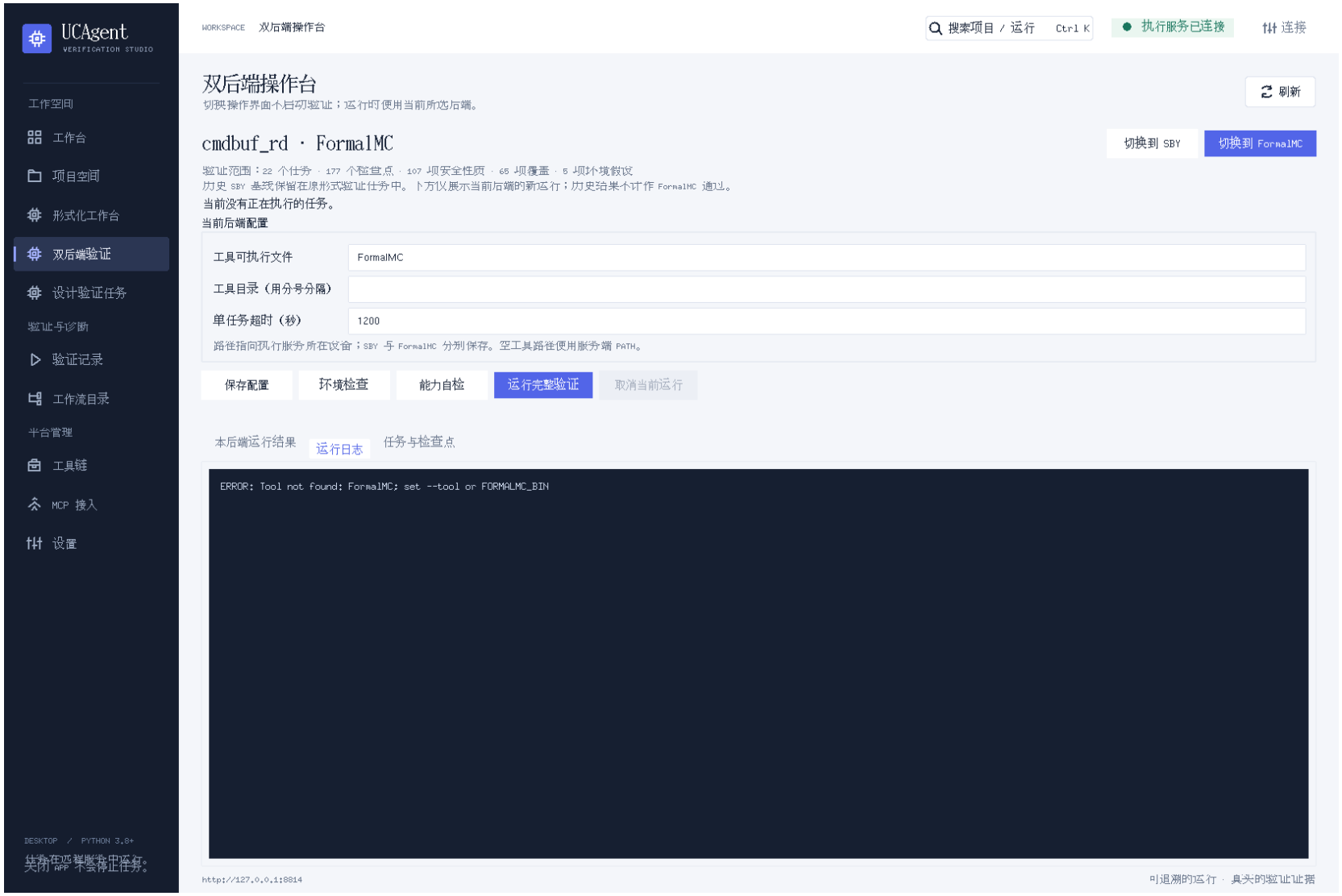


图 14：真实 FormalMC 缺工具日志

真实窗口截图。下页按操作位置放大，文字序号对应下方控件说明。

操作位置放大 图 14 真实 FormalMC 缺工具日志

1 真实日志



真实窗口局部放大。此图保留当时的任务、结果和运行状态。

日志中的 `Tool not found: FormalMC` 表示执行服务没有找到指定工具。按以下顺序排查：

1. 工具文件是否存在于执行设备。
2. “工具可执行文件”是否填写正确，并已保存。
3. 服务用户是否有执行权限、所需 PATH / 动态库和厂商环境。
4. 工具能够启动后，再排查许可证或编译问题。

这张图不是 FormalMC 发现 RTL 错误的反例。本次原 SBY 已验收的结果也不会因此变成 FormalMC 通过。

10 取消 退出和恢复

“**取消当前运行**”没有任务时为灰色，如图 09。该项目有活动任务时可以点击，等待状态从取消请求转为最终取消，再查看日志。

如果运行 SBY 后切到 FormalMC 页面，取消按钮仍针对该项目实际运行中的 SBY 任务；先看“正在执行”提示中的后端和运行 ID。若活动运行属于另一个登记项目，当前项目的取消入口不会控制它。

当前入口连接远程执行服务，关闭 Tk 客户端不会停止远程服务。重新打开客户端并连接同一服务、同一包，可以读取服务保存的配置和运行结果。客户端未保存草稿不保证在退出后恢复，重要配置先点击保存。

电脑重启后，重新建立 SSH 转发，再打开桌面。原 SBY 阶段会话和新运行结果保存在服务端；本机 `.pyw` 文件并不包含完整远程数据库。如果用 UCAgent 的本机服务模式，退出行为应以该模式窗口的实际提示为准。

11 原 14 阶段工作台的按钮和页签

11 1 查看原阶段结果　图 05 原任务检查结果与事件

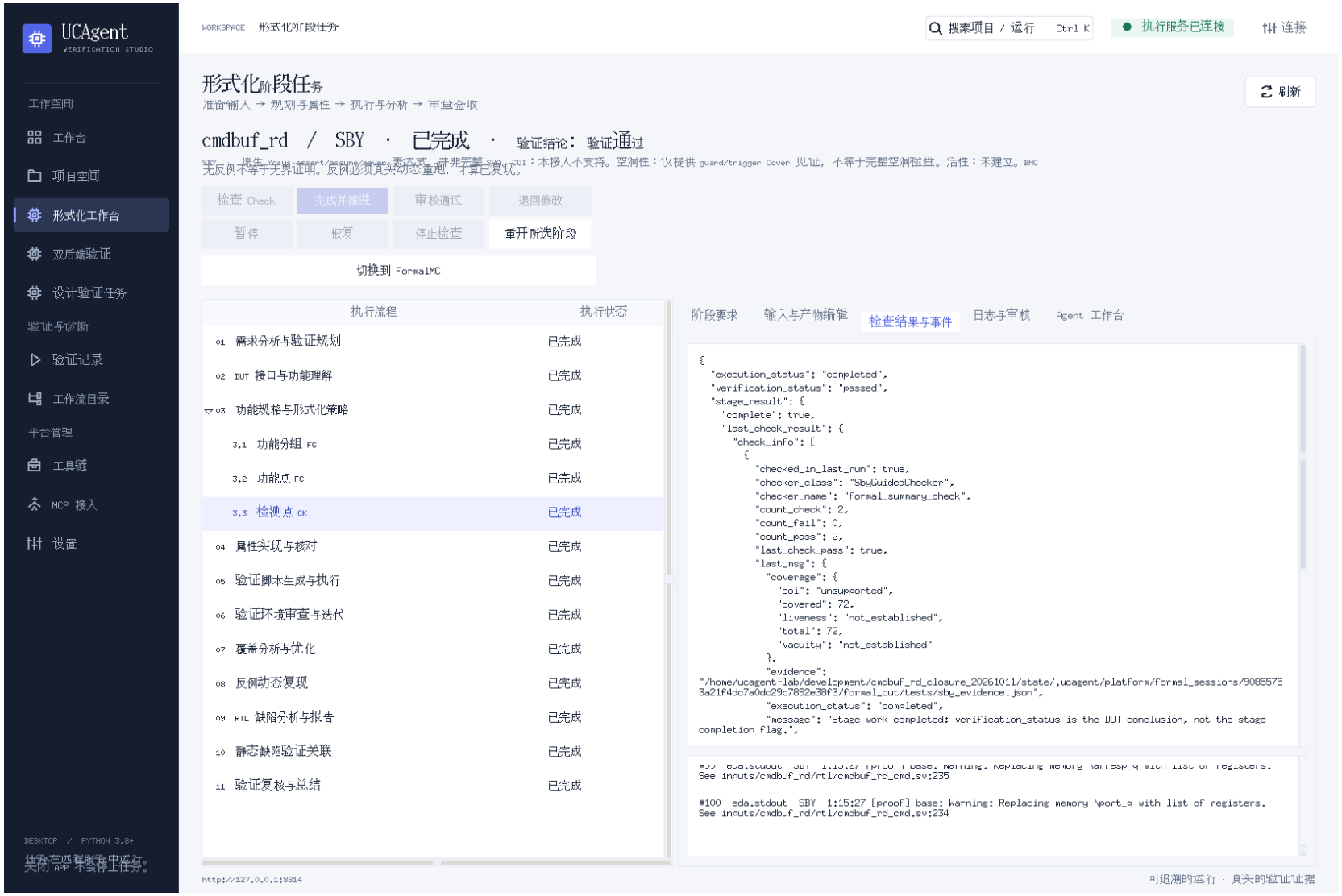


图 05：原任务检查结果与事件

真实窗口截图。下页按操作位置放大，文字序号对应下方控件说明。

操作位置放大 图 05 原任务检查结果与事件

1 检查结果与事件页签



2 原任务结果


```
{
  "execution_status": "completed",
  "verification_status": "passed",
  "stage_result": {
    "complete": true,
    "last_check_result": {
      "check_info": [
        {
          "checked_in_last_run": true,
          "checker_class": "SbyGuidedChecker",
          "checker_name": "formal_summary_check",
          "count_check": 2,
          "count_fail": 0,
          "count_pass": 2,
          "last_check_pass": true,
          "last_msg": {
            "coverage": {
              "coi": "unsupported",
              "covered": 72,
              "liveness": "not_established",
              "total": 72,
              "vacuity": "not_established"
            },
            "evidence":
"/home/ucagent-lab/development/cmdbuf_rd_closure_20261011/state/.ucagent/platform/formal_sessions/9085575
3a21f4dc7a0dc29b7892e38f3/formal_out/tests/sby_evidence.json",
            "execution_status": "completed",
            "message": "Stage work completed; verification_status is the DUT conclusion, not the stage
completion flag.",
```


真实窗口局部放大。此图保留当时的任务、结果和运行状态。

页签	用途
阶段要求	查看选中阶段的要求、参考文件、产物路径和门禁信息
输入与产物编辑	查看工作空间文件；允许编辑的文件提交前审查差异
检查结果与事件	查看执行状态、验证结论、实际检查结果和事件
日志与审核	保存阶段日志、填写审核理由、处理实际人工门禁
Agent 工作台	让已配置的 Agent 准备当前阶段产物并调用原检查流程

阶段树会将分组与子节点展开显示。例如 FG / FC / CK 属于规格分解部分，屏幕上的顶层编号不等于原子阶段总数。当前验收记录确认 **14 个启用阶段全部完成**，以正式阶段记录为准。

11 2 按钮怎么用

按钮	含义	原已完成任务
检查 Check	实际执行当前阶段检查	通常灰色，不需要为查看结果重新点击
完成并推进	满足门禁后完成阶段并进入下一阶段	当前已完成
审核通过 / 退回修改	处理确实启用的人工审核门禁	依据当前阶段和审核状态启用
暂停 / 恢复	处理阶段任务的暂停与恢复	依据实际状态启用
停止检查	停止阶段工作台当前活动检查	不等于双后端包的取消按钮
重新开启所选阶段	让既有任务重新进入阶段工作	查看历史结果或重跑封存包时无需点击
切换到 FormalMC	当前 cmdbuf_rd 的完整包导航入口	可用；只切换页面

审核通过不能替代 Check，保存文件和 Agent 回复也不会自动完成阶段。已完成任务的灰色按钮反映真实状态，不表示 GUI 失效。

11 3 Agent 工作台　图 06 已完成任务中的 Agent 工作台

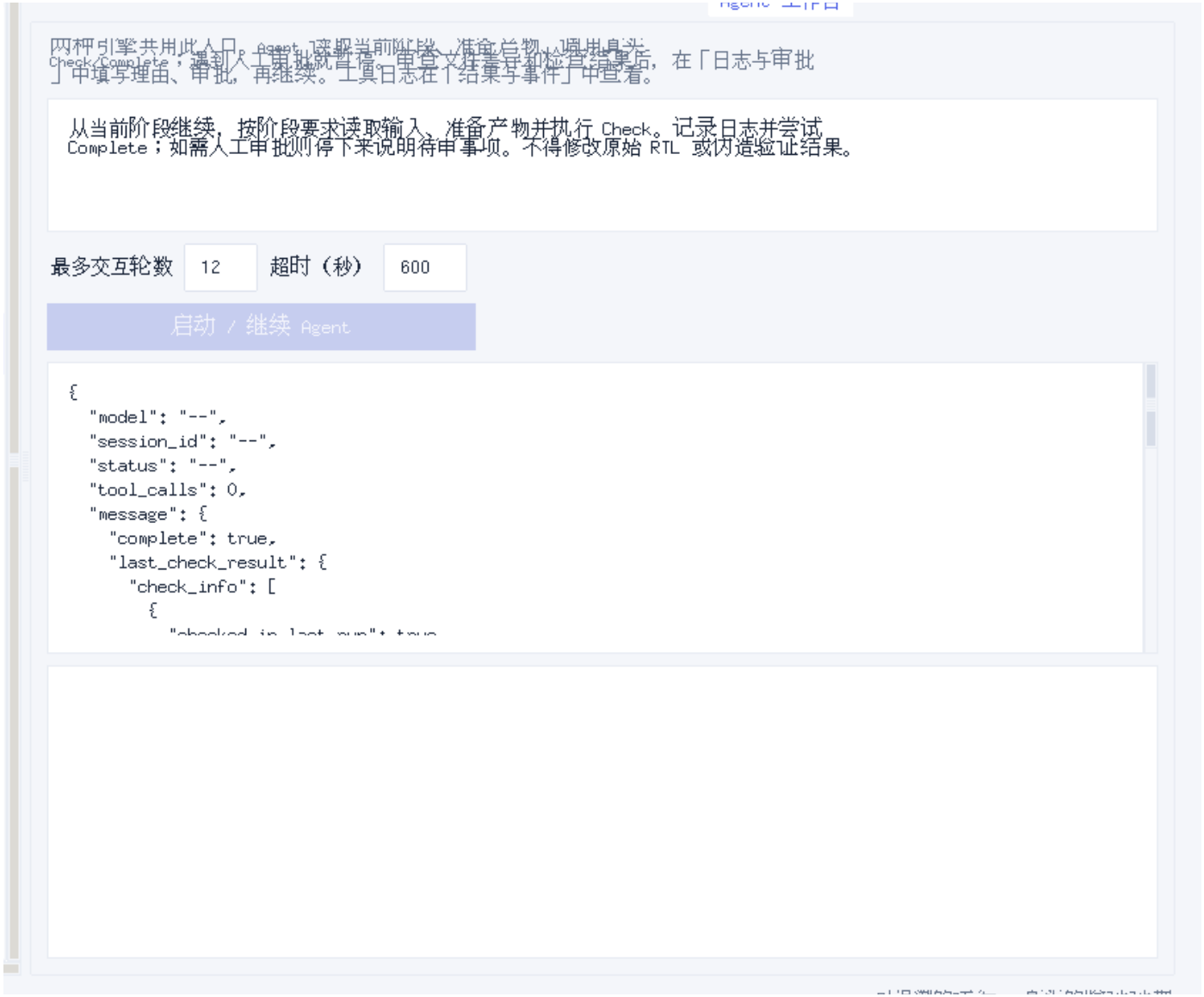


图 06：已完成任务中的 Agent 工作台

真实窗口截图。下页按操作位置放大，文字序号对应下方控件说明。

操作位置放大 图 06 已完成任务中的 Agent 工作台

1 Agent 工作台



真实窗口局部放大。此图保留当时的任务、结果和运行状态。

新建的阶段任务需要使用 Agent 时，在该页填写任务说明、运行轮数与时间预算，再使用“启动 / 继续 Agent”。执行服务应已配置适用的 Agent 身份和模型入口；本仓库的 Tk 客户端本身不提供模型凭据。

遇到人工审核门禁，先审查“输入与产物编辑”的差异和“检查结果与事件”，再在“日志与审核”填写实际理由并审核，通过后继续。当前原任务已完成，Agent 操作不作为重跑 22 任务的必要步骤。

12 新设计如何建立阶段任务

当前封存 cmdbuf_rd 的重跑入口已经在第 04～08 节说明。需要对新设计建立规格、测试点和属性时，使用原阶段流程：

1. 在“项目空间”导入执行设备上的真实工程，核对顶层和源码路径。
2. 在“工具链”核对可用的引擎和工具链 ID。
3. 进入“形式化工作台”，点击“新建阶段任务”。
4. 在下图表单中填写自己的项目、DUT、引擎及对应工具链。
5. 按工作要求设置人工审核、动态反例复现和静态缺陷关联选项，再提交。
6. 进入阶段工作台，按实际阶段要求准备产物、执行 Check / Complete，并处理审核门禁。

图 15 新建阶段任务表单

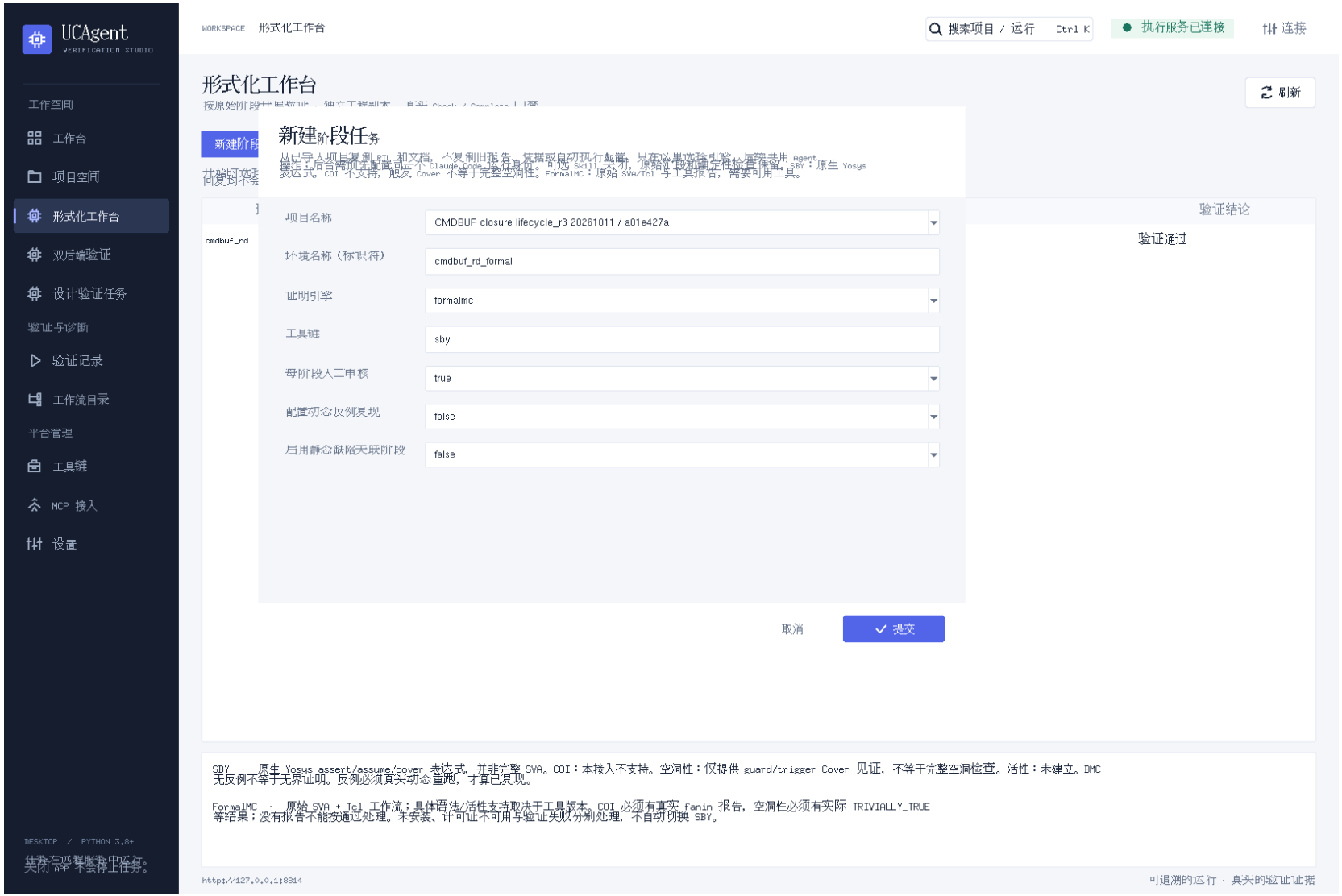
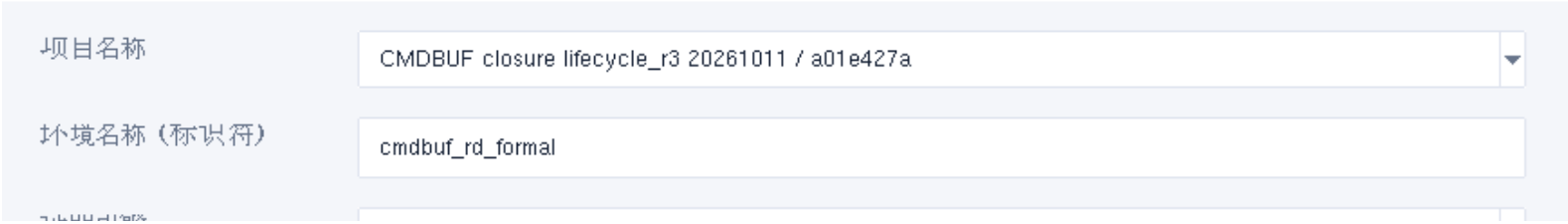


图 15：新建阶段任务表单

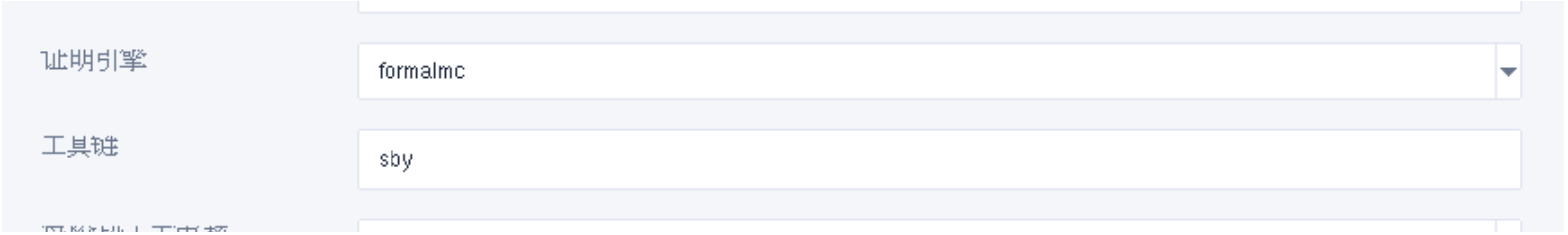
真实窗口截图。下页按操作位置放大，文字序号对应下方控件说明。

操作位置放大 图 15 新建阶段任务表单

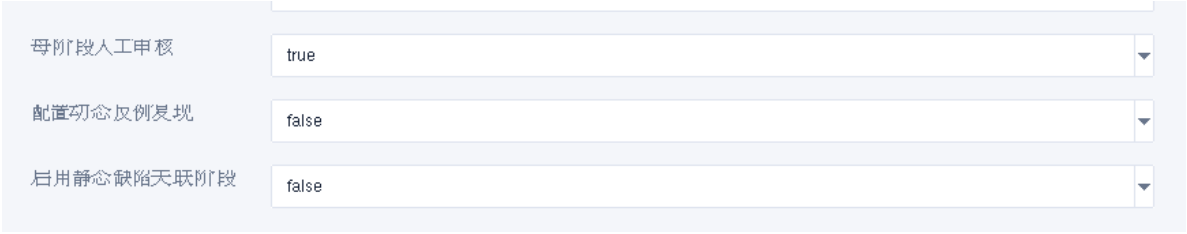
1 目标项目和 DUT 标识符



2 证明引擎与对应工具链



3 审核与辅助验证选项



4 提交：拍图未点击



字段	操作要点
项目名称	选择已导入的目标设计，不能照抄截图里作为示例的辅助证明项目
环境名称（标识符）	当前界面对 DUT 名称字段的标签；填写目标 DUT 的实际顶层模块标识符
证明引擎	选择 `sby` 或 `formalmc`
工具链	填写该引擎在执行服务中真实登记的工具链 ID
每阶段人工审核	`true` 启用真实人工门禁，`false` 按检查器流程推进
配套动态反例复现	根据项目所需的动态回放环境配置
启用静态缺陷关联阶段	根据项目的静态检查能力配置

图 15 保留了窗口原始默认值，仅用于识别字段，拍图没有提交新任务。默认引擎和工具链文字并不保证成对可运行：创建 SBY 任务时选择 `sby` 和其实际工具链；创建 FormalMC 任务时选择 `formalmc` 和已登记的对应工具链。新任务不会自动继承旧报告和已通过的结论。

13 常见问题与截图中的能力提示

现象	处理方法
双击 `.pyw` 无窗口	用 Python 显式启动；核对 Tk 安装、文件关联及服务地址
服务连接不上	核对服务进程、端口、客户端地址和 SSH 转发；先连通再刷新
“双后端验证”列表为空	核对是否连接正确服务，以及完整包登记文件是否生效
原页没有当前包的“切换到 FormalMC”入口	核对客户端和服务版本，以及登记文件中的包与真实会话 ID 关联
修改配置后检查没有启动	先保存配置，再执行环境检查或完整验证

现象	处理方法
SBY 的能力自检按钮灰色	正常；该自检按钮用于 FormalMC
完整运行启动按钮灰色	检查是否有活动任务、请求是否还在处理、项目数据是否已加载
`Tool not found`	填写执行设备真实工具路径，保存后重新环境检查
许可证不可用	在执行服务设备上配置实际许可证环境，再检查日志
配置冲突 / revision 变化	刷新实际服务状态，核对自己的配置后再保存，不连续盲目重试
界面出现 `passed` 但还提示未完整通过	核对动作是否环境检查 / 自检 / 单任务，查看 `full_suite_passed`
有求解超时	检查任务和日志；必要时调整外部与 SBY 内部预算，再发起新的运行

原 SBY 页的能力说明是什么意思

提示	含义
原生 Yosys assert / assume / cover，并非完整 SVA	当前前端使用其支持的语法；时序规则可以用监视器、历史状态和断言表达
COI 本接入不支持	当前没有提供完整的性质到 RTL 影响锥报告
guard / trigger Cover 不等于完整空洞性	触发条件可达不等于所有断言都已做完语义空洞性分析
活性未建立	当前没有建立在无限等待环境下每个请求最终完成的证明
BMC 无反例不等于无界证明	有限深度没有失败输入，不能当成所有时间范围的安全证明
反例必须真实动态重跑	工具得到失败输入后，实际回放确认问题，才记为已复现

这些是当前接入和项目范围的提示，不是某条新断言报错。当前原任务验收为 107 安全检查点无界证明、65 覆盖、5 环境假设单列和 49 类代表注错回放；没有声称完整 COI、全面空洞性或无限背压下的无条件最终完成。详见 正式验收报告。

14 截图和证据索引

图号	实际画面	本次是否执行验证
01	总览和主导航	否
02	服务连接弹窗	否，仅查看
03	原 SBY 阶段任务列表	否
04	原阶段树和一键入口	否，仅选择 CK 阶段查看
05	原任务检查结果与事件	否，读取既有验收
06	原任务 Agent 工作台	否，未启动 Agent
07	FormalMC 配置与操作台	否，读取既有记录
08	已登记完整验证包列表	否
09	SBY 配置和操作按钮	否
10	22 任务清单	否
11	历史 SBY `quota_math` 单任务结果	否，读取既有运行
12	上述单任务真实日志	否，读取既有运行
13	未保存配置提示	否，仅客户端临时草稿，已恢复
14	历史 FormalMC 缺工具日志	否，读取既有运行

图号	实际画面	本次是否执行验证
15	新阶段任务表单	否，未提交

全部截图在独立虚拟显示中使用实际 Tk 应用和实际执行服务拍摄，PNG 转码保持原始像素、尺寸和颜色。离线 HTML 阅读版的编号和框线是文档叠加层，不修改截图，也不是 GUI 新增按钮。

相应源码为仓库中的 `ucagent-tool/desktop/ucagent_tk/`；拍图记录保存了源码哈希、截图哈希、GET 操作和前后状态检查。本次拍图确认原会话、包配置及运行记录均未改变。

完整验证输入与运行方法见 验证包 README。原始证据归档的完整重组与校验见 仓库首页。

CMDBUF RD Formal 验证说明

内容来源　verification/verilog_20261012/README.md

Verilog 2001 转换与实际重新验证

本次将芯片 RTL 转换为 IEEE 1364-2001 Verilog。当前集成文件为 `rtl/cmdbuf_rd.v`、`rtl/cmdbuf_rd_cmd.v`、`rtl/cmdbuf_rd_otb.v`，详细接口见 RTL 说明。开发和验证在已授权虚拟机 `192.168.31.116` 完成，工作目录为 `/root/cmdbuf_rd/verilog_20261012`。

`rd_mem_status` 的实际端口为 `[15:0]`，保留原 `[1:0][7:0]` 的位序：低 8 位偶 Bank、高 8 位奇 Bank。其余功能、接口位宽、时序、五条窗口和容量保底规则保持一致。

当前实际结果

检查	结果
Icarus 严格 Verilog-2001 编译	仿真诊断启用和 `SYNTHESIS` 两种模式通过，使用 `-g2001`
Verilator 严格 Verilog lint	通过，`--Wall --language 1364-2001`，0 告警
新旧实现顺序等价性	3162 个等价检查项全部证明，0 未证明
既有独立参考模型回归	基础 19/19、补充 4/4 通过
纯 Verilog DUT 与测试台的 Icarus 功能回放	FI001、FI035、FI049 三组原版输入全部通过
当前 `.v` DUT 的完整 SBY 运行	22/22 任务通过，107 项安全无界证明、65 项覆盖、5 项环境假设单列
重建 FormalMC 模型的 SBY 交叉验证	22/22 模型通过，实际模型哈希与当前输入逐项核对
导出模型选择条件审计	22 任务、271 个 pmux、8492 对选择条件满足二态互斥证书

检查	结果
当前 Verilog 原版与变异版的历史输入回放	FI001 ~ FI049 全部检出，原版无语义差异，故障版产生真实差异
既有精确片段注错运行器回归	M01 ~ M12 全部检出，基线和验证输入前后哈希一致

机器可读汇总记录了输入哈希、范围与方法。原始 SBY 结果见 sby_result.json，模型交叉结果见 normalized_result.json，49 类回放见 fault_replay_result.json。

本次等价检查使用 Yosys `equiv_simple`、`equiv_induct -seq 5` 和 `equiv_status -assert`。它证明时钟可观测的二态合成语义，包含 `async2sync` 前端归一化；不声称四态全等价或物理时序签核。Icarus 回放另行实际执行，不能替代上述范围说明。

验证输入与观测器

形式任务中的 DUT 文件已同步为 `.v`。原先内嵌的形式观测器放入 `inputs/formal_hooks/*.svh`，通过 `FORMAL` 条件 include 接入；正式芯片集成使用顶层 `rtl/` 三份文件。

观测器和验证模型保留 SystemVerilog 属性语法。22 个 FormalMC 模型从转换后的当前任务输入重新生成。177 个 CK 的 ID、需求、性质种类和证明供应关系保留；自动生成的性质名称重新对应到当前源位置。对应关系见 转换记录和包内 `provenance/verilog_property_correspondence.json`。

历史 UCAgent 的 14 阶段验收、原始签名和先前失败尝试保留原时间与输入。本次是新的编译、仿真和求解记录，不把旧 PASS 写成新结果。

虚拟机默认 `/root/cmdbuf_rd/rtl` 已切换到当前三份 `.v`；原文件另行保留。GUI 的同名 `cmdbuf_rd` 项目已关联到新封存包，SBY / FormalMC 配置保持原值，原阶段投影逐项核对未变。新服务由 `cmdbuf-rd-ucagent.service` 管理，支持 `systemctl start cmdbuf-rd-ucagent` 恢复。详见 部署记录。GUI 实际执行的 环境检查记录对应当前输入哈希，它不替代上述已完成的完整求解。

已处理的执行环境问题

首次 49 类回放调用了 OSS-CAD 内的 Verilator 5.051 与系统旧 GCC，编译器拒绝相关参数，运行器记录为构建失败。这一轮没有计为检出。使用已验证的 GCC 11 / Verilator 5.024 环境重新执行后，49/49 通过；首次记录与修正后的记录均保留在原始证据归档中。

当前设备仍没有真实 FormalMC。交叉验证和选择条件审计不能替代商业工具的实际执行；目标设备使用包内入口先做能力自检，再运行其完整套件。

证据与复现

- execution_evidence.tar.gz：原始日志、XML、状态、运行输入、见证和回放波形，包含已处理的首次编译失败记录。
- evidence_inventory.json：归档内每个载荷的字节数与 SHA256。
- equivalence.ys、equivalence.log：新旧实现等价流程与真实工具输出。
- legacy_mutation_result.json：12 类精确片段注错运行器的新回归结果。
- pmux_audit.json：当前导出模型的互斥检查证书。

在有对应工具的设备上重新运行当前包：

```
python3 formal/cmdbuf_rd_dual_backend/run.py --backend sby --tool /your/oss-cad-suite/bin/sby --
tool-dir /your/oss-cad-suite/bin --timeout 1200 --sby-timeout 900
python3 formal/cmdbuf_rd_dual_backend/replay.py
```

本目录的开发取证脚本记录了实际虚拟机路径；跨设备入口使用验证包的 `run.py` / `replay.py`。复现顺序等价需要 Git 提交 `3c21c39` 中的旧三份 `.sv` 作为 `baseline/`，以及当前三份 `.v` 作为 `rtl/`。

内容来源　verification/closure/verification_summary.md

cmdbuf_rd 完整验证报告

验证于 2026-10-11 启动，2026-10-12 完成本轮验收。当前已对齐规格中的验证缺口已闭合；全部 14 个 UCAgent 阶段实际完成，正式结果为 **passed**。原始三份 RTL 未修改。

实际结果

项目	结果
功能分解	15 FG → 35 FC → 177 CK
需求关联	335 条，逐项保留并核对
安全检测点	107/107，实际无界证明；含已证组合依赖
业务覆盖	65/65，保留实际场景见证
环境约束	5 项，单列，不计为安全证明
附加 guard 可达性	7/7，单列，不混入 177 CK
故障注入	49/49 个代表变异检出并完成相同输入双版本回放
正式流程	14/14 个启用阶段，实际 Check / Complete

计数按原子检测点给出。合取性质可支持多个明确的 CK；去除别名后为 123 个唯一性质：53 个安全、65 个覆盖、5 个环境约束。辅助不变量和数学供应证明另外保留，不重复堆入上述 CK 数量。

上一轮缺口怎样补齐

原缺口	本轮处理及证据
正式分解只有接口子集，完整调度未进入 FG/FC/CK	重新细化为 177 CK；`spec_catalog.md/json` 列出每项义务、需求与故障关联

原缺口	本轮处理及证据
requirements 在工具读写中丢失	修复本任务隔离工具的字段模型；主会话实际持久化核验 196/196，通过 SHA 固定输入快照
原生属性游离于正式门禁	156 个原生绑定与 21 个 guided 点进入同一会话；逐项校验签名、源文件、模式、property 身份与结果
容量、年龄及全局顺序无界证明未闭合	采用显式组合证明；精确供应性质、guard、任意下标全称范围、状态/端口/时钟对应和无环依赖均核验
场景命中不足或条件过弱	65 个业务覆盖均有见证；重新加强错拍 grant、连续 RR 和 ready 掩码场景后实际重新求解
注错清单缺少实际检出与复现	49 个 RTL 变异逐项取得真实反例；同一外部输入在原版通过，在故障版复现语义错误；交付入口再次全量重放 exit 0
只看阶段完成标志，不能判断实际验证结果	同时要求 14 阶段完成、全部安全/覆盖及依赖闭合、49 类复现成功；公开 API 事件、运行日志和失败尝试一并交付

关键行为的验证范围

- 严格取最老 5 条未成功发射命令再匹配 Bank；前五条都忙时，第六条即使匹配也不得逃逸发射。
- 三周期前向流水、不同命令重叠执行、5A 失败保留后续已选命令、6B 成功同拍窗口前馈，以及选中后 Bank 状态变化。
- 地址 `[17:6]`、Bank `[5:3]`、奇偶位 `[6]`、状态 `[1:0][7:0]`，同 Bank 的双 grant 同拍成功。
- 成功后第 4 拍捕获唯一返回，完整保存 128 位数据与 pc_err；捕获当拍不得 R 旁路，最早下一拍响应。
- 同 XAU 按接受顺序返回；ID 可重复和非单调；只在 ready 的已完成队头之间 RR。
- 元数据与数据归属、地址越界高于 parity 异常、越界仍发 MEM、零数据和单周期返回。
- 九项共享容量、每源保留一项加六项共享、单源最多七项、10B 同拍退休与重用、所有 entry 的完整生命周期。

上游来源公平性没有被假设为已确定。容量保底保证其他来源保有准入空间，不等于保证上游一定送来它们的请求。

UCAgent 的实际执行方式

本轮由 Codex 工程师编写和审查正式规格与属性，通过 UCAgent 公开 read / save / Check / Complete 接口推进完整会话；实际 SBY 求解采用记录在证据中的 Z3、Yices 或 ABC PDR。没有手改阶段状态、求解结果或签名，也没有给原生复杂性质使用恒真占位。

完整阶段为：需求规划、功能理解、功能分组、功能点、检测点、规格审查、属性实现、验证脚本、环境调试、覆盖分析、反例回放、缺陷分析、静态关联、总结审查。

第 10 阶段针对原版 RTL 缺陷的自动反例项在未发现原版缺陷时不适用；49 类故障版 campaign 另由真实回放入口执行。不能从第 10 阶段存在入口文件推断 49 类已经运行。本轮保留了全量重放的独立退出码、日志、脚本哈希及逐项结果。

工具字段、原生证据绑定、读写上限与 SBY 前端修复均限制在本任务隔离环境，并保留补丁、版本和真实回归。原有 UCagent 服务及共享 SBY 不改动。SBY 的未知值处理使用任意值建模，未通过强行置零获得通过。

如何复核

- `index.html`：可搜索全部 CK，显示实际结果与运行编号；包含完整 49 类注错表。
- `spec_catalog.md/json`：正式测试点分解。
- `mutation_results.json`：每个 FI 的真实反例、首个复现差异以及历史失败/超时尝试。
- `final_acceptance.json`：本轮最终只读验收，包含正式结果文件与记录的 SHA256。
- `cmdbuf_rd_evidence.tar.gz`：完整原始证据包；解压后先读 `evidence/README.md`，按说明运行离线 SHA 校验器。

会话 ID：`90855753a21f4dc7a0dc29b7892e38f3`。验收观察到的 revision：`253`。正式覆盖文件 SHA256：`97e99df1b27239d26746621dc2c57c9f8033470dfd6ed3c01bfa8b21e6514951`。正式记录 SHA256：`e7604a1ff74fd15a13d2c9b35fc3e18e5a6156fbf7941528ce153049f3876231`。

旧超时、取消、前端错误和不充分归纳尝试完整保留，但不计入成功证明。部分同版本局部证明按签名、输入及适用范围复用，保留原日期，不冒称全部于本轮重新计算。

边界

本结论适用于已确认的 **9 entry / 5 窗口 / 4 拍 / 128 bit** 配置和输入协议，针对初始复位后不再次复位的事务域。运行中 MEM 协同复位尚未定义；永久 Bank 忙、持续仲裁失败、无限背压及上游不公平条件下不宣称无条件最终完成。

自动 COI 报告和完整语义空洞性分析没有建立，不记作已通过；业务 cover、guard 见证和代表性注错用于检验可达性与检测能力，不等同于穷举全部可能错误。本轮未执行 STA 或物理时序签核。

内容来源　formal/cmdbuf_rd_dual_backend/README.md

cmdbuf_rd SBY　FormalMC 可迁移验证包

此包把当前 `cmdbuf_rd` 的验证输入、检查点映射和证明依赖放在同一目录，使用相对路径运行。新设备不需要原虚拟机、原平台数据库或历史会话。RTL 的接口及功能没有为迁移而修改。

2026-10-12 的当前输入修订为 **Verilog-2001**：芯片 RTL 和注错变异文件为 `.v`，Bank 状态按原位序使用 `[15:0]`。22 个任务的 DUT 输入已同步转换，FormalMC 模型由这些新输入重新生成；原先嵌在 RTL 内的形式观测器移到 `formal_hooks/*.svh`。177 个 CK、性质种类和供应证明依赖保留。转换与对应关系见 `provenance/verilog_2001_conversion.json`、`provenance/verilog_property_correspondence.json`；新验证记录见 Verilog 迁移验证。

在有 FormalMC 的 Linux 设备上运行

要求：Python 3.9 或更新版本、可启动且许可证有效的 FormalMC。沿用当前 UCAgent 的 FormalMC 命令接口；不锁定未经核实的小版本。不需要在 FormalMC 设备安装 SBY、Yosys、Python 第三方库或 UCAgent 服务。

```
tar -xzf cmdbuf_rd_dual_backend.tar.gz
cd cmdbuf_rd_dual_backend
python3 run.py --backend formalmc --tool /your/install/bin/FormalMC
```

工具已在 PATH 时，可以省略 `--tool`。许可证和厂商启动环境继续由目标设备配置；包不携带许可证或登录凭据。运行器会自动先做能力自检，再执行完整证明依赖图。结果进入新建的 `results/` 子目录。

可先单独检查环境及工具能力：

```
python3 run.py --backend formalmc --tool /your/install/bin/FormalMC --preflight
python3 run.py --backend formalmc --tool /your/install/bin/FormalMC --self-test
```

`--preflight` 只检查文件与工具启动；`--self-test` 检查初始值、符号输入、可达覆盖和故意失败的性质。两者都不表示 DUT 已通过。完整运行自动执行同一自检，无需手动先做。

保留 SBY 入口

```
python3 run.py --backend sby --tool /your/oss-cad-suite/bin/sby --tool-dir /your/oss-cad-suite/bin
--timeout 1200 --sby-timeout 900
```

SBY 后端使用原来的 RTL / 监视器输入和证明任务，设备需提供 SBY、Yosys、SMTBMC 及配置所需的 Yices / Z3。此包不会安装或覆盖设备上的 EDA 工具，也不会改写旧项目和已完成的 UCAgent 会话。

`python3 run.py --list` 查看任务。`--job JOB_ID` 自动加入真实证明供应者及相关检查点所需任务；部分运行不会标成完整套件通过。`--timeout 3600` 设置每项任务的外部等待上限；`--sby-timeout 3000` 设置 SBY 自身预算，只调整本次新建副本并记录变更，不改源码、证明深度或性质。

检查点 模型与结果

- 总计 177 个检查点：107 个安全性质、65 个业务覆盖、5 个环境假设。5 个假设不计为证明通过。辅助性质和额外 guard witnesses 随任务一起保留。
- `manifest.json` 是任务依赖、检查点到性质的映射及输入哈希清单。供应者未通过时，依赖它的任务不能作为已证明结果使用。
- `spec_catalog.md` / `spec_catalog.json` 保留需求拆分。`provenance/` 保留迁移输入来源与转换记录。
- FormalMC 使用随包提供的规范化模型。模型在开发机上由与 SBY 相同的前端语义展开，显式保留初始状态、历史寄存器、约束、符号变量及性质，避免目标设备依赖 Yosys 私有语法。原始可读 RTL 和监视器仍随包提供。
- 这是当前已验证设计的快照包。顶层 `rtl/` 用于阅读及注错回放；形式任务读取 `jobs/` 中各自锁定的原始输入或转换模型。仅修改顶层 `rtl/` 不会更新形式任务。设计迭代必须重新生成任务输入和模型，并完成新的验证。
- 每次运行创建新的结果目录，记录工具探测、命令、返回码、日志、逐性质结果和逐检查点结果。超时、缺失结果、未识别状态、编译/许可证错误均不能算通过。旧 SBY PASS 不会被用作新 FormalMC PASS。
- 独立运行器的结果不冒充 UCAgent 的签名阶段验收。原 14 阶段验收是历史 SBY 证据；在新设备执行本包会产生新的独立求解记录。

49 个注错场景的回放

包内保留 FI001 ~ FI049 的注错定义、选定变异 RTL、已归档的外部输入和独立参考模型。目标机装有 Verilator 5.x、C++ 编译器与 make 时可运行：

```
python3 replay.py
python3 replay.py --case FI001 --case FI035 --case FI049
```

每条输入会重新驱动原版与变异版 RTL，要求原版无语义差异、变异版实际出现差异。该入口复用历史 SBY 反例的外部输入，属于新的仿真回放；它不宣称重新生成了 FormalMC 反例，也不替代完整形式证明。

验证边界

本包采用已对齐的初始复位协议，不覆盖尚未定义的运行中 MEM 复位协调。永久 Bank 阻塞、上游缺少公平性等情况不被擅自改成无条件活性保证。严格五条待发窗口及其他已对齐规则继续保留。

交付时实际执行过的测试及结果见 `validation/validation_summary.json`。当前开发设备未安装真实 FormalMC；FormalMC 的完整实机编译、求解和版本兼容性以目标设备的新运行结果为准。启动自检发现不支持的命令或结果格式时会保留日志并停止，不能把此情况解释成 RTL 已通过。

`validation/validation_summary.json` 保留的是转换前的历史迁移验收。当前 Verilog 修订的重新执行记录另存于上述迁移验证目录；历史报告中的旧源码哈希不会改写成新哈希。

迁移验证包括：新入口的原始 SBY 全套回归、标准 SV 模型的 SBY 交叉验证、Verilator 语法检查和运行器测试。模型中并行选择条件的二态结构互斥检查另存于 `validation/audit_pmux_v2.tar.gz`；此检查不是 FormalMC 实测，也不是四态等价证明。超时、早期转换错误及后续修复后的实际运行记录分别保留。

本包的完整性可由外部 `.sha256` 校验；目录内 `release_hashes.json` 覆盖交付文件，形式验证输入另由 `manifest.json` 校验。修改性质或 RTL 后，需要重新生成输入清单和转换模型，再重新验证；不要只改哈希并沿用旧结果。

目录解释：以下是 20261011.1 的正式义务分解，完整列出 15 FG、35 FC 和 177 CK。目录中的 pending 是编制时的原字段；后续执行结果与输入绑定应查看本分册开头的当前 Verilog 验证报告及正式闭环报告，不以此目录的旧字段作为最终运行状态。

内容来源 formal/cmdbuf_rd_dual_backend/spec_catalog.md

cmdbuf_rd 完整正式功能与检测点目录

版本：20261011.1。15 FG / 35 FC / 177 CK：5 项环境约束、107 项安全义务、65 项业务可达见证。**这是一份规格和义务目录，本轮全部验证状态 pending，不是通过报告。**

每个 CK 的稳定 ID、R01–R18、FI001–FI049、期望行为、证明义务和旧证据引用都保存在同名 JSON。native_binding 仍需绑定本轮真实运行的唯一编译属性、源码 hash 和受信证据；不存在恒真占位属性。环境 Assume 的正文是真实输入约束，其 guard=1 仅表示无附加门控。

使用边界

仅初始采样复位：首采样 rst_n=0，此后恒为 1。无运行中带在途事务复位证明。

不假设上游源公平、bank 最终空闲、grant 最终成功或 rready 最终为 1；不承诺无条件最终响应。

每个 CK 定义一个判定关系；向量相等、任意源/任意 entry 量化属于一个关系，多个时序场景单列 cover。环境和 cover 均不计安全证明通过。

采用逻辑保底一项/源与六项共享：sum(max(occ[x],1))<=9。不是固定 3/3/3 分区，也不能仅靠每源 <=7。重复 ID、精确 RR 方向/初值、错误数据透传、物理清零和选择后 status 变化等采用实现行为在合同表单列。

需求映射

需求	名称	环境 / 安全 / cover 数
R01	共享接口与合法握手	2 / 6 / 4
R02	ID/port/address 元数据	0 / 12 / 3
R03	9 项配对分配	0 / 11 / 1
R04	来源保底与 10B 准入	0 / 13 / 8
R05	pending 与生命周期	0 / 13 / 0
R06	最老 5 项窗口	0 / 9 / 5
R07	bank/half 解码和最老候选	0 / 7 / 18
R08	三周期流水及去重	0 / 12 / 4
R09	双 grant 和 5A	0 / 9 / 9
R10	6B 成功前馈	0 / 1 / 1
R11	成功后固定 4 拍	0 / 8 / 3
R12	MEM 单拍捕获及完成同步	0 / 15 / 5
R13	禁止同拍 MEM 到 R 旁路	0 / 3 / 2

需求	名称	环境 / 安全 / cover 数
R14	单源保序与 ready 内 RR	0 / 15 / 11
R15	返回归属及错误优先级	1 / 17 / 5
R16	退休清除和同拍复用	0 / 15 / 6
R17	初始复位及集成边界	2 / 9 / 0
R18	约束、cover 和故障对照	5 / 0 / 65

同一 CK 可关联多个需求；不能累加本表当独立属性总数。

正式 FG FC CK 分解

FG API 环境与证明域

FC ENV RESET 初始复位假设

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-ENV-RESET-FIRST	environment R17,R18 —	首个采样时钟 rst_n=0。 义务：环境假设 init_sample -> !rst_n；不得以复位永久有效令安全性质空洞。	WRAPPER；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-ENV-RESET-RELEASE	environment R17,R18 —	首采样之后 rst_n=1。 义务：环境假设 !init_sample -> rst_n；与运行中复位未定义边界一并披露。	WRAPPER；本轮 pending

FC ENV LEGAL 合法请求与输入约束

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-ENV-SRC	environment R01,R18 FI001	arvalid=1 时 src 仅取 0、1、2。 义务：只约束有 valid 的 src；无 valid 时 src 保持任意。	WRAPPER；本轮 pending
CK-ENV-ARRESP	environment R15,R18 FI036,FI037	arvalid=1 时 arresp 仅取 00 或 11。 义务：不限制 pc_err；不得把 arresp=11 排除。	WRAPPER；本轮 pending
CK-ENV-READY	environment R01,R18 FI001,FI039,FI045	实际 aready=000 时 arvalid=0。 义务：按 DUT 实际 ready 约束，ready 部分为 1 时不强制 src 所指位为 1；ready 正确性另行 assert。	WRAPPER；本轮 pending

FG AR AR 接收与元数据

FC AR HANDSHAKE 共享接收

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-AR-ACCEPT	safety R01 FI001,FI045	每拍接收事件严格等于 arvalid && src<3 && arready[src]。 义务：用外部接口握手与分配事件等价；禁止把 |arready 当目标 ready。	W_ALLOC, O_ACCEPT；本轮 pending
CK-AR-SINGLE	safety R01,R03 FI001,FI009	一次共享 AR 握手恰好创建一条新事务。 义务：独立接收序号/总 live 计数每次仅+1；其余已有身份不受影响。	O_COUNT, O_IDENTITIES, O_VALID；本轮 pending
CK-AR-NOACCEPT	safety R01,R03 FI001,FI045	没有目标源握手时不得新增事务。 义务：按外部 AR 与 R 净事件追踪 live 集合；当非目标 ready 为 1 而目标 ready=0 时也不分配。	O_ACCEPT, O_VALID；本轮 pending

FC AR METADATA 接受时保存并保持元数据

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-AR-ADDR-SAVE	safety R02 FI002,FI007	接收的完整 18 位地址保存到本次事务直到退休。 义务：按接受序号保存任意 araddr；后续实时 AR 变更不得修改活事务地址。	L_ADDR, O_LIVE_META；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-AR-SRC-SAVE	safety R01,R02 FI001,FI007	每条事务始终归属于接收时的 src。 义务：独立接受模型保存 src 并逐活事务比较；不能用当前 src 标记历史事务。	L_SRC, O_LIVE_META；本轮 pending
CK-AR-ID-SAVE	safety R02 FI007,FI032	arid 全部 8 位原样保存，与年龄排序无关。 义务：任意 8 位、重复或非单调 arid 均不约束；活事务 ID 等于其接受值。	L_ID, O_LIVE_META；本轮 pending
CK-AR-PORT-SAVE	safety R02 FI007	port_id 全部 5 位原样保存。 义务：任意 5 位 port 逐事务保存并保持。	L_PORT, O_LIVE_META；本轮 pending
CK-AR-RESP-SAVE	safety R02,R15 FI007,FI036,FI037	arresp 原样保存为各事务的地址状态。 义务：保存接收时的 00 或 11，不能被后续请求覆盖。	L_ARRESP, O_LIVE_META；本轮 pending

FG LIFE CMDBUF OTB 配对与生命周期

FC LIFE ALLOC 原子配对分配

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-LIFE-PAIR	safety R03 FI009,FI023	每个已接收未退休事务独占一组同索引 CMDBUF/OTB 资源。 义务：建立独立 live 身份映射；OTB 存储及返回 token 必须绑定同一身份，不能只证明两个内部 event 相等。	O_VALID, O_IDENTITIES, O_ALLOC_OWNER, D_STORE；需加强：须补全身份/配对的合并义务；旧局部 transition 不单独证明配对整个生命周期。；本轮 pending
CK-LIFE-ALLOC-FREE	safety R03,R16 FI009,FI040	分配只使用当前空闲项或当拍退休的同一项。 义务：任意 alloc_entry 必须非 live，唯一例外为当前外部 R 退休身份。	O_ALLOC_OWNER；本轮 pending
CK-LIFE-RANGE	safety R03 FI003,FI009	所有分配、发射、捕获、退休 entry 索引处于 0 至 8。 义务：分别对实际事件断言索引有效，不能以输入 assume 约束内部索引。	W_RANGE；本轮 pending
CK-LIFE-NEW-STATE	safety R03,R05 FI009,FI040	新事务状态初始化为 valid=1、issued=0、done=0。 义务：AR 握手后一拍比较新身份状态；同拍旧项退休不得覆盖新状态。	L_NEW；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-LIFE-COUNT	safety R03,R05 FI009,FI023,FI041	live 总数等于累计接收数减累计退休数，始终 0 至 9。 义务：独立队列计数与有效集合一一对应；发射、捕获不改变 live 总数。	O_COUNT, O_IDENTITIES, O_VALID；本轮 pending

FC LIFE STATE 已发 完成状态与留存

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-LIFE-ISSUED	safety R05,R09 FI010,FI019,FI020,FI021,FI023	issued 只能由该事务实际双 grant 成功置位。 义务：逐事务 issued 等于独立握手历史，失败、选择和杂散 grant 不得置位。	O_ISSUED, L_ISSUED；本轮 pending
CK-LIFE-DONE	safety R05,R12 FI024,FI028,FI029	done 在该事务固定到期 MEM 捕获时置位并保持。 义务：逐事务 done 由独立成功+4 日历推进，比较 CMDBUFdone；不得以内部 capture 作为唯一期望来源。	O_DONE, O_CAPTURE；本轮 pending
CK-LIFE-HOLD	safety R03,R05 FI023,FI041	从接收至 R 退休，发射成功或 MEM 返回均不能释放所有权。 义务：独立 live 集合只随 AR/R 变化，包含 pending、inflight、done 三类。	O_VALID；本轮 pending
CK-LIFE-NODUP-ISSUE	safety R05,R08 FI010,FI017,FI023	一条事务至多成功发射一次。 义务：身份在成功后至退休不得再次产生成功；重复 ID 仍按不同接受身份区分。	O_ISSUED, O_ISSUE_PENDING；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-LIFE-NODUP-R	safety R05,R16 FI023,FI041	每次 R 只退休一条已接收且已完成的活事务；同一身份不重复返回。 义务：外部 R 源/身份映射到独立队列并删除，不能用 DUTretire 内部事件驱动唯一期望。	O_RVALID, O_RETIRE, O_RETIRE_ID；本轮 pending

FC LIFE PENDING 待发命令数

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-PENDING-EXACT	safety R05 FI022,FI044	pending_cnt 等于全部有效且未成功发射命令数，范围 0 至 9。 义务：独立 live/issued 历史全九项计数，不受五项窗口限制。	O_PENDING；本轮 pending
CK-PENDING-UPDATE	safety R05 FI021,FI022	相邻拍 pending 更新为旧值+当拍 AR 接收-当拍发射成功。 义务：覆盖两事件同时发生和均不发生；选择、失败、捕获及 R 本身不得额外改变。	O_PENDING；本轮 pending

FG CAPACITY 共享容量与来源保底

FC CAPACITY ACCOUNT 按来源统计全部未退休事务

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CAP-OCC	safety R04,R05 FI044,FI048	occ[x]等于来源 x 全部已接收未退休事务数。 义务：独立接受/退休计数，pending/inflight/done 都占容量。	O_OCC, Q_OCC；本轮 pending
CK-CAP-RESERVE	safety R04 FI008,FI043,FI046	所有可达状态满足 sum(max(occ[x],1))<=9。 义务：证明三源全局保底不变量；仅每源<=7 不足以排除(4,5,0)。	Q_RESERVE；本轮 pending
CK-CAP-MAX7	safety R04 FI008,FI046	每个 XAU 的未退休占用不超过 7。 义务：三源任意选择断言 occ<=7，不能限制上游只送某源。	Q_MAX7, O_RESERVE；本轮 pending
CK-CAP-EMPTY-CAN-ACCEPT	safety R04 FI008,FI043,FI046	合法状态下零占用来源在无退休时具有至少一项准入资源。 义务：occ[x]=0 -> aready[x]=1，前提仅 rst_n 与已证保底不变量，不假设公平选择 src。	Q_READY, O_READY；本轮 pending

FC CAPACITY READY 退休后准入计算与 10B

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CAP-READY-FORMULA	safety R04 FI008,FI039,FI043,FI045,FI046,FI047	ready[x]按当拍退休后的空位和共享额度逐源精确计算。 义务：occ_after=occ-retire_src；free_after=9-sum(occ_after)；shared_after=sum(max(occ_after-1,0))；ready[x]=rst_n&&free_after>0&&(occ_after[x]==0||shared_after<6)。	Q_READY, O_READY；本轮 pending
CK-CAP-700	safety R04 FI008,FI046	无退休且占用(7,0,0)时 aready=110，源置换同理。 义务：证明源对称的 7 项单源保底响应，禁止借用其他源保留项。	Q_READY；本轮 pending
CK-CAP-440	safety R04 FI043,FI045	无退休且占用(4,4,0)时 aready=100，源置换同理。 义务：两个活跃源不得消耗第三个空源的一项保留资源。	Q_READY；本轮 pending
CK-CAP-711-UNIQUE	safety R04,R16 FI039,FI047	(7,1,1)中源 1 唯一项当拍退休时 aready=010。 义务：必须保留新空源的保底槽，不能因有物理空位广播 111；源置换同理。	Q_READY；本轮 pending
CK-CAP-711-SHARED	safety R04,R16 FI039,FI048	(7,1,1)中源 0 共享项退休时 aready=111。 义务：当拍归还共享额度可由任意源利用，保底策略不得制造额外气泡。	Q_READY；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CAP-NET-UPDATE	safety R04,R16 FI040,FI048	每源占用同拍按+accept_src-retire_src 合并更新。 义务：同源进出净零、异源各自加减；优先级 if/else 不能丢失任一事件。	O_OCC；本轮 pending
CK-CAP-NONBLOCK-COMB	safety R04 FI045,FI048	aready 只依赖已登记状态和当拍 R 退休，不依赖 arvalid 决定自身。 义务：用组合依赖静态审查或双实例同状态不同 arvalid 证明 ready 相等；本项不得以有约束的单迹恒真替代。	缺失：须新增；采用实现选择；本轮 pending

FG WINDOW 最老五项窗口与 bank 匹配

FC WINDOW RANK 窗口年龄与成员资格

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-WIN-AGE	safety R06 FI012,FI016,FI032,FI040	年龄严格按实际 AR 接受先后排列；重试不改年龄，复用新项最年轻。 义务：独立接受 FIFO 位置定义年龄，不从 RTLolder 矩阵计算期望。	O_AGE；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-WIN-MEMBERS	safety R06 FI010,FI011,FI019,FI049	窗口恰为当前最老 5 条未成功发射命令，少于 5 时全部入窗。 义务：独立顺序模型排除已 issued，当前失败请求仍占名额；当前成功按 6B 处理。	O_WINDOW；本轮 pending
CK-WIN-ISSUED-EXCLUDE	safety R05,R06 FI010,FI023	已成功发射但尚未退休的项不占乱序窗口名额。 义务：issued 保持 live 但不参与 pending 排名；禁止使用最老 5 条未释放项。	O_WINDOW；本轮 pending
CK-WIN-SIXTH-BLOCK	safety R06 FI011,FI049	前 5 条均不匹配时，即使第 6 条匹配也不得被选择。 义务：在稳定或变化 status 下均断言 select 只来自窗口；不因超时、源保底准入或空闲 bank 滑动窗口。	O_WINDOW, O_CANDIDATES, O_SELECT；本轮 pending
CK-WIN-6B	safety R06,R10 FI018	当前发射成功当拍即剔除该命令参与窗口排名。 义务：独立 rank_pool=pending & ~successmask；之前第 6 条可当拍进入判断、下一拍 req。	O_WINDOW, O_SELECT；本轮 pending

FC WINDOW MATCH bank half 解码与最老选择

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-WIN-BANK	safety R07 FI003,FI004	bank 由保存地址[5:3]选择 0 至 7。 义务：对任意地址验证 bank 编号与 req 位置，不能混用 entry 编号。	L_BANK, O_REQ；本轮 pending
CK-WIN-HALF	safety R07 FI002,FI004	half 由保存地址[6]选择 status[half][bank]，0 表示空闲。 义务：对 packed[1:0][7:0]全部 16 个位置进行通用断言和逐位 cover，禁止维度转置或极性反转。	O_CANDIDATES；本轮 pending
CK-WIN-CANDIDATES	safety R06,R07,R08 FI004,FI012,FI017,FI019	候选集合为窗内、目标 half 空闲且不是当前请求身份的命令。 义务：独立候选集合逐位相等；仅当前成功可在排名前删除，失败仍占窗口。	O_CANDIDATES；本轮 pending
CK-WIN-OLDEST-MATCH	safety R06,R07 FI012,FI016	有候选时选择其中按接受顺序最老的一条；无候选不选择。 义务：独立 FIFO 按年龄扫描候选，不以 ID、物理槽、bank 或 src 优先。	O_SELECT；本轮 pending
CK-WIN-STATUS-REQUEST	safety R07,R08 FI004,FI013,FI017	判断拍选中的命令在请求拍不因 status 改变而取消或换命令。 义务：select 寄存进入 req；请求拍是否成功只由当拍对应双 grant 决定。	L_PIPE, O_REQ；采用实现选择；本轮 pending

FG ISSUE 三拍流水 双 grant 与重试

FC ISSUE PIPE 前向流水边界

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-PIPE-SELECT-NEXT	safety R08 FI013,FI014,FI017	判断所得 valid/entry 在恰好下一拍成为请求级。 义务：完整比较 issue_valid/identity 与上一拍独立 select；无选择则下一拍请求为空。	L_PIPE, O_ISSUE；本轮 pending
CK-PIPE-ACCEPT-NOBYPASS	safety R08 FI013,FI042	刚接受的请求不得在同拍参与选择或直接发 req。 义务：新接受身份最早下一拍判断、再下一拍请求；同拍复用亦同。	O_REQ, O_SELECT；本轮 pending
CK-PIPE-OVERLAP	safety R08 FI014	请求 A 与判断 B 可同拍发生，前级输入也可同时接受 C。 义务：每拍独立计算 nextselect，不因当前 issue_valid 统一暂停选择。	O_SELECT, O_ISSUE；本轮 pending
CK-PIPE-CURRENT-EXCLUDE	safety R08,R09 FI015,FI017,FI019	当前正尝试下发的同一命令不能同时再次被选中。 义务：candidate 不含 currentmask，无论当拍 grant 成功或失败。	O_CANDIDATES；本轮 pending
CK-PIPE-REQ-ONEHOT	safety R01,R08 FI003,FI014,FI017	req 始终 onehot0，每拍最多请求一个 bank。 义务：对外部 8 位 req 断言 onehot0。	W_REQ；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-PIPE-REQ-ADDR	safety R02,R07,R08 FI002,FI003,FI007	有效 req 的 12 位 Arbit 地址严格等于该事务接受地址[17:6]。 义务：按独立请求身份取完整保存地址，包括 bit6；不能使用实时 AR 或 11 位[17:7]。	O_ARB_ADDR, L_PIPE；本轮 pending

FC ISSUE GRANT 同拍同 bank 双 grant

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-GRANT-EXACT	safety R09 FI020,FI021	成功严格等于	(req & ar_rd_gnt & rw_rd_gnt)。 义务：同 bank、同周期两个 grant 都有效才成功；无请求、错误 bank、跨拍 grant 均失败。
CK-GRANT-FAIL-STATE	safety R05,R09 FI016,FI019,FI020,FI026	失败尝试不改变该事务的 issued、pending 资格和原始年龄。 义务：按身份比较失败前后独立未发状态；允许并行其他事务的 AR/R 影响总状态。	O_ISSUED, O_AGE；本轮 pending
CK-GRANT-NOSTRAY-TOKEN	safety R09,R11 FI021,FI026	无实际成功就不能为该尝试创建 MEM 返回 token。 义务：独立每成功事件生成一个+4 事件，单 grant 或无 req 不入日历。	T_DELAY, O_CAPTURE；本轮 pending

FC ISSUE 5A 5A 失败与后续预选并行

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-RETRY-5A-B	safety R08,R09 FI015	A 请求失败时，同拍已选 B 仍在下一拍发 req。 义务：条件 A 失败且 B 选择成立，下一拍 req 身份必须为 B，不能 A 抢占或整级 flush。	L_PIPE, O_ISSUE；本轮 pending
CK-RETRY-5A-A	safety R06,R09 FI015,FI016,FI019	A 失败后保持原龄，下一判断拍按正常窗口规则重新参与。 义务：不得无条件立即重发 A；若 A 此时最老匹配且不在当前请求级，应被重新选中。	O_AGE, O_SELECT；本轮 pending
CK-RETRY-ONCE-TOKEN	safety R09,R11 FI017,FI025,FI026	多次失败后首次成功只生成一次该身份的返回事件。 义务：独立成功历史至退休最多一次，失败次数不改变 4 拍基准。	O_ISSUED, O_CAPTURE；本轮 pending

FG MEM 返回日历与完成时序

FC MEM CALENDAR 成功后固定四拍

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-MEM-EXACT4	safety R11 FI025,FI026,FI027	每个成功发射在恰好 4 个 cycle 后产生捕获，且无其他捕获。 义务：建立由外部成功事件启动的独立四拍历史/倒计时，capture_valid 与 due 等价；禁止 3 拍、5 拍或按接受/选择计时。	T_DELAY, O_CAPTURE；本轮 pending
CK-MEM-IDENTITY	safety R11,R12 FI006,FI024,FI030	到期捕获身份等于 4 拍前成功发射身份。 义务：按独立请求队列身份延迟，不使用当前 entry、接受顺序或 ID 作为唯一索引。	T_ID, O_CAPTURE_ID；本轮 pending
CK-MEM-CALENDAR-BUBBLES	safety R11 FI026,FI027	日历每周期推进，空泡保持空泡，不受新请求、占满或 rready 影响。 义务：对任意间断成功序列逐周期检查到期集合，不能仅在 success 时移位。	T_DELAY；本轮 pending
CK-MEM-UNIQUE	safety R11,R12 FI006,FI025,FI027	单组 MEM 每拍至多有一个到期身份。 义务：由单成功/拍和固定时延推出独立 due_count<=1；不能以 assume 限制 DUT 返回。	O_DUE；本轮 pending
CK-MEM-NO-EARLY	safety R11,R17 FI025,FI026	初始复位释放后尚未达到首个成功+4 时不得捕获。 义务：有界历史有效位不足 4 时禁止伪 token；全空闲与全失败同样不捕获。	T_DELAY；本轮 pending

FC MEM COMPLETE 单拍返回与 CMDBUF 同步

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-MEM-DONE-SYNC	safety R12 FI024,FI028	一次合法到期捕获同时使对应 CMDBUF 项完成。 义务：独立 due 后下一状态 done=1，其他未到期项 done 不变；不能仅检查 OTB 存了数据。	O_DONE, L_DONE；本轮 pending
CK-MEM-NO-R-BYPASS	safety R13 FI031	刚在当拍捕获的事务不能在同拍从 R 返回。 义务：R 仅选择周期开始已有 done 的队头；capture 身份与当拍 retire 身份不同。	O_NO_BYPASS, O_RVALID；本轮 pending
CK-MEM-NEXT-R-ELIGIBLE	safety R13,R14 FI031	捕获后的下一拍，该事务在满足同源队头/ready/RR 时可被正常选择。 义务：精确 R 获选关系必须包含上一拍完成；不要求其无条件下一拍返回。	O_DONE, O_RVALID；本轮 pending
CK-MEM-CONCURRENT-OLD-R	safety R12,R13 FI031	捕获 A 不阻止另一已完成事务 B 在同拍正常 R 返回。 义务：R 精确选择不因 capture_valid 统一拉低；A/B 身份必须不同。	O_RVALID；本轮 pending

FG DATA OTB 数据与奇偶错误

FC DATA CAPTURE 全宽单拍采样

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-DATA-CAPTURE128	safety R12 FI006,FI028,FI029	到期拍的 MEM 全部 128 位写入且仅写入该事务 OTB 项。 义务：任意 128 位 MEM 数据，无值约束；每位均须验证，不以低字节、校验和或相同 pattern 代替。	D_STORE；本轮 pending
CK-DATA-PCERR-CAPTURE	safety R12,R15 FI006,FI028,FI038	pc_err 在数据同拍按事务保存。 义务：任意 pc_err 单拍值；与独立 due 身份配对，不能留为实时组合输入。	D_ERR；本轮 pending
CK-DATA-CAPTURE-NOREADY	safety R12 FI028	rready=000 也必须在到期拍完成数据和 pc_err 捕获。 义务：不允许 R 反压令 MEM 单拍数据丢失；错误位与 128 位数据均检查。	D_STORE, D_ERR；本轮 pending
CK-DATA-VALID-NOTVALUE	safety R12 FI029	数据全 0 且 pc_err=0 同样是有效返回。 义务：capture 只由成功+4 日历决定，不能从数据/错误位推断有效。	O_CAPTURE, D_STORE；本轮 pending

FC DATA RETENTION 保持 选路与外部数据归属

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-DATA-HOLD128	safety R12,R15 FI028,FI030	捕获后至退休期间 128 位数据不受后续 MEM 总线变化影响。 义务：任意 entry 若无 alloc/retire/自身 capture，则全宽保持；全局顺序另证。	D_HOLD；本轮 pending
CK-DATA-PCERR-HOLD	safety R12,R15 FI038	每条事务的 pc_err 在捕获后保持至退休。 义务：非本项 capture 及后续其他事务 pc_err 不得覆盖本项。	D_ERR_HOLD；本轮 pending
CK-DATA-R-ROUTE	safety R15 FI005,FI006,FI030	共享 R 的 128 位数据等于本次获选事务保存的数据。 义务：连接外部 rvalid 获选身份→rsp_entry→OTB 全 128 位；必须与独立 R 身份闭合。	W_DATA, D_RDATA, O_RETIRE_ID；本轮 pending
CK-DATA-ENDTOEND128	safety R03,R11,R12,R14,R15 FI006,FI028,FI030,FI032,FI040	任意 R 响应全 128 位等于该事务实际成功+4 拍时的 MEM 数据。 义务：独立接受身份、发射日历、采样 data 及退休四段完整闭合；局部存储/选路 PASS 不能独自充当本性质。	O_RETIRE_ID, T_ID, D_STORE, D_HOLD, W_DATA；需加强：旧证明只有局部 128 位存储/选路 PASS，独立 order 组 TIMEOUT；需同次全局联合证明或受信逐 lemma 组合。；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-DATA-ENDTOEND-ERR	safety R12,R15 FI030,FI037,FI038	响应使用的奇偶错误属于该事务成功+4 拍采样值。 义务：独立 pc_err 历史或已证日历+存储+R 身份组合，禁止只把 DUT rsp_pc_err 当期望来源。	O_RETIRE_ID, T_ID, D_ERR, D_ERR_HOLD；需加强：旧 order 模型 rresp 使用 DUT rsp_pc_err，需独立采样历史或日历/存储/选择组合证明。；本轮 pending

FG RESPONSE 单源队头与 ready 内 RR

FC RESPONSE HEAD 单 XAU 接受顺序

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-R-HEAD	safety R14 FI032,FI034	每个来源只允许最早接收且未退休的事务成为其 R 候选。 义务：独立每源接受队列取头，不按发射顺序、完成顺序或 ID 排序。	O_RVALID, O_RETIRE_ID；本轮 pending
CK-R-HEAD-DONE	safety R14 FI032,FI034	同源队头未完成时，已完成的年轻事务不能越过队头。 义务：源资格=head_valid && head_done && ready；年龄由独立接受序号。	O_RVALID；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-R-CROSS-SRC	safety R14 FI033	某源队头未完成不能阻塞其他已完成且 ready 的来源。 义务：全局 R 选择在三个源各自队头中做精确 RR，不采用全局 FIFO 头阻塞。	O_RVALID；本轮 pending
CK-R-ID-INDEPENDENT	safety R02,R14 FI007,FI032	不同 ID、非单调 ID 和重复 ID 均按同源接收先后返回。 义务：输入不约束 ID 唯一/递增；身份使用独立接受序号而非 arid。	O_RVALID, O_RMETA；采用实现选择；本轮 pending

FC RESPONSE RR RR 精确选择

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-R-ONEHOT	safety R01,R14 FI005,FI034	rvalid[2:0]始终 onehot0。 义务：单共享负载每拍最多授予一个来源。	W_RONEHOT；本轮 pending
CK-R-READY	safety R14 FI034	rvalid 只可置位 rready=1 的来源。 义务：断言(rvalid & ~rready)==0；不采用反压锁定型 valid。	W_RREADY；本轮 pending
CK-R-RR-WINNER	safety R14 FI033,FI034,FI035	有资格来源时，从当前指针起按 0→1→2 环形顺序选首个。 义务：完整 rvalid 等于独立 head/done/ready/RR 计算；不能只证明 onehot 和 ready。	O_RVALID；采用实现选择；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-R-RR-ADVANCE	safety R14 FI035	一次实际 R 退休后指针移到获选来源的下一来源。 义务：参考指针由独立 R 事件更新，非按请求、MEM 捕获或任何非退休事件推进。	O_RRPTR；采用实现选择；本轮 pending
CK-R-RR-HOLD	safety R14 FI034,FI035	没有 R 退休时 RR 指针保持。 义务：包括 rready=000、有数据但头未完成、三个源均空三种情形。	O_RRPTR；采用实现选择；本轮 pending
CK-R-EXACT-ELIGIBILITY	safety R14 FI033,FI034	存在已完成且 ready 的源队头时当拍必须产生一个 R；否则 rvalid=0。 义务：精确获选而非单向合法性，防止永远 rvalid=0 空洞实现。	O_RVALID；本轮 pending

FG RMETA R 负载与错误码

FC RMETA FIELDS 同一获选事务的共享负载

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-R-ID	safety R02,R15 FI005,FI007,FI032	rid 等于本次 R 事务接受时的 8 位 arid。 义务：按独立 R 身份比对全 8 位，重复 ID 不能掩盖 port/地址/数据错配。	O_RMETA；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-R-PORT	safety R02,R15 FI005,FI007	r_port_id 等于本次 R 事务接受时的 5 位 port_id。 义务：按同一独立获选身份比对全 5 位。	O_RMETA；本轮 pending
CK-R-ADDR	safety R02,R15 FI005,FI007	raddr 等于本次 R 事务接受时 araddr[2:0]。 义务：按独立获选身份比对 3 位低地址；不使用下游 12 位地址推导。	O_RMETA；本轮 pending

FC RMETA ERROR 错误优先级与越界正常流转

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-RESP-ADDRERR	safety R15 FI036,FI037	arresp=11 时最终 rresp=11，与 pc_err 无关。 义务：分别保留 pc_err=0/1 两个 cover；安全性质不屏蔽 pc_err=1。	O_RMETA；本轮 pending
CK-RESP-PARITY	safety R15 FI037,FI038	arresp=00 且本事务捕获 pc_err=1 时 rresp=01。 义务：使用该事务历史 pc_err；不能使用实时 MEMpc_err。	O_RMETA, D_ERR；本轮 pending
CK-RESP-NORMAL	safety R15 FI037	arresp=00 且本事务捕获 pc_err=0 时 rresp=00。 义务：按同一事务的保存字段组合，无其他响应码。	O_RMETA, D_ERR；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-RESP-OOB-ISSUE	safety R07,R09,R15 FI036	越界请求与普通请求同样参加选择、发射与 MEM 返回。 义务：选择、成功及到期规则不按 arresp=11 旁路/丢弃；不得本地提前置 done。	O_SELECT, O_SUCCESS, O_DONE；本轮 pending
CK-RESP-ERROR-DATA	safety R12,R15 FI006,FI028,FI037	错误响应的数据仍为该事务捕获的 128 位 MEM 数据。 义务：实现采用错误数据透传；与错误状态码分别验证，不把数据自动清零视为等价。	W_DATA, D_RDATA, O_RETIRE_ID；采用实现选择；本轮 pending

FG REUSE 退休 清除与同拍复用

FC REUSE RETIRE 退休事件和清除

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-RETIRE-EXACT	safety R16 FI005,FI041	退休事件严格等于外部 rvalid 与 rready 握手。 义务：内部 retire 必须与外部握手等价；不能以内部 retire 作为外部正确性的假设。	W_RETIRE, O_RETIRE；本轮 pending
CK-RETIRE-IDENTITY	safety R14,R16 FI030,FI041	退休身份和 OTB 读出身份都等于独立 R 获选身份。 义务：retire_entry/rsp_entry 绑定同一独立队头；其他 entry 不得一并释放。	O_RETIRE_ID；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-RETIRE-CLEAR-STATE	safety R16 FI023,FI041	单独退休后对应 CMDBUF valid/issued/done 清零。 义务：若没有该槽新分配，下一状态全清；别的活事务保持。	L_CLEAR, O_VALID；本轮 pending
CK-RETIRE-CLEAR-DATA	safety R16 FI028,FI038,FI041	退休且未复用的 OTB 数据 128 位与 pc_err 清零。 义务：明确验证物理清零这一采用实现；零值不能作为 valid 判据。	D_CLEAR, D_ERR_CLEAR；采用实现选择；本轮 pending

FC REUSE 10B 当拍释放当拍重新准入

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-REUSE-FULL-ACCEPT	safety R04,R16 FI039,FI047,FI048	满 9 项时有 R 退休可立即按来源额度准入新 AR。 义务：aready 用退休后占用；有合法 AR 时不得固定推迟至下一拍。	O_READY, O_ACCEPT, Q_READY；本轮 pending
CK-REUSE-OLD-R	safety R15,R16 FI007,FI038,FI040	同槽复用当拍对外 R 仍对应旧事务数据、ID、port、raddr 和 rresp。 义务：边沿前 R 取旧身份，不能被实时新 AR 覆盖；逐负载已有原子 CK 均适用。	O_RETIRE_ID, O_RMETA, W_DATA；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-REUSE-NEW-WINS	safety R03,R16 FI040,FI041	同槽退休+接收后，新事务有效状态和元数据初始化优先。 义务：边沿后 valid=1 且 issued/done=0、字段为新 AR；不能退休清除覆盖新分配。	L_NEW, L_ADDR, L_SRC, L_ID, L_PORT, L_ARRESP；本轮 pending
CK-REUSE-NEW-OTB	safety R12,R16 FI038,FI040	同槽复用后新 OTB 数据和 pc_err 初始化为 0，旧返回不可污染。 义务：验证 128 位 data 和 pcerr 重置；新的合法捕获由新 success+4 唯一确定。	D_CLEAR, D_ERR_CLEAR；本轮 pending
CK-REUSE-YOUNGEST	safety R06,R16 FI016,FI040	复用物理 entry 的新事务在所有此前活事务之后。 义务：独立接受 FIFO 追加末尾，禁止沿用旧 age/旧 ID 身份。	O_AGE；本轮 pending
CK-REUSE-NEW-PIPELINE	safety R08,R16 FI013,FI042	复用新事务仍须经历接受、判断、请求三个 cycle。 义务：旧 slot 索引不能令新命令绕过判断寄存或继承旧 issue token。	O_REQ, O_SELECT, O_CAPTURE；本轮 pending

FG RESET 初始复位与无效输出

FC RESET INITIAL 初始复位结果

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-RESET-EMPTY	safety R03,R17 —	首个有效工作状态无活事务、已发项和完成项。 义务：在初始复位释放后检查独立 live 集合为空；不把运行中复位假装已验证。	O_VALID, O_ISSUED, O_DONE；本轮 pending
CK-RESET-REQ	safety R08,R17 —	初始复位期间及无历史选择时 req=0。 义务：检查流水 valid 清零，不产生伪请求。	G_RESET_REQ；本轮 pending
CK-RESET-R	safety R14,R17 —	初始复位期间 rvalid=000，无虚假 R 退休。 义务：复位后未有完成事务前 R 仍为零。	G_RESET_R, O_RVALID；本轮 pending
CK-RESET-PENDING	safety R05,R17 —	复位后 pending_cnt=0。 义务：全部 9 项未发数初值为零。	G_RESET_PENDING, O_PENDING；本轮 pending
CK-RESET-READY	safety R04,R17 —	复位期间 aready=000，释放且空表时 aready=111。 义务：区分复位门控与首个工作态准入。	G_RESET_AR, G_RESET_READY, Q_READY；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-RESET-RR	safety R14,R17 —	初始 RR 指针为源 0。 义务：与 0→1→2 实施选择一致；属于采用实现，不宣称原始需求定义所有 RR 细节。	O_RRPTR；采用实现选择；本轮 pending

FC RESET IDLE 无效负载约定

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-IDLE-ARB	safety R02,R17 —	req=0 时 arb_addr=0。 义务：只声明采用的无效输出清零约定，不将其作为成功/有效判断条件。	G_IDLE_ADDR；采用实现选择；本轮 pending
CK-IDLE-RPAYLOAD	safety R02,R15,R17 —	rvalid=000 时 rdata/rid/r_port_id/raddr/rresp 全部为 0。 义务：向量零化为一个总线空闲关系；有效负载功能仍由各事务 CK 证明。	G_IDLE_R, W_DATA；采用实现选择；本轮 pending

FG COVER TRANSACTION 接口 流水和错误可达性

FC COVER API 逐来源与拒绝后恢复

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-AR0	cover R01,R18 FI001	可达源 0 一次合法 AR 接收。 义务：cover 真实外部握手且 src=0；不是仅 arvalid。	G_COVER_AR0；本轮 pending
CK-CV-AR1	cover R01,R18 FI001	可达源 1 一次合法 AR 接收。 义务：cover 真实外部握手且 src=1。	G_COVER_AR1；本轮 pending
CK-CV-AR2	cover R01,R18 FI001	可达源 2 一次合法 AR 接收。 义务：cover 真实外部握手且 src=2。	G_COVER_AR2；本轮 pending
CK-CV-R0	cover R14,R18 FI005	可达源 0 一次合法 R 退休。 义务：cover 外部 rvalid[0]&&rready[0]。	G_COVER_R0；本轮 pending
CK-CV-R1	cover R14,R18 FI005	可达源 1 一次合法 R 退休。 义务：cover 外部 rvalid[1]&&rready[1]。	G_COVER_R1；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-R2	cover R14,R18 FI005	可达源 2 一次合法 R 退休。 义务：cover 外部 rvalid[2]&&rready[2]。	G_COVER_R2；本轮 pending
CK-CV-UNREADY-SRC	cover R01,R04,R18 FI001,FI045	部分 arready 有效但 valid 指向未 ready 来源，未接收；之后切 src 成功接收。 义务：记录拒绝与后续另一 src 握手，输入始终满足全零 ready 时 valid 为 0。	缺失：须新增；本轮 pending

FC COVER PIPELINE 三拍 并行 重试

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-MIN3	cover R08,R18 FI013,FI042	某身份 C1 接受、C2 判断、C3 请求成功。 义务：cover 同一接受身份连续三拍关系，不能三个无关事务事件拼接。	缺失：须新增；本轮 pending
CK-CV-PIPE-ALL	cover R08,R11,R12,R16,R18 FI014,FI027,FI031,FI048	同拍出现接受、判断、成功、MEM 捕获和 R 退休。 义务：五个合法阶段边界同时成立，各自身份按年龄和日历合法。	C_ALL_EVENTS；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-ISSUE-CONSECUTIVE	cover R08,R18 FI014	相邻周期发射两条不同事务成功。 义务：cover 连续 success 且独立身份不同，避免同条重发充当吞吐。	缺失：须新增；本轮 pending
CK-CV-5A-B	cover R09,R18 FI015	A 失败同拍选 B，下一拍 B 请求。 义务：追踪 A/B 身份，B 不能被失败 A 抢占。	C_5A_B；本轮 pending
CK-CV-5A-A	cover R09,R18 FI015,FI016	A 失败后的下一判断拍重新选 A，并再下一拍 A 请求。 义务：原旧 cover 只见重新选择；本点补齐直到 A 再次 req 的身份链。	C_5A_A；需加强：旧 cover 止于重新选择 A，新义务包含下一拍请求 A。；本轮 pending
CK-CV-RETRY-TO-R	cover R09,R11,R16,R18 FI015,FI016,FI026	同一身份至少失败一次，随后成功、捕获、最终 R 退休。 义务：失败身份必须与最后退休一致，不能任意其他事务退休代替。	C_RETRY_R；本轮 pending
CK-CV-AR-ONLY-GRANT	cover R09,R18 FI020	有 req 且仅目标 ar_rd_gnt=1，失败。 义务：显式失败 cover，不约束全局 grant onehot。	G_COVER_ARONLY；本轮 pending
CK-CV-RW-ONLY-GRANT	cover R09,R18 FI020	有 req 且仅目标 rw_rd_gnt=1，失败。 义务：显式单边读冲裁失败 cover。	G_COVER_RWONLY；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-WRONG-GRANT	cover R09,R18 FI020,FI021	非请求 bank 的两个 grant 同时为 1 仍失败。 义务：目标 grant 未双有效，同时另一 bank 双有效。	G_COVER_WRONGBANK；本轮 pending
CK-CV-STAGGER-GRANT	cover R09,R18 FI020	同一失败尝试中两 grant 分开出现在不同时钟，不能合成成功。 义务：追踪相邻尝试/源身份并证明每次单边时都未成功；合法重试另行 cover。	缺失：须新增；本轮 pending
CK-CV-STRAY-GRANT	cover R09,R18 FI021	req=0 时出现双 grant，无成功及对应 token。 义务：见证杂散 grant 后 4 拍无由该 grant 产生的捕获；可允许其他旧合法 token 到期。	缺失：须新增；本轮 pending

FC COVER ERROR 错误组合与数据值

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-ERR00	cover R15,R18 FI037	arresp=00、采样 pc_err=0，最终 rresp=00。 义务：同一事务从接受/采样到 R，禁止只 cover 当拍组合输入。	缺失：须新增；本轮 pending
CK-CV-ERR01	cover R15,R18 FI037,FI038	arresp=00、采样 pc_err=1，最终 rresp=01。 义务：同一事务错误历史跟踪直到 R。	缺失：须新增；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-ERR10	cover R15,R18 FI036,FI037	arresp=11、采样 pc_err=0，仍发 MEM 并最终 rresp=11。 义务：同一事务已真实发射且+4 捕获后 R。	缺失：须新增；本轮 pending
CK-CV-ERR11	cover R15,R18 FI036,FI037,FI038	arresp=11、采样 pc_err=1，最终 rresp=11。 义务：同一事务同时两种异常证明地址优先场景可达。	缺失：须新增；本轮 pending
CK-CV-ZERO-DATA	cover R12,R18 FI029	到期采样全 0 数据且 pc_err=0 后最终正常 R。 义务：旧局部 cover 只到 capture；本点要求对应事务最终 R，不能被值零误判无效。	C_ZERO_DATA；需加强：旧 cover 仅在采样时见零数据，新义务需对应身份走到 R。；本轮 pending
CK-CV-UPPER-DATA	cover R12,R15,R18 FI006,FI028	一条事务高 64 位和低 64 位不同且高位非零，R 完整透传。 义务：cover 区分高半部截断；128 位安全证明仍需任意向量。	缺失：须新增；本轮 pending

FG COVER BOUNDARY 窗口 资源 响应边界可达性

FC COVER WINDOW 窗口与 ID 身份

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-SIXTH-BLOCKED	cover R06,R18 FI011	前 5 条不能匹配、第 6 条可匹配，status 连续至少 2 拍不变且无请求级干扰，持续不发。 义务：cover 至少 6pending、当前及前拍无请求、全部窗内不匹配、第 6 空闲；安全 CK 保证任意延续长度仍不越窗。	C_SIXTH；本轮 pending
CK-CV-6B	cover R06,R10,R18 FI018	成功 A 当拍前馈使原第 6 条 B 进入判断。 义务：记录 B 不在未前馈基础窗口、A 成功且 B 被选，下一拍发 B。	C_6B；需加强：旧 cover 止于原第 6 项被选择，新义务还要求下一拍请求身份相同。；本轮 pending
CK-CV-FAIL-KEEPS-WINDOW	cover R06,R09,R18 FI019	A 失败仍占窗口，原第 6 条虽空闲也不能提前进入。 义务：见证失败与 5 条排名同时成立；不能把失败当成功剔除。	缺失：须新增；本轮 pending
CK-CV-QUOTA-NO-ESCAPE	cover R04,R06,R18 FI049	源 0 的 5 条老命令均忙，源 1 保底接收第 6 条空闲命令仍不发。 义务：同时覆盖接收资源独立和严格 5 项窗口阻塞。	缺失：须新增；本轮 pending
CK-CV-OUT-OF-ORDER	cover R06,R07,R18 FI012,FI030	较年轻命令在更老 pending 命令前成功发射。 义务：cover 确有更老未发身份，不仅不同物理 slot。	C_OOO；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-DUPLICATE-ID	cover R02,R14,R18 FI007,FI032	同源两条同时 live 且 arid 相同，元数据/数据不同，依接受顺序 R 返回。 义务：跟踪两条独立序号并直至两次响应，禁止 ID 唯一 assume。	缺失：须新增；本轮 pending
CK-CV-NONMONOTONIC-ID	cover R02,R14,R18 FI007,FI032	同源后接收 ID 数值较小的事务仍后返回。 义务：至少两条 live、ID 逆序且两者 R 先后可观察。	缺失：须新增；本轮 pending
CK-CV-CROSS-SRC-ID	cover R02,R14,R18 FI007,FI032,FI033	不同来源同 ID 并存且都能正确返回。 义务：来源/port/地址区分事务，不只按 ID 匹配。	缺失：须新增；本轮 pending
CK-CV-STATUS-FLIP	cover R07,R08,R18 FI004,FI013	判断拍目标 half 空闲，请求拍变忙，但双 grant 成功。 义务：同一身份记录 status 翻转，仍保持请求并成功。	缺失：须新增；本轮 pending

FC COVER CAPACITY 保底与满表复用

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-CAP700	cover R04,R18 FI008,FI046	达到(7,0,0)且无退休，aready=110。 义务：包含 ready 值，旧 cover 只有占用状态不足以单独覆盖完整点。	C_700；需加强：旧 cover 仅占用 700，新义务同时要求无退休与 ready110。；本轮 pending
CK-CV-CAP440	cover R04,R18 FI043,FI045	达到(4,4,0)且无退休，aready=100。 义务：禁止把非法(4,5,0)设为 cover 目标；该状态应不可达。	C_440；需加强：旧 cover 仅占用 440，新义务同时要求无退休与 ready100。；本轮 pending
CK-CV-CAP711	cover R04,R18 FI008,FI043	达到(7,1,1)且无退休，aready=000、arvalid=0。 义务：满 9 项见证不得违反全零 ready 输入限制。	缺失：须新增；本轮 pending
CK-CV-UNIQUE-REFILL	cover R04,R16,R18 FI039,FI047	满表(7,1,1)源 1 唯一项退休，ready=010 并接收源 1。 义务：合法同拍 10B 见证。	C_UNIQUE_REFILL；本轮 pending
CK-CV-SHARED-REFILL	cover R04,R16,R18 FI039,FI048	满表源 0 共享项退休，ready=111 并接收另一源。 义务：同拍退休与接收来源不同，下一占用如(6,1,2)。	缺失：须新增；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-SAME-SRC-NETZERO	cover R04,R16,R18 FI040,FI048	单源 7 项时同拍退休一项且接收同源新项，占用仍 7。 义务：比较前后占用及新旧接受身份，不以计数没变掩盖丢请求。	C_SEVEN_REFILL；本轮 pending
CK-CV-FULL-SLOT-REUSE	cover R03,R16,R18 FI039,FI040,FI041	满表时同一物理槽退休+新分配，随后新事务独立完成。 义务：旧 cover 只到当拍 reuse；本点补新身份正常路径至 R。	C_FULL_REUSE；需加强：旧 cover 只到同槽复用当拍，新义务需新事务后续完整退休。；本轮 pending

FC COVER MEM 数据捕获与 R 边界

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-CAPTURE-NOREADY	cover R12,R18 FI028	MEM 到期单拍返回时 rready=000，之后同身份数据正确返回。 义务：捕获后 MEM 任意变化、后续 ready 释放；不得用 ready 约束保障不丢失。	缺失：须新增；本轮 pending
CK-CV-CALENDAR-BUBBLE	cover R11,R18 FI026,FI027	两个成功之间至少一个空泡，相应到期序列也保留空泡。 义务：逐成功身份与+4 时钟关系，不把发射序号当时间。	缺失：须新增；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-MEM-NEXT-R	cover R13,R18 FI031	某身份在 MEM 捕获后恰好下一拍 R 返回。 义务：capture 身份与下一拍 retire 身份相同，上一拍不得已返回。	C_NEXT_R；本轮 pending
CK-CV-MEM-OTHER-R	cover R12,R13,R18 FI031	捕获 A 的同一拍返回之前已完成 B。 义务：两个不同身份，A 最早后续拍 R。	缺失：须新增；本轮 pending

FC COVER RR 保序和轮询见证

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-YOUNGER-DONE	cover R14,R18 FI032,FI034	同源年轻事务已完成但老队头尚未完成，年轻事务不 R。 义务：跟踪至少两条同源不同接受身份。	C_YOUNGER_DONE；本轮 pending
CK-CV-REORDER-TO-ORDER	cover R14,R18 FI030,FI032	同源 B 晚接受先完成，最终 A 先 R、B 再 R。 义务：必须同一 A/B 对完成完整重排恢复；旧 RETIRE_AFTER_REORDER 仅任意退休不足。	C_REORDER_R；需加强：旧 cover 见过乱序后任意退休不足，必须同一 A/B 对恢复为 A 先 B 后。；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-CROSS-SRC-PROGRESS	cover R14,R18 FI033	源 0 队头未完成时源 1 完成队头 R 成功。 义务：局部有限见证，不声称无条件最终公平。	缺失：须新增；本轮 pending
CK-CV-RR-ROTATE	cover R14,R18 FI035	三源队头均完成且 ready 时连续三次服务按 0→1→2 环序。 义务：固定起始指针或记录实际起始源，三次均有资格；旧 ALL_READY 单次见证不足。	C_RR_ALL；需加强：旧 cover 只一次三个源均有资格，不等于连续三次轮转。；本轮 pending
CK-CV-RR-MASKS	cover R14,R18 FI034,FI035	有完成数据时 ready 依次 110、001、000、100，观察跳过、空拍和恢复。 义务：四拍连续见证，模型 RR 保持/推进正确；不是四个无关单拍 cover。	缺失：须新增；本轮 pending

FG COVER DECODE 16 个 bank half 精确可达性

FC COVER EVEN 偶 half 的 8 个 bank

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-H0-B0	cover R07,R18 FI002,FI003,FI004	只有 status[0][0]=0、其余 15 位=1 时，对应 half/bank 事务可被选中并成功发射。 义务：记录该事务判断拍的唯一空闲模式和下一拍正确 req[0]、arb_addr[0]=0 以及同 bank 双 grant；每个组合独立见证。	缺失：须新增；本轮 pending
CK-CV-H0-B1	cover R07,R18 FI002,FI003,FI004	只有 status[0][1]=0、其余 15 位=1 时，对应 half/bank 事务可被选中并成功发射。 义务：记录该事务判断拍的唯一空闲模式和下一拍正确 req[1]、arb_addr[0]=0 以及同 bank 双 grant；每个组合独立见证。	缺失：须新增；本轮 pending
CK-CV-H0-B2	cover R07,R18 FI002,FI003,FI004	只有 status[0][2]=0、其余 15 位=1 时，对应 half/bank 事务可被选中并成功发射。 义务：记录该事务判断拍的唯一空闲模式和下一拍正确 req[2]、arb_addr[0]=0 以及同 bank 双 grant；每个组合独立见证。	缺失：须新增；本轮 pending
CK-CV-H0-B3	cover R07,R18 FI002,FI003,FI004	只有 status[0][3]=0、其余 15 位=1 时，对应 half/bank 事务可被选中并成功发射。 义务：记录该事务判断拍的唯一空闲模式和下一拍正确 req[3]、arb_addr[0]=0 以及同 bank 双 grant；每个组合独立见证。	缺失：须新增；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-H0-B4	cover R07,R18 FI002,FI003,FI004	只有 status[0][4]=0、其余 15 位=1 时，对应 half/bank 事务可被选中并成功发射。 义务：记录该事务判断拍的唯一空闲模式和下一拍正确 req[4]、arb_addr[0]=0 以及同 bank 双 grant；每个组合独立见证。	缺失：须新增；本轮 pending
CK-CV-H0-B5	cover R07,R18 FI002,FI003,FI004	只有 status[0][5]=0、其余 15 位=1 时，对应 half/bank 事务可被选中并成功发射。 义务：记录该事务判断拍的唯一空闲模式和下一拍正确 req[5]、arb_addr[0]=0 以及同 bank 双 grant；每个组合独立见证。	缺失：须新增；本轮 pending
CK-CV-H0-B6	cover R07,R18 FI002,FI003,FI004	只有 status[0][6]=0、其余 15 位=1 时，对应 half/bank 事务可被选中并成功发射。 义务：记录该事务判断拍的唯一空闲模式和下一拍正确 req[6]、arb_addr[0]=0 以及同 bank 双 grant；每个组合独立见证。	缺失：须新增；本轮 pending
CK-CV-H0-B7	cover R07,R18 FI002,FI003,FI004	只有 status[0][7]=0、其余 15 位=1 时，对应 half/bank 事务可被选中并成功发射。 义务：记录该事务判断拍的唯一空闲模式和下一拍正确 req[7]、arb_addr[0]=0 以及同 bank 双 grant；每个组合独立见证。	缺失：须新增；本轮 pending

FC COVER ODD 奇 half 的 8 个 bank

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-H1-B0	cover R07,R18 FI002,FI003,FI004	只有 status[1][0]=0、其余 15 位=1 时，对应 half/bank 事务可被选中并成功发射。 义务：记录该事务判断拍的唯一空闲模式和下一拍正确 req[0]、arb_addr[0]=1 以及同 bank 双 grant；每个组合独立见证。	缺失：须新增；本轮 pending
CK-CV-H1-B1	cover R07,R18 FI002,FI003,FI004	只有 status[1][1]=0、其余 15 位=1 时，对应 half/bank 事务可被选中并成功发射。 义务：记录该事务判断拍的唯一空闲模式和下一拍正确 req[1]、arb_addr[0]=1 以及同 bank 双 grant；每个组合独立见证。	缺失：须新增；本轮 pending
CK-CV-H1-B2	cover R07,R18 FI002,FI003,FI004	只有 status[1][2]=0、其余 15 位=1 时，对应 half/bank 事务可被选中并成功发射。 义务：记录该事务判断拍的唯一空闲模式和下一拍正确 req[2]、arb_addr[0]=1 以及同 bank 双 grant；每个组合独立见证。	缺失：须新增；本轮 pending
CK-CV-H1-B3	cover R07,R18 FI002,FI003,FI004	只有 status[1][3]=0、其余 15 位=1 时，对应 half/bank 事务可被选中并成功发射。 义务：记录该事务判断拍的唯一空闲模式和下一拍正确 req[3]、arb_addr[0]=1 以及同 bank 双 grant；每个组合独立见证。	缺失：须新增；本轮 pending

稳定 CK	类型 / 需求 / 故障	期望行为和应有证据	现有属性关系
CK-CV-H1-B4	cover R07,R18 FI002,FI003,FI004	只有 status[1][4]=0、其余 15 位=1 时，对应 half/bank 事务可被选中并成功发射。 义务：记录该事务判断拍的唯一空闲模式和下一拍正确 req[4]、arb_addr[0]=1 以及同 bank 双 grant；每个组合独立见证。	缺失：须新增；本轮 pending
CK-CV-H1-B5	cover R07,R18 FI002,FI003,FI004	只有 status[1][5]=0、其余 15 位=1 时，对应 half/bank 事务可被选中并成功发射。 义务：记录该事务判断拍的唯一空闲模式和下一拍正确 req[5]、arb_addr[0]=1 以及同 bank 双 grant；每个组合独立见证。	缺失：须新增；本轮 pending
CK-CV-H1-B6	cover R07,R18 FI002,FI003,FI004	只有 status[1][6]=0、其余 15 位=1 时，对应 half/bank 事务可被选中并成功发射。 义务：记录该事务判断拍的唯一空闲模式和下一拍正确 req[6]、arb_addr[0]=1 以及同 bank 双 grant；每个组合独立见证。	缺失：须新增；本轮 pending
CK-CV-H1-B7	cover R07,R18 FI002,FI003,FI004	只有 status[1][7]=0、其余 15 位=1 时，对应 half/bank 事务可被选中并成功发射。 义务：记录该事务判断拍的唯一空闲模式和下一拍正确 req[7]、arb_addr[0]=1 以及同 bank 双 grant；每个组合独立见证。	缺失：须新增；本轮 pending

49 类故障逐项映射

下表每类至少对应一个安全义务。原始注错变体、最小激励、检查意图和时序图编号完整保留在 JSON；这里没有声称变异已运行。

故障	名称	安全 CK	cover CK
FI001	共享 AR 的接收与源归属	CK-AR-ACCEPT, CK-AR-SINGLE, CK-AR-NOACCEPT, CK-AR-SRC-SAVE	CK-CV-AR0, CK-CV-AR1, CK-CV-AR2, CK-CV-UNREADY-SRC
FI002	Arbit 地址保留奇偶位	CK-AR-ADDR-SAVE, CK-WIN-HALF, CK-PIPE-REQ-ADDR	CK-CV-H0-B0, CK-CV-H0-B1, CK-CV-H0-B2, CK-CV-H0-B3, CK-CV-H0-B4, CK-CV-H0-B5, CK-CV-H0-B6, CK-CV-H0-B7, CK-CV-H1-B0, CK-CV-H1-B1, CK-CV-H1-B2, CK-CV-H1-B3, CK-CV-H1-B4, CK-CV-H1-B5, CK-CV-H1-B6, CK-CV-H1-B7

故障	名称	安全 CK	cover CK
FI003	八个 Bank 的编号解码	CK-LIFE-RANGE, CK-WIN-BANK, CK-PIPE-REQ-ONEHOT, CK-PIPE-REQ-ADDR	CK-CV-H0-B0, CK-CV-H0-B1, CK-CV-H0-B2, CK-CV-H0-B3, CK-CV-H0-B4, CK-CV-H0-B5, CK-CV-H0-B6, CK-CV-H0-B7, CK-CV-H1-B0, CK-CV-H1-B1, CK-CV-H1-B2, CK-CV-H1-B3, CK-CV-H1-B4, CK-CV-H1-B5, CK-CV-H1-B6, CK-CV-H1-B7
FI004	rd_mem_status 维度与空闲极性	CK-WIN-BANK, CK-WIN-HALF, CK-WIN-CANDIDATES, CK-WIN-STATUS-REQUEST	CK-CV-STATUS-FLIP, CK-CV-H0-B0, CK-CV-H0-B1, CK-CV-H0-B2, CK-CV-H0-B3, CK-CV-H0-B4, CK-CV-H0-B5, CK-CV-H0-B6, CK-CV-H0-B7, CK-CV-H1-B0, CK-CV-H1-B1, CK-CV-H1-B2, CK-CV-H1-B3, CK-CV-H1-B4, CK-CV-H1-B5, CK-CV-H1-B6, CK-CV-H1-B7

故障	名称	安全 CK	cover CK
FI005	共享 R 负载与唯一接收对象	CK-DATA-R-ROUTE, CK-R-ONEHOT, CK-R-ID, CK-R-PORT, CK-R-ADDR, CK-RETIRE-EXACT	CK-CV-R0, CK-CV-R1, CK-CV-R2
FI006	单组 MEM 返回接口	CK-MEM-IDENTITY, CK-MEM-UNIQUE, CK-DATA-CAPTURE128, CK-DATA-PCERR-CAPTURE, CK-DATA-R-ROUTE, CK-DATA-ENDTOEND128, CK-RESP-ERROR-DATA	CK-CV-UPPER-DATA
FI007	事务元数据完整保存与透传	CK-AR-ADDR-SAVE, CK-AR-SRC-SAVE, CK-AR-ID-SAVE, CK-AR-PORT-SAVE, CK-AR-RESP-SAVE, CK-PIPE-REQ-ADDR, CK-R-ID-INDEPENDENT, CK-R-ID, CK-R-PORT, CK-R-ADDR, CK-REUSE-OLD-R	CK-CV-DUPLICATE-ID, CK-CV-NONMONOTONIC-ID, CK-CV-CROSS-SRC-ID
FI008	共享池防止单个 XAU 独占容量	CK-CAP-RESERVE, CK-CAP-MAX7, CK-CAP-EMPTY-CAN-ACCEPT, CK-CAP-READY-FORMULA, CK-CAP-700	CK-CV-CAP700, CK-CV-CAP711
FI009	CMDBUF 与 OTB 原子分配和一一对应	CK-AR-SINGLE, CK-LIFE-PAIR, CK-LIFE-ALLOC-FREE, CK-LIFE-RANGE, CK-LIFE-NEW-STATE, CK-LIFE-COUNT	由相关序列 cover 及 guard 见证补查
FI010	窗口排除已成功发射但未退休命令	CK-LIFE-ISSUED, CK-LIFE-NODUP-ISSUE, CK-WIN-MEMBERS, CK-WIN-ISSUED-EXCLUDE	由相关序列 cover 及 guard 见证补查
FI011	五项全部阻塞时不得越窗选择第六项	CK-WIN-MEMBERS, CK-WIN-SIXTH-BLOCK	CK-CV-SIXTH-BLOCKED
FI012	窗口内选择最老可用命令	CK-WIN-AGE, CK-WIN-CANDIDATES, CK-WIN-OLDEST-MATCH	CK-CV-OUT-OF-ORDER

故障	名称	安全 CK	cover CK
FI013	接收判断请求的三拍路径	CK-WIN-STATUS-REQUEST, CK-PIPE-SELECT-NEXT, CK-PIPE-ACCEPT-NOBYPASS, CK-REUSE-NEW-PIPELINE	CK-CV-MIN3, CK-CV-STATUS-FLIP
FI014	不同命令之间的流水重叠	CK-PIPE-SELECT-NEXT, CK-PIPE-OVERLAP, CK-PIPE-REQ-ONEHOT	CK-CV-PIPE-ALL, CK-CV-ISSUE-CONSECUTIVE
FI015	5A 仲裁失败保留下一条预选	CK-PIPE-CURRENT-EXCLUDE, CK-RETRY-5A-B, CK-RETRY-5A-A	CK-CV-5A-B, CK-CV-5A-A, CK-CV-RETRY-TO-R
FI016	失败重试保留原年龄	CK-WIN-AGE, CK-WIN-OLDEST-MATCH, CK-GRANT-FAIL-STATE, CK-RETRY-5A-A, CK-REUSE-YOUNGEST	CK-CV-5A-A, CK-CV-RETRY-TO-R
FI017	屏蔽当前请求级命令的重复预选	CK-LIFE-NODUP-ISSUE, CK-WIN-CANDIDATES, CK-WIN-STATUS-REQUEST, CK-PIPE-SELECT-NEXT, CK-PIPE-CURRENT-EXCLUDE, CK-PIPE-REQ-ONEHOT, CK-RETRY-ONCE-TOKEN	由相关序列 cover 及 guard 见证补查
FI018	6B 成功当拍前移窗口	CK-WIN-6B	CK-CV-6B
FI019	请求失败不能提前腾出窗口名额	CK-LIFE-ISSUED, CK-WIN-MEMBERS, CK-WIN-CANDIDATES, CK-PIPE-CURRENT-EXCLUDE, CK-GRANT-FAIL-STATE, CK-RETRY-5A-A	CK-CV-FAIL-KEEPS-WINDOW

故障	名称	安全 CK	cover CK
FI020	目标 Bank 双 grant 同拍成功	CK-LIFE-ISSUED, CK-GRANT-EXACT, CK-GRANT-FAIL-STATE	CK-CV-AR-ONLY-GRANT, CK-CV-RW-ONLY-GRANT, CK-CV-WRONG-GRANT, CK-CV-STAGGER-GRANT
FI021	无请求时 grant 不得制造成功	CK-LIFE-ISSUED, CK-PENDING-UPDATE, CK-GRANT-EXACT, CK-GRANT-NOSTRAY-TOKEN	CK-CV-WRONG-GRANT, CK-CV-STRAY-GRANT
FI022	待发计数统计全部未成功命令	CK-PENDING-EXACT, CK-PENDING-UPDATE	由相关序列 cover 及 guard 见证补查
FI023	命令保留到 R 返回	CK-LIFE-PAIR, CK-LIFE-COUNT, CK-LIFE-ISSUED, CK-LIFE-HOLD, CK-LIFE-NODUP-ISSUE, CK-LIFE-NODUP-R, CK-WIN-ISSUED-EXCLUDE, CK-RETIRE-CLEAR-STATE	由相关序列 cover 及 guard 见证补查
FI024	MEM 完成同时通知 CMDBUF	CK-LIFE-DONE, CK-MEM-IDENTITY, CK-MEM-DONE-SYNC	由相关序列 cover 及 guard 见证补查
FI025	固定四拍从成功接收起算	CK-RETRY-ONCE-TOKEN, CK-MEM-EXACT4, CK-MEM-UNIQUE, CK-MEM-NO-EARLY	由相关序列 cover 及 guard 见证补查
FI026	失败请求不启动返回计时	CK-GRANT-FAIL-STATE, CK-GRANT-NOSTRAY-TOKEN, CK-RETRY-ONCE-TOKEN, CK-MEM-EXACT4, CK-MEM-CALENDAR-BUBBLES, CK-MEM-NO-EARLY	CK-CV-RETRY-TO-R, CK-CV-CALENDAR-BUBBLE

故障	名称	安全 CK	cover CK
FI027	返回标记每拍前进且保留空泡	CK-MEM-EXACT4, CK-MEM-CALENDAR-BUBBLES, CK-MEM-UNIQUE	CK-CV-PIPE-ALL, CK-CV-CALENDAR-BUBBLE
FI028	MEM 单拍返回必须立即捕获	CK-LIFE-DONE, CK-MEM-DONE-SYNC, CK-DATA-CAPTURE128, CK-DATA-PCERR-CAPTURE, CK-DATA-CAPTURE-NOREADY, CK-DATA-HOLD128, CK-DATA-ENDTOEND128, CK-RESP-ERROR-DATA, CK-RETIRE-CLEAR-DATA	CK-CV-UPPER-DATA, CK-CV-CAPTURE-NOREADY
FI029	pc_err 与零数据不能充当有效信号	CK-LIFE-DONE, CK-DATA-CAPTURE128, CK-DATA-VALID-NOTVALUE	CK-CV-ZERO-DATA
FI030	返回标记绑定原命令	CK-MEM-IDENTITY, CK-DATA-HOLD128, CK-DATA-R-ROUTE, CK-DATA-ENDTOEND128, CK-DATA-ENDTOEND-ERR, CK-RETIRE-IDENTITY	CK-CV-OUT-OF-ORDER, CK-CV-REORDER-TO-ORDER
FI031	MEM 捕获当拍禁止该命令 R 旁路	CK-MEM-NO-R-BYPASS, CK-MEM-NEXT-R-ELIGIBLE, CK-MEM-CONCURRENT-OLD-R	CK-CV-PIPE-ALL, CK-CV-MEM-NEXT-R, CK-CV-MEM-OTHER-R

故障	名称	安全 CK	cover CK
FI032	同 XAU 按接收年龄保序	CK-AR-ID-SAVE, CK-WIN-AGE, CK-DATA-ENDTOEND128, CK-R-HEAD, CK-R-HEAD-DONE, CK-R-ID-INDEPENDENT, CK-R-ID	CK-CV-DUPLICATE-ID, CK-CV-NONMONOTONIC-ID, CK-CV-CROSS-SRC-ID, CK-CV-YOUNGER-DONE, CK-CV-REORDER-TO-ORDER
FI033	不同 XAU 不施加全局返回顺序	CK-R-CROSS-SRC, CK-R-RR-WINNER, CK-R-EXACT-ELIGIBILITY	CK-CV-CROSS-SRC-ID, CK-CV-CROSS-SRC-PROGRESS
FI034	RR 只选 ready 的完成队头	CK-R-HEAD, CK-R-HEAD-DONE, CK-R-ONEHOT, CK-R-READY, CK-R-RR-WINNER, CK-R-RR-HOLD, CK-R-EXACT-ELIGIBILITY	CK-CV-YOUNGER-DONE, CK-CV-RR-MASKS
FI035	RR 推进与空闲保持	CK-R-RR-WINNER, CK-R-RR-ADVANCE, CK-R-RR-HOLD	CK-CV-RR-ROTATE, CK-CV-RR-MASKS
FI036	地址越界命令仍访问 MEM	CK-AR-RESP-SAVE, CK-RESP-ADDRERR, CK-RESP-OOB-ISSUE	CK-CV-ERR10, CK-CV-ERR11
FI037	越界异常优先于奇偶异常	CK-AR-RESP-SAVE, CK-DATA-ENDTOEND-ERR, CK-RESP-ADDRERR, CK-RESP-PARITY, CK-RESP-NORMAL, CK-RESP-ERROR-DATA	CK-CV-ERR00, CK-CV-ERR01, CK-CV-ERR10, CK-CV-ERR11

故障	名称	安全 CK	cover CK
FI038	pc_err 与所属 OTB 命令一起保存	CK-DATA-PCERR-CAPTURE, CK-DATA-PCERR-HOLD, CK-DATA-ENDTOEND-ERR, CK-RESP-PARITY, CK-RETIRE-CLEAR-DATA, CK-REUSE-OLD-R, CK-REUSE-NEW-OTB	CK-CV-ERR01, CK-CV-ERR11
FI039	10B 满表同拍退休与接收	CK-CAP-READY-FORMULA, CK-CAP-711-UNIQUE, CK-CAP-711-SHARED, CK-REUSE-FULL-ACCEPT	CK-CV-UNIQUE-REFILL, CK-CV-SHARED-REFILL, CK-CV-FULL-SLOT-REUSE
FI040	同槽复用保留旧响应并初始化新命令	CK-LIFE-ALLOC-FREE, CK-LIFE-NEW-STATE, CK-CAP-NET-UPDATE, CK-WIN-AGE, CK-DATA-ENDTOEND128, CK-REUSE-OLD-R, CK-REUSE-NEW-WINS, CK-REUSE-NEW-OTB, CK-REUSE-YOUNGEST	CK-CV-SAME-SRC-NETZERO, CK-CV-FULL-SLOT-REUSE
FI041	R 退休同时清除两侧归属	CK-LIFE-COUNT, CK-LIFE-HOLD, CK-LIFE-NODUP-R, CK-RETIRE-EXACT, CK-RETIRE-IDENTITY, CK-RETIRE-CLEAR-STATE, CK-RETIRE-CLEAR-DATA, CK-REUSE-NEW-WINS	CK-CV-FULL-SLOT-REUSE
FI042	复用新命令仍遵守三拍路径	CK-PIPE-ACCEPT-NOBYPASS, CK-REUSE-NEW-PIPELINE	CK-CV-MIN3
FI043	两个 XAU 联合占用仍为第三路保留名额	CK-CAP-RESERVE, CK-CAP-EMPTY-CAN-ACCEPT, CK-CAP-READY-FORMULA, CK-CAP-440	CK-CV-CAP440, CK-CV-CAP711

故障	名称	安全 CK	cover CK
FI044	占用额度统计到真正 R 退休	CK-PENDING-EXACT, CK-CAP-OCC	由相关序列 cover 及 guard 见证补查
FI045	逐来源 ready 判定	CK-AR-ACCEPT, CK-AR-NOACCEPT, CK-CAP-READY-FORMULA, CK-CAP-440, CK-CAP-NONBLOCK-COMB	CK-CV-UNREADY-SRC, CK-CV-CAP440
FI046	空闲来源的保底项不可借出	CK-CAP-RESERVE, CK-CAP-MAX7, CK-CAP-EMPTY-CAN-ACCEPT, CK-CAP-READY-FORMULA, CK-CAP-700	CK-CV-CAP700
FI047	满表退休最后一项只恢复该来源保底	CK-CAP-READY-FORMULA, CK-CAP-711-UNIQUE, CK-REUSE-FULL-ACCEPT	CK-CV-UNIQUE-REFILL
FI048	同拍退休接收的每来源计数合成	CK-CAP-OCC, CK-CAP-711-SHARED, CK-CAP-NET-UPDATE, CK-CAP-NONBLOCK-COMB, CK-REUSE-FULL-ACCEPT	CK-CV-PIPE-ALL, CK-CV-SHARED-REFILL, CK-CV-SAME-SRC-NETZERO
FI049	容量保护不改变五项发射窗口	CK-WIN-MEMBERS, CK-WIN-SIXTH-BLOCK	CK-CV-QUOTA-NO-ESCAPE

合同与未证明范围

- **CT-RESET**（R17；U01）：采用低有效异步复位实现；本轮 formal 仅首采样复位后永久释放。 边界：运行中带未完成 MEM 返回时复位冲刷/跨时钟域/复位去使能硬件时序未定义，不能声称已证明。

- **CT-ILLEGAL**（R01,R15,R18；U02）：有 valid 时 src=3、arresp=01/10 在输入域外；无 valid 元数据可任意。 边界：非法编码行为没有强行定义成 DUT bug，合法域证据不覆盖非法协议输入。
- **CT-ID**（R02,R14；U03）：现有实现及本轮验证域允许同源/跨源重复 ID、非单调 ID；不添加 ID 唯一性 assume。 边界：原故障目录 U03 曾未定，此为采用的实现兼容行为，不改写用户原始确认历史。
- **CT-CLEAR**（R16；U04）：采用退休/新分配时 OTB 数据和 pc_err 物理清零，并单列安全义务。 边界：原用户要求清除可能仅指逻辑失效，物理清零是采用实现选择。
- **CT-ERROR-DATA**（R12,R15；U05）：采用错误响应也透传已捕获数据；地址错误优先级高于 pc_err 为已确认要求。 边界：错误响应的数据内容规范不冒充原始用户明确要求。
- **CT-RR**（R14；U06）：采用初始指针 0、0→1→2、实际 R 服务后推进、空闲保持。 边界：不能仅从用户说 RR 就声称其已指定所有方向和初值细节。
- **CT-STATUS**（R07,R08,R09；U07）：采用判断拍采样选择，req 拍 status 变化不撤销；req 拍对应双 grant 决定成功。 边界：不假设 status 稳定至请求周期，不用 assume 隐藏该边界。
- **CT-IDLE**（R02,R17；U08）：采用无效 req/R 负载清零，功能有效性仍只看事件及有效位。 边界：无效负载零值不证明交易正确，也不得从零数据推导未返回。
- **CT-FAIRNESS**（R04,R06,R14,R18；U08,U09）：上游允许切 src，但未定义公平性；grant、status、rready 均无最终服务假设。 边界：仅保证资源准入保底和精确有限步行为，不声称每请求必最终被输入/发射/返回；前 5 项永忙可合法永久阻塞后续第 6 项。
- **CT-FIXED-CONFIG**（R01,R03,R17；implementation_scope）：固定 9 项、3 源、单时钟、128 位 MEM 返回、固定 4 拍；无 burst/rlast/MEMvalid。 边界：不证明 MEM 阵列内容正确性、数据与地址的内存语义、CDC、STA、参数化所有配置或复位模拟电气时序。
- **CT-FORMAL-STATUS**（R18；verification_method）：safety 需要 prove；cover 是可达见证；Assume 是域声明；compile、BMC 未发现反例、timeout 不能升级为 prove PASS。 边界：49 故障类的映射不等于 49 变异已经执行或全部被检出。

旧证据使用规则

旧 native 的 wiring/local_transition/calendar_delay/data_transition 曾有局部 prove 结果；order、capacity、structure 存在未决运行。各旧属性的精确源码位置、hash、run_id、compiled_property_id、原始 status 和证据路径在 JSON evidence_registry 中保留。它们是可复核的候选或组合依赖，不能自动作为本轮对应 CK PASS。

全 128 位端到端数据需要接受身份、独立发射顺序、+4 返回日历、保存及 R 选择一起闭合。仅证明 OTB 每次保存输入，不足以证明对外数据归属；仅用 DUT rsp_pc_err 计算期望，也必须另有独立错误采样/归属证明。

原始 cover 若只记录一次占用、一次重新选择、见过乱序后任意退休，本轮增强为规定身份和前后序列；这些点标记 needs_strengthening。65 个业务 cover 和工具自动 guard witness 分别统计，cover 命中不替代安全证明。

本目录结构审查

已检查：177 个 CK ID 唯一；全部 18 需求有映射；49/49 故障类至少有安全 CK；每个 FG 包含多个 FC；所有 proof_status 均 pending；没有将合同边界当作成功属性。未运行求解，未改冻结证据、RTL 或工具。

CMDBUF RD 仿真基线与历史记录

本分册完整保留早期仿真阶段的说明和设计基线，用于追溯。其 SystemVerilog 文件名、尚未完成的验证项、旧路径或阶段结论均属于当时版本；当前 Verilog 实现和正式验证结果见分册 02 与 06。

内容来源　simulation/cmdbuf_rd/README.md

cmdbuf_rd RTL 与验证工程

当前芯片 RTL 已转换为 Verilog-2001，`rtl/*.v` 为集成文件；新的等价性与完整重新验证记录见 Verilog 迁移说明。以下早期回归与注错报告保留原执行时间。

开发与实际编译、仿真、注错验证主机：192.168.31.116，工程：`/root/cmdbuf_rd`。 本工程依据已对齐的 cmdbuf_rd 设计基线实现。以下内容描述早期功能验证；完整形式证明及本次语言转换的当前结果见仓库验证报告。综合面积和静态时序仍未签核。

实现结构

- `rtl/cmdbuf_rd.v`：顶层接口、CMDBUF/OTB 连接。
- `rtl/cmdbuf_rd_cmd.v`：9 项元数据与状态、年龄矩阵、最老 5 条待发窗口、三周期接收/判断/发射、5A/6B、容量准入、每 XAU 保序及响应 RR、10B 同拍复用。
- `rtl/cmdbuf_rd_otb.v`：9 项 128-bit 数据与 pc_err、成功发射四拍返回标记、无返回背压的数据捕获、分配/退休数据清除。
- `/root/env_base/adapters/cmdbuf_rd/`：独立 C++ 参考模型和 Verilator 驱动。
- `/root/env_base/cases/cmdbuf_rd/`：定向及随机 case；公共 runner 未加入 DUT 特例。
- `scripts/run_mutations.py`：隔离 RTL 注错副本、编译与检测结果分类。
- `docs/`：设计来源、实现选择、用例及注错说明。
- `results/`：检查结果、逐用例 JSON/日志、需要保留的波形。

开发基线

AR 和 R 均共享负载，每周期最多接收、发射、捕获、返回各一条。arid 是透传标签，顺序由内部接收年龄决定。 `rd_mem_status[1:0][7:0]` 的第一个索引为奇偶 half，第二个索引为 bank；0 表示空闲。Arbit 地址为 12-bit `araddr[17:6]`。 CMDBUF 与 OTB 的 9 项一一对应，从合法 AR 握手起保留到实际 R 返回。

保留容量采用“每 XAU 1 项逻辑保底 + 6 项共享”，物理 entry 仍统一分配。占用包含所有已接收未返回命令；`sum(max(occ[x],1)) <= 9`。这是满足用户禁止容量独占要求的实现选择。上游仲裁公平性未确定，因此只保证容量不会被其他来源侵占，不能保证上游一定调度某个来源。

选择先构造成功前馈后的最老 5 条未成功发射命令，再按 Bank 空闲过滤并选最老；当前请求级命令不可重复选中。5A 失败不冲刷下一条预选；6B 成功当拍推进窗口；10B 返回当拍允许新接收复用。 成功取同一 req bank 上 `ar_rd_gnt` 和 `rw_rd_gnt` 同为 1。成功 C3 → MEM 捕获 C7 → 最早 R 返回 C8。 地址越界仍走 MEM，R 状态优先级为地址越界 11、奇偶异常 01、正常 00。R 仅在 ready 的已完成来源队头之间 RR。

已采用但需要系统集成确认的实现选择

1. 异步低有效 `rst_n`；所有状态和数据复位清零。运行中复位的 MEM 协同协议不包含在本次接口定义中，集成时仍需确认。
2. RR 顺序 0→1→2，复位起点 0，仅实际响应后更新为获胜来源的下一来源，空闲保持。
3. 错误响应的 rdata 仍透传已捕获的 MEM 数据，rresp 指示错误。
4. 选择进入请求级后，下一周期仍发送 req；Bank 状态变化由当拍双 grant 最终决定成功。
5. 无有效请求/响应时，输出负载置零。OTB 分配、退休和复位均物理清零数据。
6. src=3 不接受；已接受请求的 arresp 只允许 00/11。其他编码属于未定义输入，内部断言会报错。
7. 所有接口均同一时钟域；没有 CDC、ECC、超时或可变返回延迟处理。MEM 必须严格成功后四拍给出有效数据。

复现

在虚拟机中执行：

```
cd /root/cmdbuf_rd
make lint validate
make test
python3 scripts/run_mutations.py --help
```

`make test` 调用既有公共 runner；使用 `/opt/rh/devtoolset-11/enable` 中的 GCC 11，Verilator 5.024，C++17。不要使用系统默认 GCC 4.8 编译 harness。 常规结果在 `results/baseline/regression.json`；单个 case 可通过公共 Makefile 的 `run-case CASE=<绝对路径>` 运行。补充检查和注错的实际命令、统计以 `docs/verification_baseline.md`、`docs/mutation_plan.md` 及最终验证报告为准。

验证边界

独立参考模型按接收序号维护队列、返回日历和每来源队头，逐拍核对 req、地址、ready、pending 与完整 R 负载。RTL 内部断言额外检查容量、年龄和生命周期一致性。 注错检测区分参考模型、RTL 断言、构建失败、运行环境错误及漏检。既有 49 组注错清单是规格覆盖来源，不表示已经实际运行 49 种 RTL 变异。 测试通过不代表所有输入序列的形式证明，也不代表目标频率已经收敛。

本目录不保存 SSH 密码；本机源文件、结果及随交付包提供的 SHA256 用于追溯实际验证版本。

内容来源　simulation/cmdbuf_rd/docs/verification_baseline.md

cmdbuf_rd 仿真验证基线报告

日期：2026-10-10。执行目录为 Linux VM 的 `/root/cmdbuf_rd`；本报告由实际 case JSON 汇总。RTL 的 Bank half 索引等价改写后已重新构建并重跑。

结果

Verilator 5.024、C++17、`--Wall --assert --trace`：基础 19/19 PASS，补充 4/4 PASS；各 case 的必达覆盖计数均满足要求。未关闭 lint 警告或 RTL 断言。

集合	case	功能周期	比较/检查	接收	成功发射	MEM 捕获	R 退休
baseline	19	6433	101329	3126	3126	3126	3126
extended	4	300	3658	52	52	52	52
combined	23	6733	104987	3178	3178	3178	3178

功能周期包含正常刺激、排空和末尾空闲检查，不包含复位拉低期间的时钟。每次检查是参考模型比较或明确断言意图的一次判定，不等价于 formal 属性覆盖数。

可复现命令

基础回归：

```
make -f /root/env_base/env/Makefile PROJECT_CONFIG=/root/env_base/projects/cmdbuf_rd.mk run-all
```

补充回归使用独立 adapter、build 和结果目录，不改变基础 harness：

```
make -f /root/env_base/env/Makefile \
  PROJECT_CONFIG=/root/env_base/projects/cmdbuf_rd.mk \
  HARNESS_FILES=/root/env_base/adapters/cmdbuf_rd/cmdbuf_rd_harness_extended.cpp \
  CASE_DIR=/root/env_base/cases/cmdbuf_rd/extended \
  BUILD_DIR=/root/cmdbuf_rd/build/verilator_extended \
  RESULT_DIR=/root/cmdbuf_rd/results/extended run-all
```

用例明细

Case	结果	周期	检查	接收/退休
01_pipeline	PASS	51	793	24/24
02_same_xau_ooo	PASS	43	514	6/6
03_forward_6b	PASS	43	513	6/6
04_grants	PASS	103	1333	24/24
05_failure_5a	PASS	43	537	8/8
06_window_block	PASS	48	563	6/6
07_single_pulse	PASS	48	575	7/7
08_rr	PASS	53	648	9/9
09_errors	PASS	38	443	4/4
10_capacity_reuse	PASS	53	669	11/11
11_capacity_700	PASS	53	648	9/9
12_capacity_440	PASS	53	647	9/9
13_tokens	PASS	63	737	8/8

Case	结果	周期	检查	接收/退休
14_single_source_throughput	PASS	170	3478	160/160
15_reset_clean	PASS	96	1256	24/24
16_random_seed_1	PASS	1814	29188	935/935
17_random_seed_17	PASS	1812	29234	942/942
18_random_seed_91	PASS	1816	29172	931/931
19_rr_ready_masks	PASS	33	381	3/3
20_all_bank_halves	PASS	163	1839	16/16
21_repeated_ids	PASS	58	697	9/9
22_status_after_selection	PASS	28	331	3/3
23_nonmonotonic_ids	PASS	51	791	24/24

独立参考模型与检查范围

- 参考模型按实际 AR 接收顺序维护命令列表，不照搬 RTL 年龄矩阵或物理 entry 分配。
- 每周期独立计算原队头、RR、当拍退休后的逐源 ready、成功前馈后的最老五项窗口、下一拍请求。
- 每次成功发射建立 +4 拍返回日历；MEM 数据仅在对应周期有效，无效拍驱动不同噪声数据。R 候选只读取此前已捕获的完成状态。
- 比较 req、完整 12-bit 下游地址、pending_cnt、三位 aready/rvalid 及共享 R 的全部 128-bit 数据与元数据。
- 失败用例打印 CHECK_FAILED、返回非零并保留 VCD；配置错误为 CONFIG_ERROR/退出 2，不能作为成功检出 RTL 注错。

定向覆盖审查

- 复位：所有用例检查初始低有效异步复位输出；15_reset_clean 额外排空后重新复位并再发送一批请求，两个批次共 24 条。未将运行中仍有未完成 MEM 返回的复位冲刷行为擅自定义为已支持。
- 所有 16 个 Bank half：20_all_bank_halves 每次只放开目标 half，其余 15 个 half 均忙，8 Bank × 2 half 全部成功发射并比对返回。
- ID：21_repeated_ids 使用同一 arid=0x5a，同时存在同源重复 ID，通过不同地址、port_id 和数据确认归属；23_nonmonotonic_ids 显式命中非单调 ID。
- 选择后 status 翻转：22_status_after_selection 中两条请求在判断拍空闲、请求拍变忙，但双 grant 有效；仍正确发射，不二次撤销。
- 单源吞吐：14_single_source_throughput 连续接收 160 条，159 次相邻拍连续接收；每条均满足最短三拍前向路径，153 次七项占用下退休/接收并行和四事件并发。
- 容量：明确覆盖 (7,0,0) ready=110、(4,4,0) ready=100、满表来源唯一项退休 ready=010、共享项退休 ready=111，以及两次满表同拍复用。
- 错误：arresp=00/11 与 pc_err=0/1 四组合均返回正确优先级；零数据且 pc_err=0 的单拍返回单独检查。

以下为基础回归累计事件计数；每个定向 case 同时有独立最小门槛，不能以其他 case 的覆盖替代：

覆盖事件	次数
three_cycle_path	449
failed_keep_next	2117
retry_success	1280
sixth_success_forward	2
outside_window_blocked	14
ar_only_grant	918
rw_only_grant	703
wrong_bank_grant	408
capture_without_ready	404
zero_data_no_error	149
capture_after_bubble	1355
same_xau_ooo_issue	446
same_xau_head_block	1062
rr_all_three	45
rr_skip	1892
capacity700_ready110	22
capacity440_ready100	18
rejected_unready_src	150
unique_retire_ready010	2

覆盖事件	次数
shared_retire_ready111	6
full_reuse	1442
single_source_turnover	181
four_events	745
error_combo_00	2476
error_combo_01	370
error_combo_10	244
error_combo_11	36
drained_reset	1

补充回归的明确覆盖：

覆盖事件	次数
duplicate_live_id	6
issued_half_0_bank_0	7
issued_half_0_bank_1	1
issued_half_0_bank_2	6
issued_half_0_bank_3	1
issued_half_0_bank_4	5
issued_half_0_bank_5	1
issued_half_0_bank_6	5

覆盖事件	次数
issued_half_0_bank_7	1
issued_half_1_bank_0	1
issued_half_1_bank_1	6
issued_half_1_bank_2	1
issued_half_1_bank_3	5
issued_half_1_bank_4	1
issued_half_1_bank_5	5
issued_half_1_bank_6	1
issued_half_1_bank_7	5
nonmonotonic_live_id	21
request_busy_but_granted	2

注：上表 half 总数为四个补充 case 合计；20_all_bank_halves 自身对每个 half 都设置且通过至少一次的门槛。

产物

- `/root/env_base/adapters/cmdbuf_rd/cmdbuf_rd_harness.cpp`：基础参考模型。
- `/root/env_base/adapters/cmdbuf_rd/cmdbuf_rd_harness_extended.cpp`：补充定向刺激与覆盖版本，原模型算法保持一致。
- `/root/env_base/cases/cmdbuf_rd/*.case`：基础 19 用例。
- `/root/env_base/cases/cmdbuf_rd/extended/*.case`：补充 4 用例。
- `/root/cmdbuf_rd/results/baseline/` 与 `/root/cmdbuf_rd/results/extended/`：逐 case JSON、日志、runner summary。

- `/root/cmdbuf_rd/results/baseline/05_failure_5a.vcd、10_capacity_reuse.vcd、19_rr_ready_masks.vcd`：指定保留波形。
- `/root/cmdbuf_rd/results/extended/22_status_after_selection.vcd`：选择后 Bank 状态翻转波形。
- `/root/cmdbuf_rd/docs/verification_summary.json`：本报告同源的结构化汇总。

这是固定参数配置的动态仿真基线，不是全输入空间形式证明、CDC 证明、综合面积或 STA 频率结论。随机种子固定为 1、17、91；每次请求的最终接收公平性仍依赖上游调度协议。注错运行结果由独立 mutation 报告记录，不混入本报告 PASS 统计。

最终输入约束与源码版本

最后一次回归已采用修正后的输入门控：每拍先令 arvalid=0，驱动其他输入并求值；按实际 DUT aready 是否全零决定是否送出 valid，再求值。若异常实现因 valid 改变再次使 ready 全零，也会在采样前撤销 valid。参考模型的 expected_ready 仍独立计算并比较，绝不使用实际 ready 替代正确性判据。因此即使注错把实际 ready 错误压低，输入仍满足 ready=000 时 valid=0 的约束，且 ready 差异会被检出。

修正后重新构建、运行基础 19 例和补充 4 例，均 PASS，上述总数与必达覆盖不变。

主线程另外执行 Icarus 的 -g2012 -DSYNTHESIS 编译/展开检查并通过；它输出 always_* sensitivity 保守扩展提示。该检查不包含 Icarus 行为仿真，也不是综合面积、综合时序或功能正确性的证明。

Harness	SHA256
cmdbuf_rd_harness.cpp	639bc3d0ebd6dee75e7b3c0060aee39fc50efb75029b8d0dfd7e4528bcbd2e65
cmdbuf_rd_harness_extended.cpp	9d38710a535afc536162e92fe9d6c1a460ff1a102385ea47ed84e5be397d3149

每例覆盖数据索引

下列 JSON 各自包含 checks、coverage、status 和 failure，覆盖门槛定义保存在对应 .case。

Case	完整检查和覆盖计数
01_pipeline	JSON
02_same_xau_ooo	JSON
03_forward_6b	JSON
04_grants	JSON
05_failure_5a	JSON
06_window_block	JSON
07_single_pulse	JSON
08_rr	JSON
09_errors	JSON
10_capacity_reuse	JSON
11_capacity_700	JSON
12_capacity_440	JSON
13_tokens	JSON
14_single_source_throughput	JSON
15_reset_clean	JSON
16_random_seed_1	JSON
17_random_seed_17	JSON
18_random_seed_91	JSON

Case	完整检查和覆盖计数
19_rr_ready_masks	JSON
20_all_bank_halves	JSON
21_repeated_ids	JSON
22_status_after_selection	JSON
23_nonmonotonic_ids	JSON

源码与 case 的完整校验值保存在 verification_sources.json。

cmdbuf_rd 测试敏感性检查与结果

本检查把单个错误植入真实 RTL 副本，再运行与无注错基线完全相同的独立 scoreboard。它是所选关键行为的测试敏感性检查，不代表历史注错目录全部用例已实施。

隔离与可复现

- 基线源码位于 `/root/cmdbuf_rd/rtl`，运行器只读。每次注错前后核对全部 SystemVerilog 文件的 SHA-256。
- 每个变异复制至 `/root/cmdbuf_rd/mutations/Mxx/rtl`，只替换一个且仅出现一次的精确片段。片段缺失、重复或未改变文本均拒绝运行。
- `RTL_ROOT` 指向该变异的项目根目录；`BUILD_DIR`、`RESULT_DIR` 使用变异 ID 和本次运行编号隔离。
- 每批冻结 harness、所选 case 和项目配置，构建显式传入快照的 `HARNESS_FILES`、`CASE_DIR`、`PROJECT_CONFIG`。逐项校验快照及通用环境文件哈希，避免构建期间跨版本。
- 先独立构建无注错副本并跑完选定用例；基线失败立即停止，不计算变异结果。
- 所有命令均通过 `/root/env_base/env/Makefile` 及 `projects/cmdbuf_rd.mk`，复用相同 case 文件和检查器。
- 编译与仿真分别保存日志；基线及每个变异保存结果 JSON，完整汇总位于 `mutations/campaigns/<run-id>/summary.json`。

结果口径

结果	含义
`killed`	构建成功；scoreboard 明确报告失败，或日志明确出现 RTL `Assertion failed in`，运行返回失败且其余用例没有基础设施错误。
`survived`	构建成功；全部所选用例明确通过，当前测试未发现该变异。
`compile_error`	编译失败或编译超时，不计为检测成功。

结果	含义
`runtime_error`	仿真超时、崩溃、缺少结果文件、配置错误或不一致的退出状态，不计为检测成功。
`mutation_error`	精确替换条件不满足或无法创建隔离副本，未有效运行变异。

只有 `killed` 才表示验证确实发现设计错误。`detection_kind=scoreboard` 与 `rtl_assertion` 分开记录；RTL 断言直接终止时可能来不及生成 harness JSON，此时保留具体断言日志作为证据，普通崩溃不算检测成功。检查器失败报告及相应波形保留在变异结果目录。固定接口/容量约束下的长期合法等待不能仅因等待而算作错误。

计划覆盖

已根据真实 RTL 绑定以下 12 个唯一片段，均保留精确变更文本和前后 SHA-256：

ID	故意植入的错误
M01	五项窗口扩大为九项，允许越窗。
M02	双 grant 的 AND 改为 OR。
M03	关闭 6B 成功当拍窗口前馈。
M04	OTB 按 `req` 而非成功握手产生返回 token。
M05	捕获许可额外使用提前一拍的 tap，造成早采样或重复采样。
M06	共享额度从六项放宽到九项，破坏每来源保底。
M07	R 候选忽略当前 `rready`。
M08	仅在已完成项中找来源队头，跳过未完成的老命令。
M09	令奇偶异常覆盖地址越界异常。
M10	当前请求失败时丢弃同拍预选结果，违反 5A。

ID	故意植入的错误
M11	准入不扣当拍退休占用，关闭 10B。
M12	选择时反转奇偶 Bank 状态索引。

它们覆盖独立敏感点，不等价于目录全部 49 组注错。M05 是提前 tap 污染，不是完整 MEM→R 同拍旁路实现的变异。

使用

```
python3 /root/cmdbuf_rd/scripts/run_mutations.py --list
python3 /root/cmdbuf_rd/scripts/run_mutations.py
python3 /root/cmdbuf_rd/scripts/run_mutations.py --only M01,M03
python3 /root/cmdbuf_rd/scripts/run_mutations.py --only M01 --case 06_window_block.case
```

`--manifest` 可提供 JSON 列表（或含 `mutations` 列表的对象）。每项包含 `id`、`file`、`before`、`after`，可选 `purpose`；文件必须是 `rtl/` 下的相对路径，ID 为 `M01` 形式。仅有构建失败与未发现错误等非理想结果时也原样留存，不隐藏失败实验。

最终结果 campaign_03

最终完整批次为 `campaign_03`，无注错快照先通过 **19/19** 用例；随后 **12/12 变异被检测**，`survived=0`、`compile_error=0`、`runtime_error=0`、`mutation_error=0`。每个变异均运行同一组 19 个用例。全部 12 项都有独立 scoreboard 的失败证据，M04、M05 还在其他用例中触发 RTL 断言。

完整证据见 最终汇总 JSON。各变异的精确替换、原文与变更后 SHA-256、编译日志、用例 JSON、失败波形及结果分类均在 `mutations/Mxx/results/campaign_03/` 及汇总中保留。

下表列出按用例执行顺序出现的首个 **scoreboard** 失败。`cycle` 是 harness 从 0 开始的测试周期编号，不直接等同于设计图中的 C1 编号；数值均为十六进制。

变异	首个 scoreboard 报错用例	cycle	首个不一致：实际 / 预期	检测来源
M01	03_forward_6b.case	7	req=0x4 / 0x0	scoreboard
M02	04_grants.case	3	pending_cnt=0x2 / 0x3	scoreboard
M03	03_forward_6b.case	10	req=0x0 / 0x4	scoreboard
M04	05_failure_5a.case	7	rvalid=0x1 / 0x0	scoreboard、RTL assertion
M05	01_pipeline.case	6	rvalid=0x1 / 0x0	scoreboard、RTL assertion
M06	07_single_pulse.case	7	aready=0x7 / 0x6	scoreboard
M07	07_single_pulse.case	7	rvalid=0x1 / 0x0	scoreboard
M08	02_same_xau_ooo.case	8	rvalid=0x1 / 0x0	scoreboard
M09	09_errors.case	10	rresp=0x1 / 0x3	scoreboard
M10	04_grants.case	3	req=0x0 / 0x2	scoreboard

变异	首个 scoreboard 报错用例	cycle	首个不一致：实际 / 预期	检测来源
M11	`01_pipeline.case`	7	`aready=0x6 / 0x7`	scoreboard
M12	`16_random_seed_1.case`	5	`req=0x4 / 0x0`	scoreboard

M04 按完整执行顺序的首个失败实际来自更早执行的 `04_grants.case`，RTL 报告 `capture ownership mismatch`。该断言中止没有 C++ 周期 JSON，不推测其周期；原始日志保留在 M04 断言日志。上表额外提供首个 scoreboard 用例的明确周期与实际/预期值。M04 有 3 个 scoreboard 检出用例、3 个断言检出用例；M05 分别为 11 个、8 个。

固定版本与工具

本批使用 VM 现有工具：`/usr/local/bin/verilator`，Verilator **5.024**；通过 `/opt/rh/devtoolset-11/enable` 选择 GCC **11.2.1**；运行器 Python **3.6.8**。构建通过现有 `/root/env_base/env/Makefile`，启用 `--Wall --assert`，未关闭警告以让变异构建通过。

最终源码 SHA-256：

文件	SHA-256
`rtl/cmdbuf_rd.sv`	`06e57c016cc74fff564c4d7c2e6e5fc8c9df317f9b172a6f3c7a6df7d74db7e8`
`rtl/cmdbuf_rd_cmd.sv`	`4de7a4022db5d69f3090691d59b3fe42ca673e77020a478ac7584d0378496339`
`rtl/cmdbuf_rd_otb.sv`	`bbca166e8facd72ec22aeb7f1b666df1da9e08fb2efe76f94fa49a25c0ed0068`
冻结的 `cmdbuf_rd_harness.cpp`	`639bc3d0ebd6dee75e7b3c0060aee39fc50efb75029b8d0dfd7e4528bcbd2e65`
冻结的 `cmdbuf_rd.mk`	`89224072b813eaef06d5e81a0b090f2336c54125cf64c5ea777d0f0e830cc4c5`

快照位于 `mutations/campaigns/campaign_03/verification/`。十九个 case 的逐文件哈希以及通用 Makefile、run_cases.py 的哈希均写入最终汇总；结束时 `baseline_source_unchanged=true`、`verification_inputs_unchanged=true`。

最终驱动先令 `arvalid=0` 稳定其他当拍输入，再依据 DUT 实际 `arready` 门控发送意图；采样前保证实际 `arready=000` 时 `arvalid=0`。期望 ready 仍由独立模型计算并比较，因此错误地拉低 ready 会被检查为接收机会损失，不靠违反输入约束检出。

中途批次与检查边界

- `campaign_01` 保留为 `interrupted`：RTL 为兼容 Icarus 做了等价 packed 索引改写，批次停止。该批 M04 原始连接替换造成未用信号编译失败，已单列 `compile_error`；后续修正变异表达式以保留信号使用。M05 当时的日志识别仅认识 `%Error`，未识别 `%Fatal` 断言前缀，保守记为 `runtime_error`；后续修正精确识别，未把普通 abort 归为检测成功。
- `campaign_02` 保留为 `interrupted`：独立审查发现驱动器应按实际 DUT ready 门控 valid，故停止并修正 harness；该批部分结果不作为最终结论。
- 最终结论只采用重新完整运行且源码、验证输入固定的 `campaign_03`。前两批日志和状态保留，未混入最终计数。
- 本次独立静态核对覆盖窗口与当前请求排除、5A/6B、四拍返回标记、MEM 捕获后下一拍 R、来源保序、容量计数、10B 同槽复用及错误优先级；没有提出尚未修复的 RTL 功能问题。上述输入驱动约束问题已由验证适配器修复并重跑。
- 12 项检出证明这组测试对这些具体错误敏感；不等同于全设计证明、全部 49 组历史注错完成或所有边界行为已定义。扩展 harness 的额外 4 个用例未用于本批变异结论。

cmdbuf_rd 开发与验证交付

实际开发与验证主机：`192.168.31.116`。工程：`/root/cmdbuf_rd`。日期：2026-10-10。

三个 SystemVerilog 模块已完成：顶层连接、CMDBUF 命令控制、OTB 数据与固定延迟返回。实现了当前已对齐的 9 项、窗口 5、三周期前向流水、5A/6B/10B、四拍 MEM 返回、下一拍才可返回 R、单 XAU 保序、ready 内 RR、错误优先级与来源容量保留。

实际结果

项目	结果
Verilator 静态检查	PASS，启用 --Wall，无警告
公共验证环境结构检查	PASS
独立模型基础回归	19/19 PASS
补充定向回归	4/4 PASS
合计功能周期 / 检查次数	6,733 / 104,987
完整接收→发射→捕获→返回	3,178 条，四阶段计数一致
Icarus 编译与展开	PASS；有工具本身的 always_* 敏感列表扩展提示，未运行 Icarus 功能仿真
最终 RTL 注错	12/12 检出；漏检 0，构建失败 0，运行错误 0，注入错误 0

注错最终批次为 `campaign_03`，使用固定源码及验证环境快照，RTL 前后 SHA256 一致。旧的两个批次中断记录保留在 VM，它们不纳入上述最终统计。注错同时统计独立参考模型和内部 RTL 断言的检测，具体类型见下表；编译失败和环境异常不计为检出。

本次运行的注错

编号	目的	结果	检测途径
M01	错误扩大最老五条窗口，允许第六条绕过阻塞窗口	killed	scoreboard
M02	将双 grant 同时命中错误放宽为任一 grant 命中	killed	scoreboard
M03	关闭 6B 成功当拍窗口前馈	killed	scoreboard
M04	仅发 req 即启动 OTB 返回 token，失败请求也被计入	killed	rtl_assertion, scoreboard
M05	固定四拍捕获错误多使用提前一拍的 tap，导致早采样或重复捕获	killed	rtl_assertion, scoreboard
M06	放宽共享额度，破坏每来源一项容量保底	killed	scoreboard
M07	响应候选忽略当拍 rready	killed	scoreboard
M08	仅在完成项中找来源队头，跳过同来源更老的未完成命令	killed	scoreboard
M09	错误地让奇偶异常覆盖地址越界异常	killed	scoreboard
M10	违反 5A，在当前请求失败时丢弃同拍预选的下一命令	killed	scoreboard

编号	目的	结果	检测途径
M11	关闭 10B 准入的退休前馈，满表时多停一拍	killed	scoreboard
M12	反转命令选择时的奇偶 Bank 状态索引	killed	scoreboard

本次实际运行的是上述 12 个具体 RTL 变异；此前 49 组注错清单仍作为完整规格覆盖来源，并未将清单条目数当作实际注错运行数量。

覆盖与实现选择

定向验证包括窗口内全阻塞而第六条空闲、失败后保持下一条、成功当拍推进窗口、双 grant、固定四拍与单拍 MEM 数据、无同拍 MEM→R 旁路、同源乱序发射但有序返回、ready 内 RR、满表同拍复用、每来源保底容量、四种错误组合、全部 16 个 Bank half、重复与非单调 ID、选中后状态翻转、初始复位及排空后复位。

参考模型按接收序号及队列建立预期，不依赖 RTL 年龄矩阵。驱动器按实际 aready 门控 arvalid，保证在注错情况下也遵守 aready=000 时 arvalid=0 的输入约束；ready 的正确值仍由独立模型检查。

容量实现采用每 XAU 1 项逻辑保底 + 6 项共享，物理 entry 不分区。尚未由系统冻结的细节采用 README 中明确记录的实现选择：异步低有效复位、RR 初始从 0 开始、错误数据透传、请求级不随状态变化撤销等。

文件和复现

- `rtl/`：3 个 RTL 文件。
- `README.md`：接口行为、实现选择和基础运行命令。
- `docs/verification_baseline.md`：每例结果、覆盖计数、基础与扩展回归命令。
- `docs/verification_summary.json`：仿真结构化汇总。
- `docs/mutation_plan.md` 与 `scripts/run_mutations.py`：注错定义和可复现 runner。
- `results/baseline/`、`results/extended/`：逐例日志、JSON 与选定波形。

- `source_manifest.json`：实际交付源码、环境和 case 的 SHA256。
- 随包的 `env_base/`：只包含本项目 adapter/cases/config 和所需公共 runner，未包含其他项目。

在原虚拟机运行：

```
cd /root/cmdbuf_rd
make lint validate
make test
```

扩展回归和注错重跑命令见上述报告，注错重跑请使用新的 run-id。压缩包以 `/root` 为目录基准保存 `cmdbuf_rd/` 和 `env_base/`，可在独立位置解包检查；原 VM 已有完整可运行工程。

验证边界

本次交付为固定配置 RTL 与动态功能验证，不是完整形式证明；未运行综合、STA 或 CDC 验证，不能据此承诺频率和面积。运行中复位涉及未完成 MEM 事务的协同协议仍需系统定义；上游调度公平性未定，不声明无条件最终接收保证。

内容来源　simulation/cmdbuf_rd/docs/design_spec.md

归档说明：本文是实施前设计方案快照。当前 RTL、已采用的实现选择和验证完成情况，以工程 README.md、verification_baseline.md 及 verification_report.md 为准。原文中的“尚未交付”描述保留用于追溯设计阶段状态。

cmdbuf_rd 详细 RTL 设计方案

版本日期：2026-10-10。设计基线：九项 CMDBUF 与九项 OTB、五条待发命令乱序窗口、5A / 6B / 10B 时序、成功发射后固定四拍返回、MEM 捕获后最早下一拍响应，以及每 XAU 保底的容量保护方案。

本方案用于 RTL 编码、模块接口评审和验证建模。已确认的行为作为实现约束；固定 entry、年龄矩阵、相同物理索引等属于本方案的实现选择。复位、精确 RR 顺序等尚未最终定义的边界在文末列出，不将建议冒充已经确认的接口要求。

时序图集 · 注错清单 · 容量隔离说明

1 设计目标和能力边界

cmdbuf_rd 位于上游共享 AR/R 接口与下游 Arbit、MEM Bank 之间。它将命令接收、Bank 可用性判断、仲裁发射、数据捕获和响应返回解耦，允许窗口内较新但目标 Bank 空闲的命令先发射，同时保证同一 XAU 的响应按接收次序返回。

项目	设计基线
上游来源	最多三个 XAU，src 为 0、1、2
AR 接口	一组共享负载，每拍最多接收一条；并非三条独立 AR 总线
总容量	九条未退休命令；CMDBUF 与 OTB 各九项、一一对应
入口容量保护	每个 XAU 逻辑保留一项，其余六项共享；物理槽位统一分配
下游命令带宽	每拍最多尝试发射一条，最多成功一条

项目	设计基线
乱序范围	最老五条未成功发射命令
最短前向路径	C1 接收、C2 判断、C3 请求并获得同拍仲裁结果
MEM 返回	成功发射与 MEM 接收同拍，四拍后返回一组数据及 pc_err，仅有效一拍
R 返回	每拍最多一条；同 XAU 保序，不同 XAU 在合格通道间 RR
最短完整路径	C1 接收、C3 成功、C7 捕获、C8 返回并释放
释放条件	实际向上游返回 R 响应后，同时释放 CMDBUF 与 OTB

本模块能够缓冲 Bank 冲突，并利用五项窗口内的其他空闲 Bank。它不会增大 MEM 的物理接收带宽，也不保证所有状态下持续有命令可发。最老五条全部被阻塞时，第六条即使 Bank 空闲也继续等待。

容量保留保证零占用 XAU 至少有首条命令的接收名额。用户已确认上游可以切换 src，但公平性未定；模块不能强迫上游选择某个来源，也不能无条件保证其请求最终被调度。

2 顶层接口

以下方向均相对于 cmdbuf_rd。为避免同名，AR/R 的 port_id 在本文分别命名为 ar_port_id 和 r_port_id；最终端口命名可按工程规范调整。

2 1 时钟和复位

信号	方向	位宽	说明
clk	输入	1	统一时钟；当前方案按单时钟域设计

信号	方向	位宽	说明
reset	输入	1	占位名称；极性、同步或异步及运行中复位协议待确认

若上游、Arbit 或 MEM 实际跨时钟域，需在模块边界增加单独的跨域协议，不能直接沿用本文固定四拍关系。

2 2 上游 AR 通道

信号	方向	位宽	含义
arvalid	输入	1	当前共享 AR 负载有效
arready	输出	3	每个 XAU 的接收资格；三位允许不同，也不要求 one-hot
araddr	输入	18	命令地址
src	输入	2	来源 XAU，合法值 0～2
arid	输入	8	事务标签，保存后透传到 rid
ar_port_id	输入	5	保存后透传到 r_port_id
arresp	输入	2	00 正常、11 地址越界；其他值未定义

```
ar_fire = arvalid && (src < 3) && arready[src]
```

实现时先判断 src 合法性，再选择 ready 位，避免对 src=3 做越界索引。上游保证 arready 全零时 arvalid=0；这不等价于“任一 ready 为 1 就可接收当前 src”。

arid 不参与年龄排序，不要求数值随命令先后递增。本实现不依赖 ID 唯一性；响应与 entry 的归属使用内部标识和接收年龄。

2 3 上游 R 通道

信号	方向	位宽	含义
rvalid	输出	3	通过哪一位为 1 指示接收 XAU；至多一位为 1
rready	输入	3	三个 XAU 当前接收响应的能力
rdata	输出	128	一组共享数据
rid	输出	8	原 arid
r_port_id	输出	5	原 ar_port_id
raddr	输出	3	原 araddr[2:0]
rresp	输出	2	00 正常、11 地址越界、01 奇偶校验异常

```
(rvalid & ~rready) == 0
onehot0(rvalid)
retire_fire = |(rvalid & rready)
```

本接口约定只在 ready=1 的通道上产生 valid，因此没有“先拉高 valid，再锁住共享负载等待 ready”的状态。所有 R 字段来自同一获选命令。

2 4 Arbit 接口

信号	方向	位宽	含义
req	输出	8	Bank 请求，one-hot 或全零
arb_addr	输出	12	请求级保存地址的 araddr[17:6]
pending_cnt	输出	4	全部已接收但未成功发射的命令数，范围 0～9
ar_rd_gnt	输入	8	下游仲裁响应

pending_cnt 表示全局待发数，不是窗口内命令数，也不是总占用数。输出使用当前寄存状态，不把当拍 grant 反馈形成的下一状态计数直接送回 Arbit。

2 5 MEM 接口

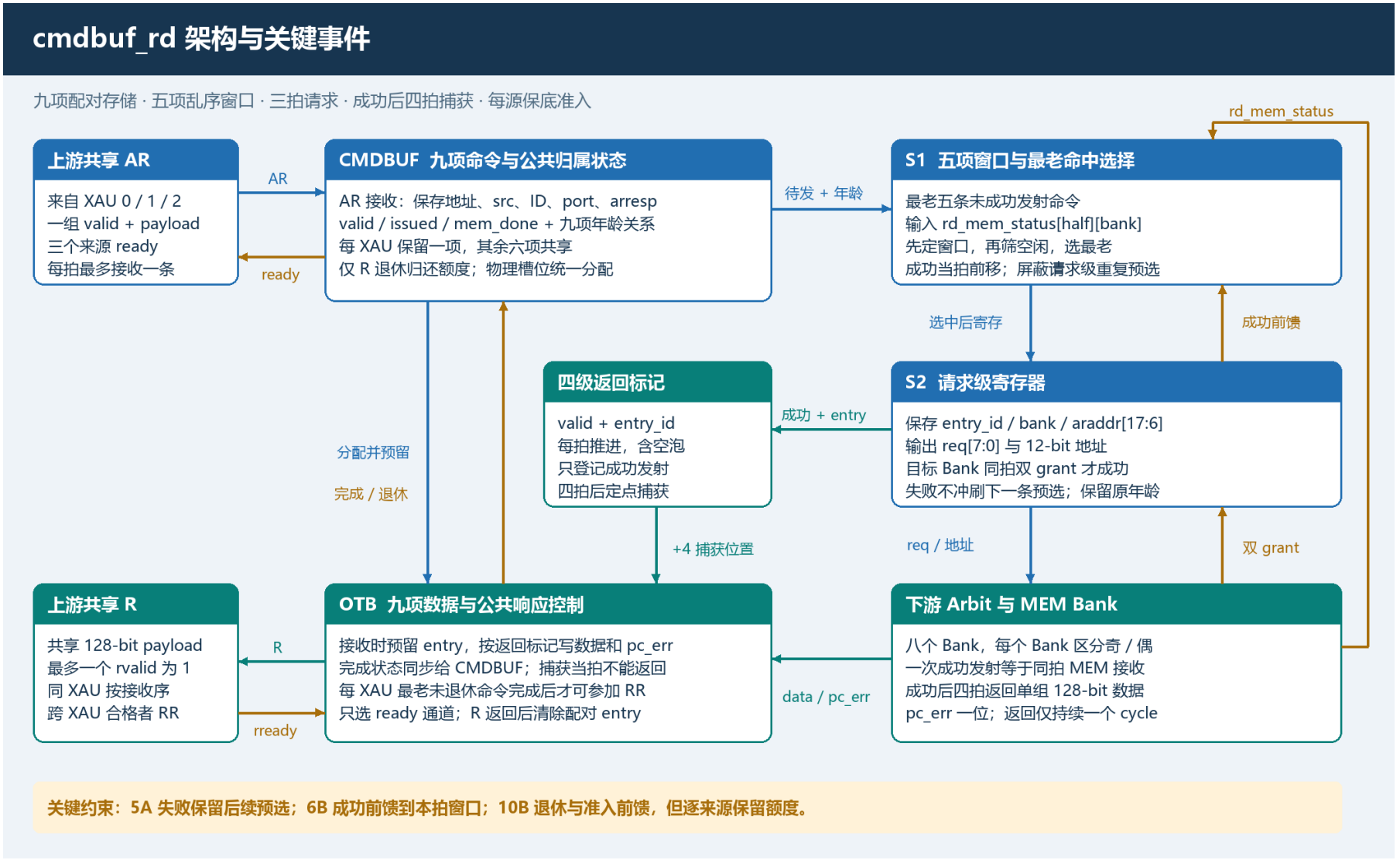
信号	方向	位宽	含义
rd_mem_status	输入	[1:0][7:0]	第一维奇偶，第二维 Bank；0 空闲、1 忙
mem_rdata	输入	128	合并后的单组返回数据
pc_err	输入	1	当前有效返回的奇偶校验异常
rw_rd_gnt	输入	8	目标 Bank 的读仲裁许可

```
bank_id   = saved_araddr[5:3]       // Bank 0～7
bank_half = saved_araddr[6]         // 0 偶、1 奇
bank_idle = !rd_mem_status[bank_half][bank_id]
arb_addr  = issue_addr_q            // 保存的 araddr[17:6]
```

arb_addr[0] 已携带原地址 bit 6，无须再添加 bank_half 外部端口。没有独立 MEM 返回 valid 的已定义端口；本模块依据成功发射事件固定延迟四拍，产生内部捕获有效信号。pc_err 不能充当返回 valid。

当前协议是一条 AR 对应一次 128-bit 返回，不额外引入 burst、rlast、AXI 重排规则或未约定的地址对齐限制。

3 总体架构　cmdbuf_rd 架构



cmdbuf_rd 架构

顶层仍由 CMDBUF 和 OTB 两个主要部分组成。控制职责建议如下：

功能	归属建议	作用
AR 接收与配对分配	CMDBUF 侧公共控制	计算每源 ready，原子分配 CMDBUF/OTB
命令元数据与年龄	CMDBUF	保存字段、维护先后关系
五项窗口和 Bank 判断	CMDBUF	从合格窗口选择最老命令
请求级寄存器	CMDBUF	下一拍输出 req/address，取得同拍 grant
成功发射标记流水	OTB 返回控制	将内部 entry 标识延迟四拍
数据与 pc_err 存储	OTB	无背压捕获 MEM 单拍返回
每源队头、RR、共享 R mux	公共响应控制	按同源接收顺序返回
完成和退休广播	公共控制	MEM 完成通知 CMDBUF；R 返回释放两侧

AR 接收时立即为 OTB 保留位置。请求级发出命令时，同步将命令归属传给 OTB；只有成功发射事件才能启动返回标记流水。MEM 数据回来时无需再次争抢容量。

4 存储组织与内部状态

4 1 固定九项 entry

本方案采用固定 entry 编号 0～8，内部 entry_id 为 4 bit，9～15 为无效编码。CMDBUF 与 OTB 使用相同 entry_id，使分配、返回和释放容易对应。

相同物理编号是实现选择；外部要求是一一对应。若实现为不同物理编号，必须增加可靠映射并保持本文的行为。

命令不会因成功发射而搬移到另一物理槽，槽位只在 R 返回后成为空闲。乱序、新老关系由年龄结构表达，不由槽位编号表达。

4 2 每项字段

字段	位宽	归属	更新时机
valid	1	公共归属状态	AR 接收置位，R 退休清除
issued	1	CMDBUF	分配置零，成功发射置位
mem_done	1	公共完成状态	分配置零，MEM 捕获置位
cmd_addr	18	CMDBUF	AR 接收保存
cmd_src	2	CMDBUF	AR 接收保存
cmd_id	8	CMDBUF	AR 接收保存
cmd_port_id	5	CMDBUF	AR 接收保存
cmd_arresp	2	CMDBUF	AR 接收保存
data	128	OTB	MEM 捕获保存
saved_pc_err	1	OTB	与 data 同拍保存

valid 与 mem_done 可以作为两部分共享的权威状态。CMDBUF 直接使用 mem_done 感知返回，OTB 以相同状态表示数据有效，避免维护两个独立完成位而发生失配。若为了模块边界而镜像状态，两侧必须在同一事件下更新并检查一致性。

本方案建议将 OTB 数据用寄存器阵列实现。九项共 1152 个数据 bit，能够为不同 entry 同拍执行捕获和清除；不能未经端口分析就替换成单写口 RAM。

4 3 派生状态

状态	条件	是否占容量	是否计入待发

正在请求级尝试的命令仍属于 WAIT，直至取得成功 grant；额外的 issue_valid_q/issue_entry_q 仅用于标识流水占用、屏蔽重复预选。失败不需要释放、重新分配或更改年龄。

```
FREE --AR 接收--> WAIT --成功发射--> INFLIGHT --MEM 捕获--> DONE --R 返回--> FREE
                    |
                    +--仲裁失败--> WAIT，年龄不变
```

4 4 全局寄存器

状态	建议宽度	用途
older_set	9 × 9	每项有哪些更老的有效命令
issue_valid_q	1	请求级有效
issue_entry_q	4	请求级 entry
issue_bank_q	3	请求级目标 Bank
issue_addr_q	12	请求级下游地址
ret_valid_q	4	固定四拍返回标记
ret_entry_q	4 × 4	每级返回标记对应 entry
occ_q	3 × 4	各 XAU 已接收、未退休总占用
used_cnt_q	4	总占用，0～9；可由 occ 求和派生
pending_cnt_q	4	总待发数，0～9
rr_ptr_q	2	RR 扫描起点，合法值 0～2

5 AR 准入和容量隔离

5 1 逻辑保留与共享额度

设计采用每 XAU 逻辑保留一项、其余六项共享。只有一个 XAU 活动时最多占七项，另两项继续为其他来源保留，不因其暂时没有请求而借出。

occ[x] 统计该 XAU 全部 valid 命令。成功发射以及 MEM 返回均不会减少它；只有 R 退休才减少。

```
sum(max(occ[x], 1)) <= 9

等价于：sum(max(occ[x] - 1, 0)) <= 6
```

只限制每 XAU 最多七项不够，因为 (4,5,0) 仍会占满九项并挡住第三路。上述不变量同时保护每个零占用来源的首条接收名额。

5 2 考虑当拍退休后的 ready

```
retire[x] = rvalid[x] && rready[x]
occ_after[x] = occ_q[x] - retire[x]
free_after = 9 - sum(occ_after)
shared_after = sum(max(occ_after[x] - 1, 0))

arready[x] = (free_after > 0)
          && ((occ_after[x] == 0) || (shared_after < 6))

accept[x] = ar_fire && (src == x)
occ_next[x] = occ_after[x] + accept[x]
```

减法、比较和加法使用足够宽的无符号中间值；occ=0 时的 max(occ-1,0) 必须先判断，不能发生下溢。reset 期间的 ready 输出另外受复位策略控制。

原占用 (XAU0,XAU1,XAU2)	当拍退休	arready[2:0]
(7,0,0)	无	110
(7,1,0)	无	100

原占用(XAU0,XAU1,XAU2)	当拍退休	arready[2:0]
(7,1,1)	无	000
(4,4,0)	无	100
(7,1,1)	XAU1 唯一一条	010
(7,1,1)	XAU0 一条	111

若某来源 occ_after[x]=0，保留不变量保证至少还有一个空槽；它接收第一条不增加 max(occ_after[x],1)，因此不依赖其他来源退休即可准入。

5 3 物理 entry 分配

```
free_mask = ~valid_q | retire_mask
alloc_valid = ar_fire
alloc_entry = select_one(free_mask)
```

可以用固定最低空闲编号选择器，分配顺序不参与年龄优先级。不存在两个 AR 同拍竞争分配的问题，因为总线每拍最多一条。

接收沿原子完成：置 valid、保存所有 AR 字段、清 issued/mem_done、初始化 OTB 状态、把新命令登记为最年轻。新命令只能从下一周期开始参与窗口判断。

6 接收年龄管理

本方案采用九项年龄矩阵，避免有限宽度时间戳溢出后比较新老的问题。

定义 older_set[i][j]=1 表示“有效 entry j 比有效 entry i 更老”。对角线恒为零；任意两个不同有效 entry 的先后关系唯一。

```
oldest(M)[i] = M[i] && !(|(older_set[i] & M))
```

其中 M 是待比较集合。例如 M 为窗口内可发集合时，返回最老可发命令；M 为某 XAU 的所有 valid 项时，返回其最老未退休命令。

6 1 更新规则

只在分配或退休改变年龄关系；发射、失败、MEM 完成不改变年龄。

在先求出 valid_next 后，对每对 i、j 使用下列优先级：

```
if (!valid_next[i] || !valid_next[j] || i == j)
    older_next[i][j] = 0;
else if (alloc_valid && i == alloc_entry)
    older_next[i][j] = 1;  // 所有其他存活项都比新 i 老
else if (alloc_valid && j == alloc_entry)
    older_next[i][j] = 0;  // 新 j 不可能比旧 i 老
else
    older_next[i][j] = older_q[i][j];
```

退休后无复用时，该项所在行列清零；同拍退休并复用同一槽时，新分配规则重建该行列，不能保留旧命令年龄。

九项矩阵只有 81 个 bit，易于实现与检查。若实现优化为三角矩阵，行为与比较方向应保持不变。

7 三拍前向流水

周期	阶段	工作
C1	S0 接收	AR 握手，配对分配，保存命令
C2	S1 判断	构建五项窗口，匹配 Bank 空闲，选最老合格项；拍末寄存
C3	S2 请求	输出 req/address，当拍获得两种 grant，拍末提交成功或失败

S0、S1、S2 针对不同命令并行运行。C3 可以同时接收 C、判断 B、请求 A。不能将三阶段做成等待前一命令完成后才处理下一条的串行状态机。

S1 没有候选则向 S2 注入空泡。S2 的每次尝试在该周期结束，不因失败而保留原请求级内容；下一周期请求级接收的是本周期 S1 的结果。

8 五项窗口和最老匹配选择

8 1 候选集合

```
pending_mask = valid_q & ~issued_q
current_issue_mask = issue_valid_q ? onehot(issue_entry_q) : 0
issue_success = |(req & ar_rd_gnt & rw_rd_gnt)
success_mask = issue_success ? current_issue_mask : 0


rank_pool = pending_mask & ~success_mask


older_count[i] = popcount(older_set[i] & rank_pool)
window_mask[i] = rank_pool[i] && (older_count[i] < 5)


bank_idle_mask[i] = !rd_mem_status[cmd_addr[i][6]][cmd_addr[i][5:3]]
candidate_mask = window_mask & bank_idle_mask & ~current_issue_mask
select_onehot = oldest(candidate_mask)
```

顺序必须是先决定最老五条，再筛 Bank 空闲。若先在全部九项中过滤空闲 Bank，会把窗口外命令错误引入候选。

所有 entry 的 Bank 状态可以并行读取。窗口 rank 最大为八，计数比较使用四位即可。若候选存在，年龄关系保证 oldest 返回唯一命令。

8 2 6B 成功当拍前移

当前请求级 A 成功时，success_mask 在本拍 S1 计算中将 A 排除。原第六条 F 可当拍进入窗口，并在下一拍发出请求。

```
N：请求 A 成功；B～E 忙；F 空闲 → S1 当拍选 F
N+1：请求 F
```

如果等 issued 寄存器下一拍更新后才前移，会多一个气泡，与所选 6B 不符。

8 3 失败时的窗口位置

当前请求级 A 失败时，A 仍留在 rank_pool 和最老五条窗口中，只在最后候选掩码中被屏蔽，避免同拍重复预选。

不能在构窗前无条件移除 current_issue_mask，否则 A 失败也会让 F 错误进入窗口。

8 4 五条全部阻塞

A ~ F 从老到新，A ~ E 对应 Bank 均忙、F 空闲且当前请求级为空时，candidate_mask=0。

rd_mem_status 若持续不变，窗口不会自动滑动，F 不发射。

这一等待是规格行为。每 XAU 容量保留只能让其他来源的命令进入 CMDBUF，不能授权窗口外命令绕过发射规则。

9 请求级和 5A 失败恢复

9 1 请求级寄存器

在每个周期结束时：

```
issue_valid_q <= select_valid
if (select_valid):
    issue_entry_q <= select_entry
    issue_bank_q  <= cmd_addr[select_entry][5:3]
    issue_addr_q  <= cmd_addr[select_entry][17:6]
```

下一周期输出：

```
req = issue_valid_q ? (8'b1 << issue_bank_q) : 8'b0
arb_addr = issue_addr_q
issue_success = issue_valid_q
             && ar_rd_gnt[issue_bank_q]
             && rw_rd_gnt[issue_bank_q]
```

req 必须来自请求级寄存器，不能由本拍 grant 决定是否产生，否则会引入仲裁组合环。无请求时的 arb_addr 可输出寄存值或零；本方案建议对外置零便于观察，此行为不是已确认功能要求。

9 2 5A 的恢复过程

周期	S1 判断	S2 请求
C2	A	空
C3	B	A 失败
C4	A	B
C5	后续命令	A 重试

A 失败不冲刷 B，也不在失败当拍重新选择 A 替换 B。A 的 valid 保留、issued 仍为零、年龄不变；下一拍自动恢复候选资格。

如果只有 A 一条命令，C3 失败后，C4 重判、C5 重试。这一重试间隔来自已经选择的流水行为。

9 3 请求拍 Bank 状态变化

已确认的成功判据始终是目标 Bank 的同拍双 grant。判断拍空闲、请求拍转忙时，是否额外撤销请求尚未独立对齐。

本方案建议锁定 S1 的选择，在 S2 正常输出请求，由双 grant 决定是否接受，避免加入第二套隐含选择规则。该建议列为待确认实现边界。

9 4 同步给 OTB 的信息

CMDBUF 在请求级同时向 OTB 提供 attempt_valid、entry_id 及 issue_success。最小功能信息是命令归属与成功提交；完整地址和元数据已由接收时保存，不必重复存入 OTB。

attempt_valid 便于观察请求尝试；只有 issue_success 才允许 OTB 产生返回标记。失败不能造成 OTB 完成计时、重复分配或提前清除。

10 固定四拍返回标记

下游成功接收后不能反压返回，因此使用四级 valid 加 entry_id 标记流水，按时钟无条件推进。

```
每个时钟沿：
    ret_valid_q[0] <= issue_success
  成功时 ret_entry_q[0] <= issue_entry_q


  对 k=1..3：
      ret_valid_q[k] <= ret_valid_q[k-1]
    前一级有效时 ret_entry_q[k] <= ret_entry_q[k-1]


  若本沿前的 ret_valid_q[3] 为 1：
      capture_entry = 本沿前的 ret_entry_q[3]
      otb_data[capture_entry] <= mem_rdata
      otb_pc_err[capture_entry] <= pc_err
      mem_done[capture_entry] <= 1
```

在 SystemVerilog 中用同一沿的非阻塞赋值读取旧末级即可。不要将 capture_valid 再额外寄存一拍，也不要在捕获时使用刚移入末级的下一状态。

时钟沿	A 标记和数据的动作
C3 末	A 成功，写入第 0 级
C4 末	A 移入第 1 级
C5 末	A 移入第 2 级
C6 末	A 移入第 3 级
C7 末	读取旧第 3 级，捕获 A 数据和 pc_err
C8 周期	A 的已寄存完成状态可参与 R 选择

空泡、仲裁失败写入无效标记；已有标记继续前进。AR 满、没有新请求、rready 全零均不能暂停该流水。

外部无返回事务标识，本方案依赖“每个成功命令恰好四拍后返回，且返回合并总线保持该对应关系”的接口约束。若下游将来改为可变延迟，必须增加可匹配的返回标签或改变协议，当前标记流水不能继续直接使用。

11 MEM 捕获和完成通知

捕获当拍将完整 128-bit 数据和 pc_err 写到标记指定的 OTB entry，并使该 entry 的 mem_done 置位。CMDBUF 使用同一完成状态或接收同拍完成通知。

捕获条件只由 ret_valid_q[3] 决定。数据全零、pc_err=0 仍是合法返回；不能以数据非零或错误有效代替捕获有效。

捕获时 entry 必须 valid、issued 且尚未 mem_done。它仍占用容量，后续 rready 不允许响应时，数据和错误保持直到退休。

当前 MEM 捕获不能组合旁路进入本拍 R 候选。R 只能读取本周期开始前已经寄存的 mem_done 和 OTB 数据，从而保证最早下一 cycle 返回。

12 每 XAU 保序及 RR 响应

12 1 每源队头

```
src_mask[x][i] = valid_q[i] && (cmd_src[i] == x)
head_onehot[x] = oldest(src_mask[x])
head_valid[x] = |head_onehot[x]
head_done[x] = mem_done[head_entry[x]]，仅在 head_valid 时读取


r_candidate[x] = head_valid[x] && head_done[x] && rready[x]
```

先找该 XAU 最老的未退休命令，再检查它是否完成。不能在该 XAU 的所有完成项里找最老，否则会跳过尚未完成的真正队头。

不同 arid 的同源请求也必须遵循同一接收序列。不同 XAU 不施加全局返回顺序：XAU0 的队头未完成时，XAU1 的合格队头仍可响应。

12 2 RR 实现建议

当前已确认的是在合格通道间 RR。为了给出可编码方案，建议 rr_ptr 表示扫描起点，按 0→1→2 循环，从起点扫描三个通道，选第一个 r_candidate=1 的通道。

```
winner = rr_first(r_candidate, rr_ptr_q)
rvalid = winner_valid ? onehot(winner) : 0
```

```
若 retire_fire: rr_ptr_next = (winner == 2) ? 0 : winner + 1
否则: rr_ptr_next = rr_ptr_q
```

图 08 使用上述方向和空闲保持；初始起点及精确推进策略尚未独立确认。本方案建议初始指针为 0，但不能把这一建议当作既定系统复位协议。

若三个通道持续有完成队头且持续 ready，上述 RR 方案会循环服务三个通道。它不为非 ready 或队头未完成的通道预留 R 总线。

12 3 共享负载选择

```
response_entry = head_entry[winner]
rdata      = otb_data[response_entry]
rid        = cmd_id[response_entry]
r_port_id = cmd_port_id[response_entry]
raddr      = cmd_addr[response_entry][2:0]
rresp      = error_encode(cmd_arresp[response_entry], otb_pc_err[response_entry])
```

winner 无效时建议将共享负载置零，便于波形和降低无意义翻转；协议检查只在对应 rvalid 有效时比较负载。

不能用实时 AR 字段或实时 MEM 数据构造 R 负载。R 被选中时对应 rready 已经为 1，因此该周期结束沿同时完成传输和退休。

13 错误处理

保存的 arresp	保存的 pc_err	rresp
00	0	00
00	1	01
11	0	11
11	1	11

地址越界优先于奇偶校验异常。arresp=11 的命令仍完整经历分配、窗口选择、请求、MEM 返回和 R 返回，不做本地立即完成。

```
if saved_arresp == 2'b11: rresp = 2'b11
else if saved_pc_err:    rresp = 2'b01
else:                    rresp = 2'b00
```

该编码逻辑仅对已定义的 arresp=00/11 给出协议承诺。错误响应的数据内容尚未独立约定；本方案建议仍输出捕获值，同时以 rresp 指示数据状态，最终需确认。

14 R 退休和 10B 同拍复用

R 成功传输产生唯一 retire_entry，同时清除旧命令在 CMDBUF、OTB 的有效归属、完成状态和年龄关系，并扣除所属 XAU 的占用。

本方案建议分配及退休时将 OTB data/pc_err 清零。功能正确性至少要求旧数据失效且不能暴露给新事务；若为功耗或 RAM 实现改为只清有效位，是否符合“清除”的最终要求需确认。

14 1 满表复用

free_mask 包含当拍 retire_mask。因此即使沿前九项均 valid，仍能选中本拍即将退休的槽位分配新请求。每来源额度也必须先扣除当拍退休再计算。

例如 (7,1,1) 中 XAU1 唯一命令退休，本拍只能对 XAU1 给出 ready=1；若退休的是 XAU0 的一条，则释放的是共享额度，三路均可具备准入资格。

14 2 同一槽位更新优先级

R 负载由沿前旧状态输出；时钟沿上同槽新分配优先于旧退休清除：

```
if reset_active:
  清控制状态
else if alloc_valid && alloc_entry == i:
    valid=1; issued=0; mem_done=0
  保存新 AR 元数据；清新 OTB 数据状态；重新登记最年轻年龄
else if retire_fire && retire_entry == i:
    valid=0; issued=0; mem_done=0
  清旧归属及对应存储
else:
  若成功发射命中 i，issued=1
```

若 MEM 捕获命中 i，保存数据和 pc_err，mem_done=1

这是每项的逻辑优先级，不代表将所有数组写成多个冲突的 always_ff。每个寄存器应有唯一写入过程；并行事件分别作用于不同 entry，不能用顶层全局 if/else 误删其他事件。

14 3 合法并发关系

同拍事件	可否同 entry	说明
R 退休与 AR 分配	可以	10B，输出旧负载后建立新归属
MEM 捕获与 R 退休	不可以	本拍捕获项此前尚未完成，不能本拍返回
成功发射与 R 退休	不可以	待发与已完成状态互斥
成功发射与 MEM 捕获	不可以	同一命令相隔四拍；出现同项通常是重复发射错误
AR 分配与旧返回标记命中	不可以	未捕获命令不能先退休，其槽位不能被重分配
上述事件作用于不同 entry	可以	可同时接收、成功发射、捕获、返回

正常协议下，旧命令退休时已不存在其未到达的返回标记，因此 entry 重用无需额外 generation tag。复位打断未完成事务是另一边界，见第 21 节。

15 计数及状态一致性

```
used_next = used_q + ar_fire - retire_fire
pending_next = pending_q + ar_fire - issue_success
occ_next[x] = occ_q[x] + accept[x] - retire[x]


sum(occ_q) == used_q == popcount(valid_q)
pending_q == popcount(valid_q & ~issued_q)
0 <= pending_q <= used_q <= 9
sum(max(occ_q[x],1)) <= 9
```

预选、失败、MEM 捕获不改变 pending_cnt。R 退休的命令早已 issued，退休本身也不再减少 pending_cnt。

6B 可以在本拍窗口计算中提前排除成功项，但 pending_cnt 对外仍表示本周期寄存状态，并在本周期末更新。这样不把下游 grant 通过计数绕回下游仲裁逻辑。

实现可以选择寄存计数或由位向量计算，但保留额度计算、对外指示和验证采样必须使用一致的周期定义。

16 周期内组合依赖

建议按以下依赖形成组合逻辑，并统一在周期结束沿提交：

```
当前命令/OTB/年龄状态 + rready
  → 各 XAU 队头和合格候选 → RR → R 负载/retire
  → 扣除 retire 后的占用 → 每来源 arready → AR 接收/分配

当前请求级寄存器
    → req/address → 下游两组 grant → issue_success
  → 成功前馈后的窗口 → Bank 匹配/最老选择 → 下一周期请求级

返回标记旧末级 + 当前 MEM 返回
  → 本沿捕获 → 下一周期可见的 mem_done/data
```

当前 R 选择不依赖新 AR，当前 req 不依赖本拍选择结果，当前 R 候选不依赖本拍 capture_next。上述隔离是保持三拍路径、禁止同拍 MEM 旁路及避免本模块内部组合环的关键。

17 性能与时序实现考虑

17 1 理想吞吐

当上游每拍供给、窗口中持续存在可发项、每次双 grant 成功、R 持续可返回时，模块可每拍接收一条、成功发射一条、捕获一条和返回一条。

单 XAU 最多七项仍能支撑这一理想吞吐：C1～C7 接收七条，第一条 C3 成功、C7 捕获、C8 返回；C8 同时接收第八条，随后每拍释放与接收相抵。这里必须对容量额度做当拍退休前馈，即使物理池中还剩另两路保留空槽也一样。

若没有 10B，单来源达到七项时会额外停一拍。若增加额外 R 流水级，或 MEM 延迟不再是四拍，则这一七项支撑一条每拍的结论也需重新评估。

17 2 关键组合路径

路径	关注点
grant → 成功前馈 → 窗口 → 最老选择 → 请求级寄存器	6B 明确要求这条路径在一拍内完成
mem_done/年龄 → 每源队头 → RR → 共享 128-bit mux	影响 R 返回周期
rready → RR/retire → 每源额度 → arready	10B 和容量保留引入的前馈路径
AR 握手 → 元数据写入及年龄更新	不能越过 S0 寄存直接影响本拍判断

九项年龄比较、Bank 空闲解码和每源队头可并行计算。可预先计算“当前请求成功/不成功”两套窗口，再由成功信号选择，以缩短 late grant 路径；必须证明与统一窗口公式等价。

为了频率随意再插一拍会改变 6B、三拍请求或最早 R 返回时序，不能在保持原规格的名义下增加延迟。目标频率、工艺、综合约束尚未给出，本方案不承诺具体 MHz。

17 3 存储量估算

按本文字段直接实现，未计组合逻辑及实现相关冗余：

项目	bit 数
命令元数据与 valid/issued/mem_done	9 × 38 = 342
OTB 数据及 pc_err	9 × 129 = 1161
年龄矩阵	81
请求级	20
返回标记流水	20
三路占用、总占用、待发数、RR 指针	22
合计	约 1646

这是寄存状态规模估算，不是综合面积结果。共享状态无需在两个模块中重复实现；是否复制、编码或优化由综合和模块边界决定。

18 典型时序

18 1 连续无阻塞

周期	AR 接收	S1 选中	S2 请求成功	MEM 捕获	R 返回
C1	A	—	—	—	—
C2	B	A	—	—	—
C3	C	B	A	—	—
C4	—	C	B	—	—
C5	—	—	C	—	—
C6	—	—	—	—	—
C7	—	—	—	A	—
C8	—	—	—	B	A
C9	—	—	—	C	B
C10	—	—	—	—	C

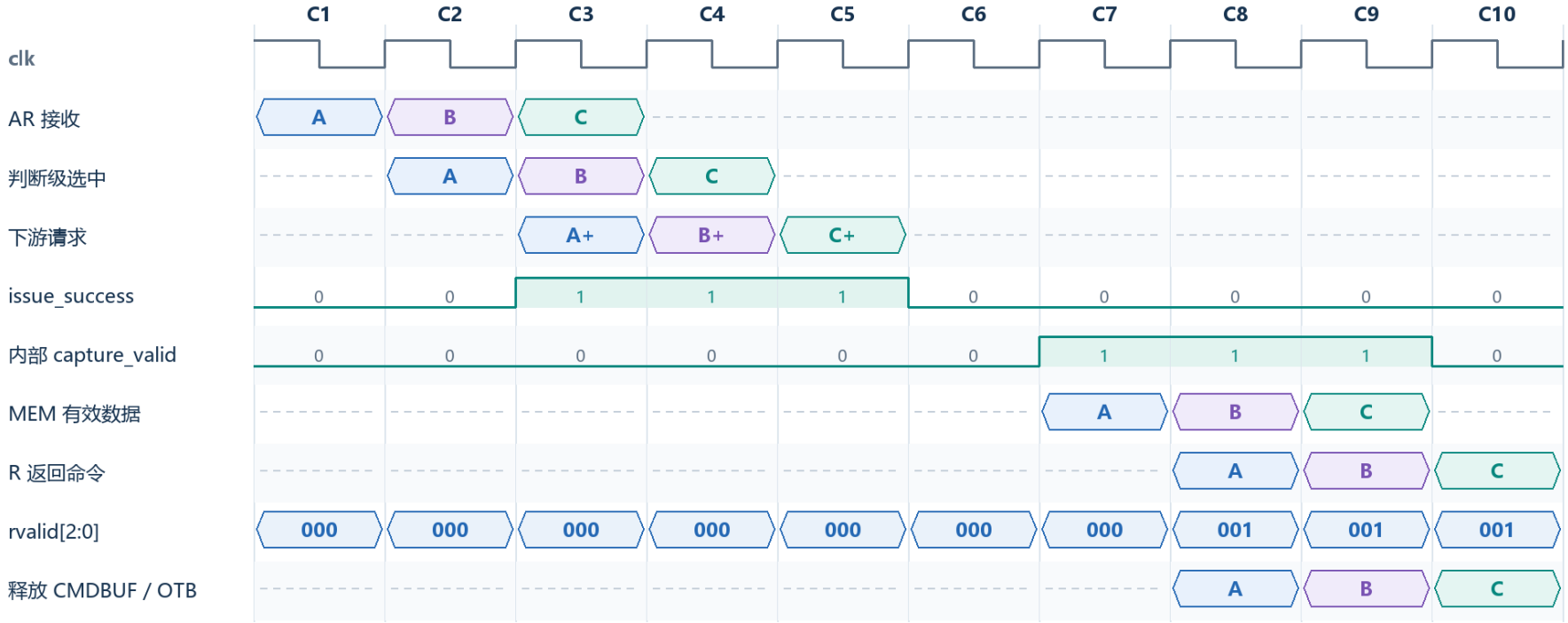
连续流水时序

01 连续正常流水

已确认 无旁路

每列一个周期，右边界上升沿提交；+ 成功，× 失败，虚线表示无对应有效数据。

A、B、C 均来自 XAU 0；仲裁成功；rready[0] 始终为 1



C3 同时接收 C、判断 B、请求 A。A 在 C3 发射成功、C7 捕获返回、C8 响应成功后释放。

已确认：MEM 返回当拍写 OTB，最早下一拍参与 R 仲裁；仍须满足源内保序、rready 和 RR。

连续流水时序

18 2 已对齐场景索引

图	核心行为
01 连续流水	三拍请求、固定四拍捕获、下一拍可返回
02 同 XAU 保序	后接收先完成的命令等待前序返回
03 窗口前移	第六条通过成功前馈进入窗口
04 双 grant 与空泡	失败没有返回标记
05 选择 5A	A 失败保留已选 B
06 选择 6B	成功当拍选入后续项
07 单拍返回	R 不 ready 时仍捕获并保持
08 RR 示例	只选择 ready 的完成队头；精确环方向见待定项
09 字段与错误	地址切片、字段透传、异常优先级
10 选择 10B	满表同拍退休和接收
11 窗口全阻塞	不因第六条可用而越窗发射
12 容量保留	一路占七项后其他来源仍可准入
13 保底复用	释放唯一项时仅恢复该来源资格

19 建议 RTL 划分

文件或逻辑块建议	主要内容
cmdbuf_rd.sv	顶层端口、公共事件、模块连接

文件或逻辑块建议	主要内容
cmdbuf_rd_cmd.sv	九项元数据、状态、年龄、窗口及请求级
cmdbuf_rd_otb.sv	九项数据、pc_err、固定四拍返回标记与捕获
cmdbuf_rd_alloc.sv	每来源占用、保留额度、free mask 与分配
cmdbuf_rd_rsp.sv	每来源队头、RR、共享 R mux 与退休

文件划分是建议，允许将 alloc/rsp 合并到顶层或 CMDBUF 控制中。核心是状态单一归属、每个寄存器唯一写入者，以及分配/完成/退休广播的一致性。

常量建议统一定义：XAU_NUM=3、ENTRY_NUM=9、WINDOW_SIZE=5、BANK_NUM=8、ADDR_WIDTH=18、DATA_WIDTH=128、ID_WIDTH=8、PORT_ID_WIDTH=5、MEM_LATENCY=4。参数化不代表任意组合均已验证；首先固定这一基线实现。

20 验证和关键检查

20 1 基准模型

每次合法 AR 握手分配独立测试序号 accept_seq，保存所有输入字段。为每个 XAU 建接收队列，并维护全局占用、待发集合、成功发射四拍日历以及完成记录。

参考模型按本方案窗口规则选下一拍请求，不使用 arid 数值决定新老。MEM 在对应日历拍提供有效数据；无效拍可以改变总线，暴露晚采样和实时总线误用。

R 只从此前已完成的各 XAU 队头选择。检查完整负载属于同一命令；错误数据值尚未确定时，错误子例优先核对状态与归属，正常子例逐位比较 128-bit 数据。

20 2 安全性检查

以下是断言意图，实际 SVA 需绑定 RTL 信号及采样语义：

- req 与 rvalid 均 onehot0；rvalid 不得指向当拍 rready=0 的通道。
- ar_fire 必须命中合法 src 的 ready；一次 AR 最多分配一个配对 entry。

- mem_done 必须蕴含 valid 和 issued；issued 不得在 grant 失败时置位。
- 每条命令最多成功发射一次、MEM 捕获一次、R 返回一次。
- 捕获 entry 必须有效、已成功发射且未完成；同条命令本拍捕获时不能本拍退休。
- 每次成功发射恰好在四拍后对应一次捕获，失败不得生成返回标记。
- 预选命令必须属于成功前馈后的最老五条集合，目标 half 空闲，且不是当前请求级项。
- 候选非空时，选中者不存在同集合内更老命令；候选为空则不产生选择有效。
- 同 XAU 响应顺序与实际 AR 握手顺序一致；所有元数据和数据归属一致。
- valid、计数、每源占用和年龄结构一致，保留容量不变量一直保持。
- 同拍复用输出旧命令，新命令在下一状态有效、最年轻、未发射、未完成。

20 3 吞吐检查

5A、6B、10B 需要检查精确相邻周期行为，不能只检查最终数据正确。额外空泡也可能是规格偏离。

重点覆盖单 XAU 达七项后的持续接收、全表九项的跨源复用、失败时保留 B、成功当拍引入 F、同时接收/发射/捕获/响应四事件。

20 4 进展条件

无条件安全性检查与有环境前提的活性检查分开。rd_mem_status 持续阻塞、上游公平性未定、rready 永久为零或仲裁持续失败时，不要求每条命令最终完成。

现有 49 组注错清单 可作为验证覆盖来源。容量准入模型已对 190 个合法占用状态及 1716 个接收状态转换进行穷举检查；这是抽象容量模型结果，不是完整 RTL 形式验证或时序收敛结果。

21 复位与异常边界

复位建议将 valid、issued、mem_done、请求级 valid、四拍返回 valid、年龄矩阵、计数和 RR 控制清零。复位期间 req/rvalid/aready 建议全零，具体极性和生效方式待系统确认。

数据阵列可在无效状态下不做复位，由分配和退休清零；外部负载在无 valid 时按实现建议置零。若要求所有存储 bit 复位为零，可调整，但会影响复位网络负担。

运行中复位不能只清本地标记而默认下游旧返回自动消失。需要系统明确同步冲刷 MEM，或停发并排空旧事务等协议，才能定义何时恢复接收。当前方案不自行选定恢复流程。

项目	已知行为	本方案建议或仍待确定内容
容量隔离	不允许 XAU 独占导致其他来源没有 entry	已纳入每源一项保底、六项共享的实现方案
复位	尚无完整协议	极性、同步方式、初始 RR 和未完成返回处置待确认
非法输入	src 0～2、arresp 00/11 有定义	src=3、arresp=01/10 的恢复行为未定义
重复 ID	arid 为透传标签	本实现能按内部归属区分；系统是否另有限制待说明
数据清除	退休后旧归属和数据不能再次有效	建议分配及退休物理清零；是否必须逐 bit 清零待确认
错误响应数据	rresp 优先级已确认	建议仍输出捕获值，最终数据语义待确认
RR 细节	合格通道间 RR	建议 0→1→2、服务后推进、空闲保持，精确规则待确认
Bank 状态变化	双 grant 决定最终成功	建议请求级不二次撤销，是否允许撤销待确认
无效周期负载	无有效请求或响应时不作功能比较	建议输出零，尚无强制值约定
上游公平性	已确认可切换 src	公平性未定，不建立无条件最终接收保证

22 实施交付边界

本方案完成了接口、状态、选择算法、返回匹配、容量准入、并发更新优先级和验证判据的设计定义。可先据此搭建 RTL 主体与参考模型；将第 21 节中会影响外部行为的选择记录为可追踪配置，最终冻结前按系统要求确定。

现有时序图是规格示意，容量检查是独立抽象模型。尚未交付可综合 RTL、运行完整仿真、综合或静态时序分析，不能据此宣称目标频率、面积或全部验证已通过。

文档来源与覆盖范围

项目说明源文件共 22 份 Markdown，另有 5 份对应 HTML 阅读页。逐字相同的检测点目录、注错清单和容量说明各导出一次，下表记录其归属，避免重复占用篇幅。UCAgent 工具自带的其他上游文档作为第三方来源保留。

README.md → 项目使用与部署说明

docs/GitHub 使用说明.md → 项目使用与部署说明

docs/design/cmdbuf_rd_design.md → CMDBUF RD 详细设计方案

docs/fault_catalog/capacity_policy.md → CMDBUF RD 注错与容量保护清单

docs/fault_catalog/cmdbuf_rd_fault_catalog.md → CMDBUF RD 注错与容量保护清单

docs/gui/README.md → UCAgent Tk GUI 详细操作说明

formal/cmdbuf_rd_dual_backend/README.md → CMDBUF RD Formal 验证说明

formal/cmdbuf_rd_dual_backend/spec_catalog.md → CMDBUF RD Formal 验证说明

gui/README.md → 项目使用与部署说明

provenance/README.md → 项目使用与部署说明

rtl/README.md → 项目使用与部署说明

simulation/cmdbuf_rd/README.md → CMDBUF RD 仿真基线与历史记录

simulation/cmdbuf_rd/docs/capacity_policy.md → CMDBUF RD 注错与容量保护清单 与规范源字节相同

simulation/cmdbuf_rd/docs/design_spec.md → CMDBUF RD 仿真基线与历史记录

simulation/cmdbuf_rd/docs/fault_catalog.md → CMDBUF RD 注错与容量保护清单 与规范源字节相同

simulation/cmdbuf_rd/docs/mutation_plan.md → CMDBUF RD 仿真基线与历史记录

simulation/cmdbuf_rd/docs/verification_baseline.md → CMDBUF RD 仿真基线与历史记录

simulation/cmdbuf_rd/docs/verification_report.md → CMDBUF RD 仿真基线与历史记录

ucagent-tool/docs/content/02_usage/13_portable_formal_gui.md → 项目使用与部署说明

verification/closure/spec_catalog.md → CMDBUF RD Formal 验证说明 与规范源字节相同

verification/closure/verification_summary.md → CMDBUF RD Formal 验证说明

verification/verilog_20261012/README.md → CMDBUF RD Formal 验证说明

架构图、13 张时序图、真实 Tk 截图和历史对齐截图均来自仓库内已有文件。原图未重新生成，内容源 SHA256 与导出文件 SHA256 记录于 documents/document_manifest.json。

新增说明继续以 PDF 或 Word 交付。RTL、脚本、断言、JSON 结果及签名证据属于工程输入或机器记录，不改成办公文档，也不改写原始验收时间。